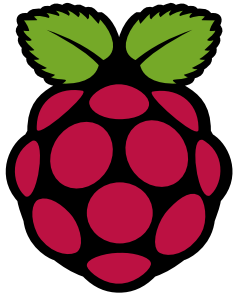


Pithon

Computer Controlled IO
Based on Raspberry Pi

Fang Yuan
(yfang@nju.edu.cn)

July 2026 in Nanjing University



Pithon

基于Raspberry Pi的计算机控制输入/输出系统

方元
(yfang@nju.edu.cn)

2026年7月，南京大学

Content

1	The Linux Operating System	2
1.1	Overview of Operating Systems	2
1.2	Introduction to Linux	3
1.3	Using Linux	8
1.3.1	The Command-Line Format	8
1.3.2	Users and Permissions	9
1.3.3	Directory Operations	11
1.3.4	File Operations	12
1.3.5	Process Control	16
1.3.6	Modifying Process States	17
1.3.7	Foreground and Background Execution	17
1.3.8	I/O Redirection and Pipelines	19
1.4	Text Editors	20
1.5	Network Connectivity	22
1.5.1	Remote Terminal Access	22
1.5.2	Remote Desktop Access	24
1.5.3	Remote Network Storage (NFS)	26
1.6	Chapter Summary	26
2	Very Short Python Introduction	28
2.1	Introduction	28
2.2	Python Environment	29
2.3	Python Basics	30
2.3.1	Running Your First Python Program	30
2.3.2	Interactive Mode	31
2.3.3	Numerical Variables	32
2.3.4	Strings	34
2.3.5	Boolean Logic	36

内容

1	Linux操作系统	2
1.1	操作系统概述.	2
1.2	Linux简介.	3
1.3	使用 Linux.	8
1.3.1	命令行格式.	8
1.3.2	用户与权限.	9
1.3.3	目录操作.	11
1.3.4	文件操作.	12
1.3.5	过程控制.	16
1.3.6	修改流程状态.	17
1.3.7	前台与后台执行.	17
1.3.8	输入/输出重定向与管道.	19
1.4	文本编辑器.	20
1.5	网络连接.	22
1.5.1	远程终端访问.	22
1.5.2	远程桌面访问.	24
1.5.3	远程网络存储 (NFS)	26
1.6	章节摘要.	26
2	《极简版Python入门指南》	28
2.1	引言.	28
2.2	Python环境.	29
2.3	Python基础.	30
2.3.1	运行您的第一个 Python 程序.	30
2.3.2	交互模式.	31
2.3.3	数值变量.	32
2.3.4	字符串.	34
2.3.5	布尔逻辑.	36

2.3.6	Functions	37
2.3.7	Order of Operations	38
2.4	Data Structures	41
2.4.1	Lists	41
2.4.2	Tuples	42
2.4.3	Sets	43
2.4.4	Dictionaries	44
2.5	Python Program Architecture	44
2.5.1	Program Structure	44
2.5.2	Conditional Execution	45
2.5.3	Loops and Iterations	46
2.5.4	Functions and Scope	48
2.5.5	Modules and Package Management	49
2.6	Summary	50
3	I/O Interfaces on Raspberry Pi	52
3.1	Raspberry Pi	52
3.1.1	Short Introduction	52
3.2	Low-Level Hardware Expansion Interfacing	53
3.3	I/O System Usages	55
3.4	GPIO Modules for Python	56
3.4.1	Install a GPIO Module	56
3.4.2	Pin Numbering	57
3.4.3	Set up Pins	58
3.4.4	Use GPIO as Input	59
3.4.5	Untrasonic Ranging	62
3.5	Using PWM	63
3.5.1	What is a PWM	63
3.5.2	Accessing PWM in Userspace	63
3.5.3	PWM Controlling using RPi.GPIO	64
3.6	I2C Module	65
3.6.1	Single Segments Display	65
3.6.2	Multiple Display	65
3.7	Addressable LED Strip WS2812	70

2.3.6	功能.	37
2.3.7	作战顺序.	38
2.4	数据结构.	41
2.4.1	列表.	41
2.4.2	元组.	42
2.4.3	集合.	43
2.4.4	词典.	44
2.5	Python程序架构.	44
2.5.1	程序结构.	44
2.5.2	条件执行.	45
2.5.3	循环与迭代.	46
2.5.4	功能与范围.	48
2.5.5	模块与包管理.	49
2.6	摘要.	50
3	树莓派上的输入/输出接口	52
3.1	树莓派.	52
3.1.1	简要介绍.	52
3.2	低级硬件扩展接口.	53
3.3	I/O系统的使用方式.	55
3.4	Python 的 GPIO 模块.	56
3.4.1	安装 GPIO 模块.	56
3.4.2	密码编号.	57
3.4.3	设置图钉.	58
3.4.4	使用 GPIO 作为输入.	59
3.4.5	非超声测距.	62
3.5	使用 PWM.	63
3.5.1	什么是 PWM ?	63
3.5.2	在用户空间中访问 PWM.	63
3.5.3	PWM 使用RPi进行控制。GPIO.	64
3.6	I2C模块.	65
3.6.1	单段显示.	65
3.6.2	多显示器.	65
3.7	可寻址LED灯带 WS2812.	70

List of Figures

1.1	Selected Linux Distributions	6
1.2	OS Market Share Worldwide	7
1.3	VIM Mode Transitions	20
1.4	XFCE4 Graphical Desktop Environment Session on a Raspberry Pi . . .	25
3.1	Different Models of Raspberry Pi	52
3.2	Alternative Single-Board Computer Configurations	53
3.3	Connecting an LED and a Switch to GPIOs	56
3.4	Ultrasonic Ranging	62
3.5	7-Segment LED Display	65
3.6	Generalized I2C Connection Diagram	66
3.7	Standard I2C Bus Signaling and Timing Constraints	66
3.8	WS2812 Addressable LED Strip	71

图列表

- 1.1 精选的 Linux 发行版. 6
- 1.2 全球操作系统市场份额. 7
- 1.3 VIM模式切换. 20
- 1.4 XFCE4 图形桌面环境在树莓派上的运行. 25

- 3.1 不同型号的树莓派. 52
- 3.2 替代性单板计算机配置方案. 53
- 3.3 将 LED 和开关连接到 GPIO 端口. 56
- 3.4 超声测距. 62
- 3.5 7段LED显示屏. 65
- 3.6 通用 I2C 连接图. 66
- 3.7 标准 I2C 总线信号传输及时序约束. 66
- 3.8 WS2812可寻址LED灯带. 71

List of Tables

1.1	Linux File Type Identifiers	10
1.2	Common File Management Wildcards	13
1.3	Common ‘tar’ Command Options	15
1.4	Standard Linux System Signals	18
1.5	Standard Shell I/O Redirection Operators	19
1.6	Summary of Core Linux Commands	27
2.1	String Format Specifiers in Python	34
2.2	Operator Precedence Hierarchy in Python Expressions	39
2.3	Bitwise Operators and Expressions	39
2.4	Compound Assignment Statements	40
2.5	Reserved Keywords in Python Programming	51
2.6	Globally Available Built-in Interpreter Functions	51
3.1	Physical GPIO Pin Mapping of the Raspberry Pi	54
3.2	Supported GPIO Pins on Raspberrypi	71

表格列表

1.1	Linux文件类型标识符.	10
1.2	常用文件管理通配符.	13
1.3	常见的 <code>tar</code> 命令选项.	15
1.4	标准Linux系统信号.	18
1.5	标准 Shell I/O 重定向操作符.	19
1.6	核心 Linux 命令概览.	27
2.1	Python 中的字符串格式指定符.	34
2.2	Python表达式中的运算符优先级层次结构.	39
2.3	位运算符与表达式.	39
2.4	复合赋值语句.	40
2.5	Python编程中的保留关键字.	51
2.6	全球通用的内置解释器函数.	51
3.1	树莓派的物理 GPIO 引脚映射.	54
3.2	Raspberrypi 上支持的 GPIO 钉.	71

Chapter 1

The Linux Operating System

1.1 Overview of Operating Systems

An operating system (OS) is the core software that manages computer hardware and software resources. Positioned between the computer hardware and the user, it serves as a bridge for communication between the two. Users input commands through the OS user interface, and the operating system interprets these commands to drive hardware devices, thereby fulfilling user requests.

The most common operating systems we use on personal computers are Windows and Linux. Apple personal computers (the Mac series) run macOS, which is a branch of the UNIX operating system family. Smartphones and tablets also require operating systems. Except for Apple's iPhones and iPads, which run iOS (another UNIX derivative), almost all other smartphones and tablets currently on the market run Android. Crucially, the underlying kernel of the Android system is Linux.

A modern operating system typically consists of several subsystems: process management, memory management, file systems, device drivers, and network subsystems. Because a general-purpose operating system lacks a specific, single application requirement, it must support a broader range of hardware and software to accommodate diverse use cases. It allocates system resources dynamically based on the overall user experience. Consequently, general-purpose computer operating systems are usually **time-sharing operating systems**, which prioritize fairness and rationality in resource allocation over strict timing constraints. In contrast, **embedded operating systems** are typically bound by stringent real-time requirements.

第 1 章

Linux操作系统

1.1 操作系统概述

操作系统（OS）是管理计算机硬件与软件资源的核心软件。它位于计算机硬件与用户之间，充当两者之间的通信桥梁。用户通过操作系统界面输入命令，操作系统则解析这些命令以驱动硬件设备，从而满足用户的操作需求。

个人电脑上最常用的操作系统是Windows和Linux。苹果个人电脑（Mac系列）运行macOS，这是UNIX操作系统家族的一个分支；智能手机和平板电脑同样需要特定的操作系统。除运行iOS（另一种UNIX衍生系统）的iPhone和iPad外，目前市场上绝大多数智能手机和平板电脑均采用Android系统。值得注意的是，Android系统的底层内核正是基于Linux开发的。

现代操作系统通常由多个子系统组成：进程管理、内存管理、文件系统、设备驱动程序以及网络子系统。由于通用操作系统并不针对特定的应用场景设计，因此必须支持广泛的硬件和软件环境以适应多样化的使用需求。它会根据整体用户体验动态分配系统资源。因此，通用计算机操作系统通常采用**分时操作系统**架构，其在资源分配中更注重公平性和合理性而非严格的时序约束；相比之下，**嵌入式操作系统**则通常需满足严苛的实时性要求。

1.2 Introduction to Linux

Linux is an operating system that adheres to the GNU General Public License (GPL). In April 1991, Linus Torvalds, then a graduate student at the University of Helsinki in Finland, began writing this operating system out of personal interest while studying an operating system course, subsequently releasing it to the Internet.



Linus Torvalds

Once published, the system sparked an immediate and enthusiastic response from programmers worldwide. Since then, its development has progressed unstoppably, turning Linux into a highly influential operating system today. Thousands of programmers have contributed code to Linux. The list of Linux kernel maintainers alone spans over four thousand lines, to say nothing of the massive community of application developers.

The primary characteristics and advantages of the Linux system can be summarized as follows:

1. Open Source Code

The Linux kernel follows an open-source license, and the vast majority of software bundles used across various distributions are open-source. Open-source software not only facilitates convenient development but, more importantly, provides robust security guarantees. Under open inspection, it is exceptionally difficult for malicious code to hide. Software engineers and Linux users worldwide actively contribute suggestions and patches to the open-source community, ensuring that bugs are resolved rapidly once discovered. As a result, the stable and reliable Linux operating system has become the premier choice for the vast majority of enterprise servers.

2. Reliability and Stability

Linux operates in protected mode, enforcing a strict separation between the kernel space and user space memory addresses. It implements rigorous read and write permission controls for different users, protected subsystems, and core authorization structures. These comprehensive security measures, paired with good administrative habits, allow a Linux system to run continuously and stably for long periods without rebooting.

3. Excellent Portability

1.2 Linux简介

Linux是一款遵循GNU通用公共许可证（GPL）的操作系统。1991年4月，时任芬兰赫尔辛基大学研究生的林纳斯·托瓦兹在学习操作系统课程期间出于个人兴趣开始开发该操作系统，并随后将其发布至互联网。

该系统一经发布，便立即在全球程序员中引发热烈反响。此后其开发进程势不可挡，使Linux发展成为当今极具影响力的操作系统。数千个



林纳斯·托瓦兹

程序员们已为Linux贡献了大量代码。仅Linux内核维护者的代码量就超过四千行，更不用说庞大的应用程序开发者社区了。

Linux系统的主要特性与优势可归纳如下：

1. 开源代码

Linux内核采用开源许可证协议，各类发行版中使用的绝大多数软件包也均为开源软件。开源软件不仅便于开发流程，更重要的是能提供可靠的安全保障。在公开透明的审查机制下，恶意代码几乎难以隐藏。全球范围内的软件工程师和Linux用户都会积极向开源社区提交改进建议和修复补丁，确保漏洞一经发现就能迅速得到解决。正因如此，稳定可靠的Linux操作系统已成为绝大多数企业服务器的首选方案。

2. 可靠性与稳定性

Linux采用受保护模式运行，严格区分内核空间与用户空间的内存地址。系统为不同用户、受保护的子系统以及核心授权机制实施了严格的读写权限控制。这些全面的安全措施结合良好的管理实践，使得Linux系统能够长时间稳定运行而无需重启。

3. 出色的便携性

Linux software development strictly adheres to the **POSIX** (Portable Operating System Interface) standard, meaning applications can be migrated across different platforms with little to no modification. POSIX is a collective term for a series of interrelated standards established by the IEEE and ISO/IEC. During the mid-1980s, various competing distributions emerged throughout the evolution of UNIX. To maintain compatibility across computer systems, POSIX defined standard application programming interfaces (APIs), command-line shells, and utility interfaces based on UNIX historical practices. This ensures portability at the source-code level. Today, the Linux kernel supports dozens of processor architectures (including Intel x86, ARM, and PowerPC) and hundreds of hardware platforms, making it highly portable to almost all mainstream processor environments.

4. Device Independence

Linux famously adheres to the philosophy that **“everything is a file.”** All physical hardware devices are abstracted and treated uniformly as files. The kernel provides a standardized interface call system for each device. Consequently, applications can manipulate and utilize these hardware components exactly as they would open, read, or write to a regular file, without needing to understand the underlying hardware complexities.

The core of device independence lies in the adaptability of the kernel. This provides a major advantage: user programs remain entirely decoupled from physical hardware configurations. When peripheral hardware is upgraded or swapped, the application code requires no modification, significantly enhancing peripheral allocation flexibility and hardware resource efficiency.

5. Multiple User Interfaces

From its early days as a minimalist text-only interface, Linux has evolved to offer both rich graphical user interfaces (GUIs) and powerful command-line interfaces (CLIs) side by side, allowing it to adapt to varying system resource constraints. The traditional command-line interface leverages the robust scripting capabilities of the shell, making it fully capable of handling server workloads even on computers completely lacking graphical hardware. Meanwhile, desktop Linux environments offer numerous GUI options (such as GNOME or XFCE), allowing users to fully customize their personal workspaces.

Note that the command-line interface is a powerful choice, not an inherent

Linux软件开发严格遵循**POSIX**（可移植操作系统接口）标准，这意味着应用程序几乎无需修改即可跨不同平台运行。**POSIX**是由IEEE和ISO/IEC制定的一系列相互关联的标准的统称。在20世纪80年代中期，随着UNIX系统的演进，出现了多种竞争性发行版。为确保跨计算机系统的兼容性，**POSIX**基于UNIX的历史实践定义了标准的应用程序编程接口（API）、命令行外壳及工具接口，从而实现了源代码层面的可移植性。如今，Linux内核支持数十种处理器架构（包括Intel x86、ARM和PowerPC）以及数百种硬件平台，使其几乎能适配所有主流处理器环境。

4. 设备独立性

Linux系统始终秉持“万物皆为文件”的核心理念。所有物理硬件设备均被抽象化，并统一作为文件进行处理。内核为每种设备提供了标准化的接口调用机制。因此，应用程序可以像操作普通文件一样对这些硬件组件进行读写操作，而无需了解底层硬件的复杂架构。

设备独立性的核心在于内核的可适应性。这带来了显著优势：用户程序完全不受物理硬件配置的影响。当外设硬件升级或更换时，应用程序代码无需修改，从而大幅提升了外设资源分配的灵活性和硬件资源利用效率。

5. 多用户界面

Linux从最初仅采用极简文本界面起步，现已发展出同时提供丰富的图形用户界面（GUI）和强大的命令行界面（CLI），从而能够适应不同的系统资源限制。传统的命令行界面充分利用了shell强大的脚本编写功能，即使在完全缺乏图形硬件的计算机上也能高效处理服务器工作负载；而桌面版Linux环境则提供多种图形界面选项（如GNOME或XFCE），让用户能够完全自定义个人工作空间。

请注意，命令行界面是一个强大的选择，而非固有特性。

limitation of Linux. For experienced users, it provides a far more efficient workflow. For consumer use cases, modern Linux desktop graphical environments are fully capable of meeting regular desktop computing needs.

6. Multi-User and Multi-Tasking Support

UNIX was originally designed from the ground up to accommodate multiple concurrent users on a single computer system. Linux inherited this architecture natively. Beyond allocating isolated system resources for individual accounts like standard personal computer operating systems, Linux can provide distinct, simultaneous terminal environments for different users. This allows multiple people to work on the exact same physical computer concurrently.

This behavior differs fundamentally from operating systems designed strictly for personal single-user workstations. While network advancements allow multiple users to log into a single server over a network today, doing so practically means an individual is utilizing his or her own computer, which differs conceptually from the native multi-user terminal design of Linux.

Multi-tasking is a hallmark of modern operating systems, referring to a computer's ability to execute multiple programs concurrently in an mutually isolated manner. The Linux scheduler independently allocates CPU resources and memory spaces to each task, ensuring they do not interfere with one another. The failure or crash of a single task generally does not impact other running processes, providing a fundamental guarantee for overall system reliability.

7. Comprehensive Networking Capabilities

As a direct descendant of the UNIX architectural lineage, Linux possesses all the technological network advantages of UNIX systems. UNIX was the first operating system to implement the TCP/IP suite natively. Linux completely inherited this network protocol stack, providing full TCP/IP client and server capabilities alongside powerful network communication facilities. Deploying standard network services on top of a Linux OS is exceptionally straightforward.

8. Extensive File System Support

Linux utilizes a **Virtual File System** (VFS) layer to abstract and support widely differing file systems. It can natively recognize almost all known disk partition formats, allowing software to read and write data across different partitions seamlessly without modifications to the application code.

这是Linux的一个局限性。对于经验丰富的用户而言，它能提供更为高效的工作流程；而对于普通用户的使用场景，现代Linux桌面图形环境完全能够满足日常办公计算需求。

6. 支持多用户和多任务处理

UNIX最初就是从零开始设计的，旨在支持单台计算机系统上同时运行多个用户。Linux天然继承了这一架构。与标准个人计算机操作系统不同，Linux不仅能为每个用户分配独立的系统资源，还能为不同用户提供独立且同步的终端环境，从而实现多人在同一台物理计算机上同时工作的功能。

这种行为与专为个人单用户工作站设计的操作系统存在根本差异。尽管网络技术的进步如今已支持多个用户通过网络登录同一台服务器，但实际操作中这相当于每个用户都在使用自己的计算机，这在概念上与Linux原生的多用户终端设计截然不同。

多任务处理是现代操作系统的核心特性，指计算机能够以相互隔离的方式同时运行多个程序。Linux调度器会独立为每个任务分配CPU资源和内存空间，确保各任务互不干扰。即使某个任务发生故障或崩溃，通常也不会影响其他正在运行的进程，从而为系统的整体可靠性提供了根本保障。

7. 全面的网络功能

作为UNIX架构体系的直接继承者，Linux具备UNIX系统所有的网络技术优势。UNIX是首个原生支持TCP/IP协议套件的操作系统；Linux完全继承了这一网络协议栈，不仅提供完整的TCP/IP客户端与服务器功能，还具备强大的网络通信能力。在Linux操作系统上部署标准网络服务极为简便。

8. 广泛的文件系统支持

Linux采用**虚拟文件系统**（VFS）层来抽象并支持多种不同的文件系统。该系统能够原生识别几乎所有已知的磁盘分区格式，使软件无需修改应用程序代码即可无缝读写不同分区中的数据。

9. Convenient Development and Maintenance Facilities

Linux distributions bundle a massive array of programming language development tools and rely heavily on scripting languages. This script-centric design not only streamlines software development but forms the bedrock of system maintainability. Most Linux background services are managed via scripts, and server parameters are configured through plain-text configuration files. Even the core system initialization and boot flows are orchestrated by scripts. This enables system administrators to manage the entire machine using any text editor they prefer.

Today, the Linux operating system spans every scale of computing hardware—from tiny wearable items and card-sized IoT devices up to massive supercomputers and distributed enterprise clusters. Outside of the mainstream personal desktop market, Linux maintains a dominant monopoly in almost every computing domain it occupies.

The Linux community is incredibly active. Historically, a new kernel version was released roughly every six months; recently, development cycles have become even faster. Kernel version 7.1.3 was released on July 4th 2026. A Long-Term Support (LTS) 6.18.38 is updated also on July 4th 2026. Standard LTS kernels generally receive a two-year maintenance window, but the LTS kernel (6.18) will be officially maintained through December 2028.



Figure 1.1: Selected Linux Distributions

Linux is categorized as **free software**, emphasizing **freedom** (as in “free speech”) rather than price (as in “free beer”). Because the English word “free” carries both meanings, it is frequently misinterpreted as merely zero-cost software. To distinguish the two concepts, pure zero-cost proprietary software is referred to as **freeware**. Due to

9. 便捷的开发与维护设施

Linux发行版集成了丰富的编程语言开发工具，并高度依赖脚本语言。这种以脚本为核心的设计不仅简化了软件开发流程，更为系统可维护性奠定了基础：大多数Linux后台服务均通过脚本管理，服务器参数则通过纯文本配置文件进行设置；甚至连核心系统的初始化和启动流程也由脚本统一协调。这使得系统管理员能够使用任意文本编辑器来管理整台计算机。

如今，Linux操作系统已覆盖所有规模的计算硬件——从微型可穿戴设备、卡片大小的物联网设备，到大型超级计算机和分布式企业集群。在主流个人桌面市场之外，Linux在其涉足的几乎所有计算领域均保持着主导地位。

Linux社区的活跃度极高。历史上，新的内核版本大约每六个月发布一次；近年来，开发周期更是显著加快。内核版本7.1.3于2026年7月4日发布，长期支持版（LTS）6.18.38同样在同年7月4日更新。标准LTS内核通常享有两年的维护期，而LTS内核（6.18）则将正式获得持续维护至2028年12月。



图 1.1: 精选的 Linux 发行版

Linux被归类为**自由软件**，其核心强调的是**自由性**（如同“言论自由”中的含义），而非价格（如同“免费啤酒”中的含义）。由于英文单词“free”同时包含这两种含义，它常被误认为仅指零成本软件。为区分这两个概念，纯粹零成本的专有软件被称为**免费软件**。由于

the terms of the GPL license, nearly all software components within a Linux distribution provide open source-code access free of charge. Many open-source licenses explicitly permit users to modify and redistribute the code. Consequently, provided the license terms are met, anyone can compile and assemble these software packages into their own custom operating system, which has given rise to a vast ecosystem of diverse Linux distributions (Figure 1.1).

Since its inception, Linux has rapidly captured the global server market due to its structural stability. Today, 100% of the top 500 fastest supercomputers in the world run Linux.¹

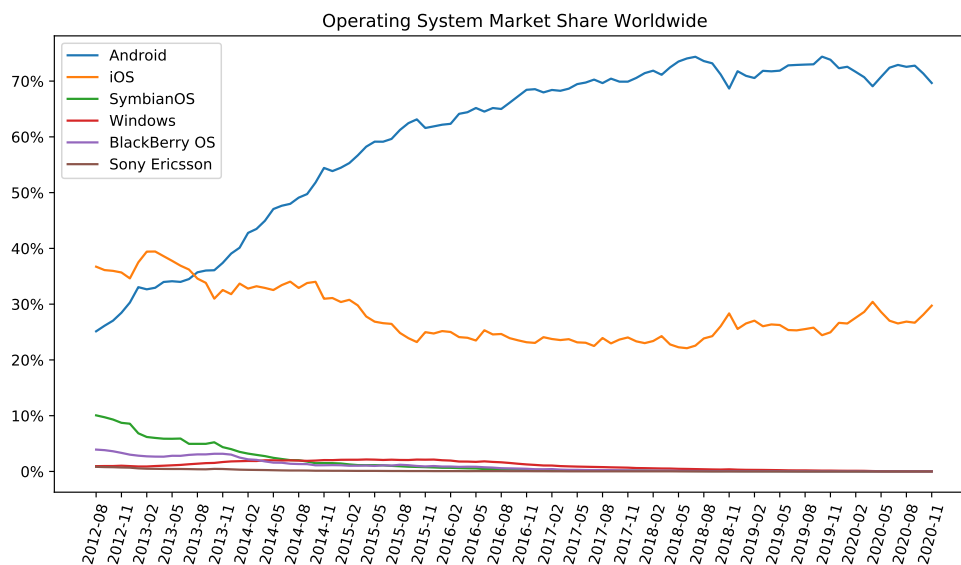


Figure 1.2: Operating System Market Share on Smartphones and Tablets²

Linux’s greatest success in the consumer market is perhaps the mobile device market, with Android being one of the most dominant operating systems on smartphones and very popular on tablets (Figure 1.2) and, more recently, on wearables. (Note: Due to distinct licensing structures and specific technical implementation variations, the Android OS is typically categorized separately from standard Linux distributions). Furthermore, because the Linux kernel natively supports dozens of processor architectures, it serves as the primary choice for single-board computers like the Raspberry Pi.

¹According to data published by <https://www.top500.org> profiling the world’s top 500 supercomputers, 498 systems ran Linux in 2017. From June 2018 onward, every single supercomputer on the Top500 list runs on a Linux-based operating system.

²Data source: <https://statcounter.com>

根据GPL许可证的条款，Linux发行版中的几乎所有软件组件均提供免费的开源代码访问权限。许多开源许可证明明确允许用户修改和重新分发代码。因此，在满足许可证条款的前提下，任何人都可以编译并整合这些软件包来构建自己的定制操作系统，从而催生了庞大的多样化Linux发行版生态系统（图1.1）。

自诞生以来，Linux凭借其结构稳定性迅速占领了全球服务器市场。如今，全球速度最快的500台超级计算机中，100%均运行Linux系统。¹

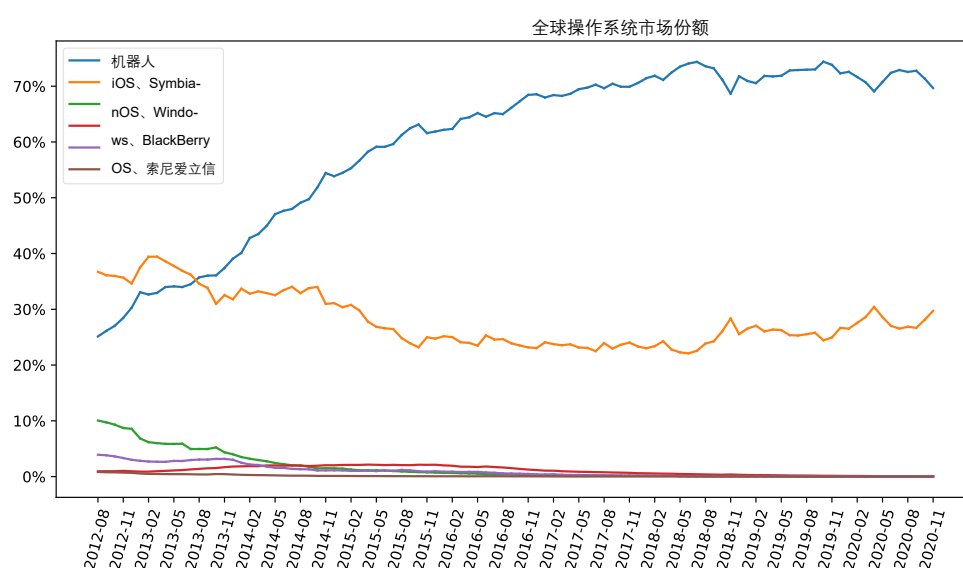


图 1.2: 智能手机和平板电脑上的操作系统市场份额²

Linux在消费市场取得的最大成功或许当属移动设备领域：Android系统不仅是智能手机上最主流的操作系统之一，在平板电脑（图1.2）以及近年来的可穿戴设备上也十分受欢迎。需要说明的是，由于独特的许可架构和技术实现差异，Android操作系统通常与标准Linux发行版分开归类。此外，由于Linux内核原生支持数十种处理器架构，它成为树莓派等单板计算机的首选操作系统。

¹根据<https://www.top500.org> 发布的全球前500台超级计算机配置数据，2017年共有498台系统运行Linux系统。自2018年6月起，Top500榜单上的每台超级计算机均采用基于Linux的操作系统。

²数据来源：<https://statcounter.com>

1.3 Using Linux

1.3.1 The Command-Line Format

Modern Linux desktop distributions ship with sophisticated Graphical User Interfaces (GUIs) that mimic the functional design of Windows, with variations mostly limited to user habits. A standard user can perform the majority of daily workflows within this graphical paradigm. However, for software developers, relying entirely on a GUI is inefficient; furthermore, production target embedded systems frequently operate completely without graphical heads.

Linux features a robust text-based management environment known as the **Command-Line Interface** (CLI). Operating via the command line requires learning specific structural rules and memorizing key syntax utilities, which occasionally creates a false impression that “Linux is difficult to use.”

It is important to separate regular consumers from computer developers. Consumers need a plug-and-play OS where a GUI is sufficient; developers, however, require the command line because it offers unmatched flexibility and detailed control, allowing complex or automated tasks to be executed in ways that are cumbersome or impossible within a standard GUI.

The command-line interface is rendered through a window called a **terminal**. The core program inside the terminal is the **shell**, which reads keyboard input, calls the requested program, executes it, and prints the result. Typing a line of text next to the terminal prompt character (`'$'` or `'#'`) and pressing Enter creates an executable command statement line.

- The `'$'` symbol represents a regular user account prompt.
- The `'#'` symbol denotes the root superuser account prompt.

These prompt layouts can be customized by the user. Commands that modify core system settings are restricted and will fail if executed from a regular user prompt without elevated privileges.

A complete command-line entry consists of three distinct parts: the command, options, and arguments:

```
$ command [options] [arguments]
```

The `'command'` specifies the action you want to perform; the `'options'` modify **how** the action is executed; and the `'arguments'` specify the target object of the operation.

1.3 使用 Linux

1.3.1 命令行格式

现代Linux桌面发行版均配备功能完善的图形用户界面（GUI），其设计风格与Windows系统高度相似，主要差异仅体现在用户操作习惯上。普通用户完全可以在这种图形界面环境下完成日常工作的大部分操作。然而对于软件开发者而言，完全依赖图形界面效率低下；此外，实际部署的嵌入式系统往往完全无需图形界面即可运行。

Linux具备一个强大的基于文本的管理环境，称为**命令行界面**（CLI）。通过命令行操作需要学习特定的结构规则并记忆关键语法命令，这有时会让人产生“Linux难以使用”的错误印象。

区分普通用户与计算机开发人员至关重要。普通用户需要即插即用的操作系统，图形界面即可满足需求；而开发人员则需要命令行界面，因其具备无可比拟的灵活性和精细控制能力，能够以标准图形界面难以实现或难以操作的方式执行复杂或自动化任务。

命令行界面通过一个名为**终端**的窗口呈现。终端内的核心程序是**外壳程序**，它负责读取键盘输入、调用指定程序、执行程序并输出结果。在终端提示符字符（`$`或`#`）旁输入一行文本并按下Enter键，即可生成可执行的命令行指令。

- `$` 符号表示常规用户账户提示。
- `#` 符号表示根超级用户账户提示。

这些提示界面布局可由用户自定义。修改核心系统设置的命令受到限制，若从普通用户提示符处执行且未具备高级权限，则会失败。

完整的命令行输入由三个独立部分组成：命令、选项和参数。

```
$ 命令 [ 选项 ] [ 参数 ]
```

`command` 定义了要执行的操作；`options` 修改了操作的执行方式；`arguments` 则指定了操作的目标对象。

The square brackets ‘[]’ signify that these components are optional depending on the utility; the brackets themselves are not typed. Some basic commands require no options or arguments, whereas others accept multiple items. Certain options require an adjacent argument directly following them.

Commands, options, and arguments must be separated by spaces. Options are typically prefixed with one dash ‘-’ (short options) or two dashes ‘--’ (long options), which form an unbroken string with the option character.

CRITICAL RULE: The Linux command line is strictly case-sensitive.

For example, to forcefully delete a Python file named ‘hello.py’, you would execute:

```
$ rm -f hello.py
```

In this statement, ‘rm’ (remove) is the core command, ‘-f’ (force) is the operational option, and ‘hello.py’ is the target file argument.

1.3.2 Users and Permissions

We can examine file systems, ownerships, and permissions using the ‘ls’ (list) directory utility. A frequent option used with this command is ‘-l’ (long listing format), which displays comprehensive file attributes. Another option, ‘-i’, displays the unique inode numbers of files. If the arguments do not conflict or cause syntax confusion, multiple short options can be combined behind a single dash (e.g., ‘-li’).

```
$ ls -li
4850428 drwxr-xr-x 2 pi pi      4096 Feb  8 11:35 graphics
4850423 -rw-r--r-- 2 pi pi 1524296 Feb  8 12:03 Pithon.pdf
4850317 -rwxr-xr-x 1 pi pi      5315 Feb  8 14:21 hello.py
4850423 -rw-r--r-- 2 pi pi 1524296 Feb  8 12:03 lecture.pdf
4850510 drwxr-xr-x 2 pi pi      4096 Feb  4 16:00 program
4850553 drwxr-xr-x 2 pi pi      4096 Feb  8 13:16 text
```

Each row in the terminal output details a single file block across 8 distinct fields:

1. inode number
2. File type & Access permissions
3. Hard link count
4. Owner user
5. Owner group

方括号 `[ ]` 表示这些组件根据具体用途可选；括号本身不带类型标识。某些基本命令无需选项或参数，而其他命令则支持多个参数；特定选项需紧随其后添加相邻参数。

命令、选项和参数必须用空格分隔。选项通常以一个短横线 `-`（短选项）或两个短横线 `--`（长选项）开头，与选项字符共同构成连续字符串。

关键规则：Linux命令行严格区分大小写。

例如，要强制删除名为 `hello.py` 的Python文件，可执行以下命令：

```
$ rm -f hello.py
```

在此语句中， `rm`（remove）为核心命令， `-f`（force）为操作选项，而 `hello.py` 为目标文件参数。

1.3.2 用户与权限

我们可以使用 `ls`（列表）目录工具来查看文件系统、所有权及权限信息。该命令常用的选项之一是 `-l`（长列表格式），可显示完整的文件属性；另一选项 `-li` 则可显示文件的唯一inode编号。若各参数互不冲突且不会导致语法混淆，多个简短选项可连用于单个短横线后（例如：`-li`）。

```
$ ls -li
4850428 drwxr-xr-x 2 pi pi      4096 二月 8 11:35 制图学
4850423 -rw-r--r-- 2 pi pi 1524296 二月 8 12:03 Python.pdf
4850317 -rwxr-xr-x 1 pi pi      5315 二月 8 14:21 你好, Py。
4850423 -rw-r--r-- 2 pi pi 1524296 二月 8 12:03 讲座.pdf
4850510 drwxr-xr-x 2 pi pi      4096 二月 4 16:00 程序
4850553 drwxr-xr-x 2 pi pi      4096 二月 8 13:16 文本
```

终端输出中的每行均详细记录了一个文件块，该记录包含8个不同的字段：

1. 资源节点编号
2. 文件类型与访问权限
3. 硬链接数量
4. 所有者用户
5. 所有者组

6. File size (in bytes)
7. Modification date/time
8. Filename

An **inode** (index node) is a data structure on a storage drive that describes the physical properties and allocation blocks of a file. Identical inode numbers indicate that separate directory entries point to the exact same physical storage data on the disk. This mechanism underpins Linux hard links. In the listing above, the third field indicates the hard link count. For instance, `'Python.pdf'` and `'lecture.pdf'` both share the inode number `'4850423'`. They are not duplicates; they are two different filenames pointing to the exact same file data on the disk.

The file type and permission string consists of 10 characters. The first character denotes the explicit file category, as detailed in Table 1.1.

Table 1.1: Linux File Type Identifiers

Symbol	File Type Category	Common Origin/Created via
–	Regular File	text editors, compilers, or applications
b	Block Device File	system initialization or the <code>'mknod'</code> utility
c	Character Device File	system initialization or the <code>'mknod'</code> utility
d	Directory	the <code>'mkdir'</code> command
l	Symbolic Link (Soft Link)	the <code>'ln -s'</code> command
p	Named Pipe (FIFO)	the <code>'mkfifo'</code> or <code>'mknod'</code> commands
s	Network Socket	the system network call <code>'bind()'</code>

The remaining 9 characters of the permission string are broken into three blocks of 3 characters each. These blocks define the permissions for the **Owner User**, the **Owner Group**, and all **Others**, respectively. The owner and group names are listed in the fourth and fifth fields.

Permissions within a block consist of read, write, and execute capabilities, represented by the characters `'r'`, `'w'`, and `'x'`. If a permission is withheld, its position is marked with a dash `'-'`.

In Linux, user configuration records are managed inside the `'/etc/passwd'` file. This file lists the username, User ID (UID), Group ID (GID), user metadata, and default shell login path for every user on the system. Historically, it also stored encrypted password strings. However, because this file must remain globally readable so processes

- 6. 文件大小（以字节为单位）
- 7. 修改日期/时间
- 8. 文件名

inode（索引节点）是存储驱动器上的一种数据结构，用于描述文件的物理属性及分配块信息。相同的**inode**编号表示不同的目录项均指向磁盘上完全一致的物理存储数据。该机制构成了Linux硬链接的基础。在上述列表中，第三个字段表示硬链接数量；例如， `ζPython.pdfζ` 与 `ζlecture.pdfζ` 均使用**inode**编号 `ζ4850423ζ`，二者并非重复项，而是指向磁盘上完全相同文件数据的两个不同文件名。

文件类型及权限字符串由10个字符组成。第一个字符表示具体的文件类别，详情参见表1.1。

表 1.1：Linux 文件类型标识符

符号	文件类型分类	共同来源/通过以下方式创建：
-	常规文件	文本编辑器、编译器或应用程序
b	阻止设备文件	系统初始化或 <code>ζmknodζ</code> 工具
c	字符设备文件	系统初始化或 <code>ζmknodζ</code> 工具
d	目录	<code>`ζmkdirζ`</code> 命令
l	符号链接（软链接）	<code>`ζln -sζ`</code> 命令
p	命名管道（FIFO）	<code>ζmkfifoζ</code> 或 <code>ζmknodζ</code> 命令
s	网络套接字	系统网络调用 <code>ζbind()ζ</code>

权限字符串中剩余的9个字符被分割为三个各含3个字符的块。这些块分别定义了所有者用户、所有者组以及所有其他用户的权限。所有者名称和组名称分别列于第四和第五字段中。

块内的权限包括读取、写入和执行三种功能，分别用字符 `ζrζ`、`ζwζ` 和 `ζxζ` 表示。若某项权限被拒绝，则其位置会标注为dash `ζ-ζ`。

在Linux系统中，用户配置记录存储于 `ζ/etc/passwdζ` 文件内。该文件列出了系统中每位用户的用户名、用户ID（UID）、组ID（GID）、用户元数据以及默认shell登录路径。历史上该文件还存储了加密后的密码字符串。但由于该文件必须保持全局可读性，因此相关操作需谨慎处理。

can map numeric IDs to names, storing passwords there created security risks from offline brute-force attacks. Consequently, password hashes are now securely isolated inside the `/etc/shadow` file, which can only be read with superuser root privileges.

Group allocations are handled via `/etc/group`, which records group names, Group IDs (GIDs), and user membership rosters. This file includes a group password field, though it is rarely used in modern systems.

Group partitioning in Linux simplifies project collaboration and serves as a fundamental security boundary. Every operation on the system is strictly bound to the permission limits of the executing user; any attempt to bypass these limits will be blocked by the kernel. The superuser account, named `root`, possesses absolute system administrative privileges. To prevent accidental system damage, administrators should avoid logging into a system directly as root for regular tasks.

1.3.3 Directory Operations

The primary command-line tools used to navigate and manipulate directories include:

- `mkdir`(make directory): Creates one or more new directories.

```
$ mkdir dir1 dir2 dir3
$ mkdir -p newdir/dir2
```

The `-p` (parents) option tells the utility to automatically generate any missing parent directories along the path instead of throwing an error.

- `rmdir`(remove directory): Deletes an empty directory.

```
$ rmdir dir1 newdir/dir2
```

This utility will fail if the target directory contains files or subdirectories, protecting users from accidental data loss.

- `cd`(change directory): Shifts the current working path of the shell.

```
$ cd /home/student/Desktop
```

Running `cd` with no arguments automatically returns the shell to the user's home directory. The tilde symbol `~` can also be used as a shortcut for the home directory.

- `pwd`(print working directory): Outputs the absolute path of the current directory. It requires no arguments.

密码哈希值可将数字ID映射为名称，并存储密码，从而有效防范离线暴力攻击带来的安全隐患。

因此，密码哈希值现已被安全地隔离在 `/etc/shadow` 文件中，仅能由超级用户root权限读取。

组分配通过 `/etc/group` 文件实现，该文件记录了组名称、组ID（GID）以及用户成员列表。此文件包含组密码字段，但在现代系统中已很少使用。

Linux中的组划分机制不仅简化了项目协作流程，更构成了基础的安全屏障。系统上的所有操作均严格受限于执行用户的权限范围；任何试图突破这些权限的行为都会被内核直接阻止。超级用户账户 `root` 拥有绝对的系统管理权限。为防止意外造成系统损坏，管理员在执行常规任务时应避免以root身份直接登录系统。

1.3.3 目录操作

用于浏览和操作目录的主要命令行工具包括：

- `mkdir`（创建目录）：创建一个或多个新目录。

```
$ 创建目录 dir1、dir2 和 dir3
$ mkdir -p newdir/dir2
```

`-p`（父目录）选项可指示工具自动生成路径中缺失的父级目录，而非抛出错误。

- `rmdir`（删除目录）：删除空目录。

```
$ rmdir dir1 newdir/dir2
```

若目标目录包含文件或子目录，该工具将无法正常运行，从而防止用户发生意外数据丢失。

- `cd`（更改目录）：切换shell的当前工作路径。

```
$ cd /home/ student / 桌面
```

不带参数运行 `cd` 会自动将shell目录返回至用户的主目录。波浪线符号 `~` 也可作为主目录的快捷方式使用。

- `pwd`（打印工作目录）：输出当前目录的绝对路径，无需参数。

Linux uses the forward slash ‘/’ as a path separator. The topmost boundary of the file system layout is known as the **root directory** (‘/’). Within every directory on the system, two special relative entries exist: ‘.’ (a single dot) represents the current directory, while ‘..’ (two dots) points to the immediate parent directory.

1.3.4 File Operations

The primary command-line tools used to manage files include:

- ‘cp’(copy): Duplicates files or directories. To copy a single file:

```
$ cp [options] source_file target_file
```

If the target file already exists, it will be overwritten silently. To prevent accidental data loss, use the ‘-i’(interactive) option, which prompts for confirmation before overwriting an existing file.

When copying multiple files, the final argument must be a directory. To copy an entire directory along with its contents, you must include the ‘-r’ (recursive) option; otherwise, directories will be skipped.

To simplify operations on groups of files with similar names, Linux supports **wildcards**. Table 1.2 lists the most common wildcards used in file operations.

For example, to recursively copy every file and folder inside your home directory to the ‘/mnt’ directory while prompting before overwriting, execute:

```
$ cp -ri ~/* /mnt
```

- ‘mv’(move): Relocates or renames files and folders. It can rename a file by moving it to a new filename, or move multiple files into a destination directory. Similar to ‘cp’, adding the ‘-i’ option prompts before overwriting files.
- ‘rm’(remove): Deletes files permanently. Unlike modern desktop environments that safely move files to a temporary “Trash” folder, the ‘rm’ command bypasses recovery mechanisms and unlinks data immediately, freeing disk space. Using the ‘-i’ option prompts for confirmation before deleting each file. The shell will also prompt you if you attempt to delete a read-only file.

```
$ ls -l file?
-rw-rw-r-- 1 pi pi 0 Feb 20 08:44 file1
-r--r--r-- 1 pi pi 0 Feb 20 08:44 file2
$ rm file?
```

Linux使用斜杠 `/` 作为路径分隔符。文件系统布局的最顶端边界称为**根目录** (`/`)。在系统中的每个目录内均存在两个特殊的相对路径项: `.` (单点) 表示当前目录, 而 `..` (双点) 则指向其直接父级目录。

1.3.4 文件操作

用于管理文件的主要命令行工具包括:

- `cp` (复制): 复制文件或目录。要复制单个文件, 请执行以下操作:

```
| 费用 [ 选项 ] 源文件 | 目标文件
```

若目标文件已存在, 系统将静默覆盖该文件。为避免意外数据丢失, 请使用 `-i` (`interactive`) 选项——该选项会在覆盖现有文件前提示用户确认。

复制多个文件时, 最后一个参数必须为目录。若需复制整个目录及其所有内容, 必须启用 `-r` (递归) 选项; 否则系统将跳过该目录。

为简化对名称相似的文件组进行操作, Linux支持**通配符**。表1.2列出了文件操作中最常用的通配符。

例如, 若需递归复制主目录下的所有文件和文件夹至 `/mnt` 目录, 并在覆盖前提示用户确认, 请执行以下命令:

```
$ cp -ri ~/* /mnt
```

- `mv` `MOVE` (移动): 用于移动或重命名文件和文件夹。该功能既可通过将文件移至新路径来重命名, 也可将多个文件批量移动至目标目录。与 `cp` 类似, 在覆盖文件前会提示添加 `-i` 选项。
- `rm` (删除): 永久删除文件。与现代桌面环境将文件安全移至临时“回收站”文件夹不同, `rm` 命令可绕过数据恢复机制并立即解除数据关联, 从而释放磁盘空间。使用 `-i` 选项时, 系统会在删除每个文件前提示确认; 若尝试删除只读文件, 系统也会发出警告。

```
$ ls -l 文件?
```

```
-rw-rw-r-- 1 π π 0 20 08年2月20日 08:44 file1
```

```
r--r--r-- 1 π π 0 20 08年2月20日 08:44 file2 $ rm file?
```


Table 1.2: Common File Management Wildcards

Wildcard	Functional Meaning	Syntax Example & Meaning
?	Matches exactly one arbitrary character	'???' (Matches filenames exactly three characters long)
*	Matches zero or more characters (except leading dots)	'*.c' (Matches all files ending with a '.c' extension)
[]	Matches any single character enclosed in the brackets	'*[Aa]' (Matches names ending in 'A' or 'a')
[-]	Matches a range of consecutive characters	'[A-Z]*' (Matches any name starting with an uppercase letter)
^ or !	Matches any character not listed inside the brackets	'[^0-9]*' (Matches filenames that do not begin with a digit)
{ }	Matches any comma-separated string within the braces	'*.c.{[a-z]*}' (Matches target extensions or string blocks)

```
rm: remove write-protected regular empty file 'file2'? y
```

To bypass these confirmation prompts during automated operations, use the `'-f'` (force) option. To delete directories along with all their nested contents, use the `'-r'` (recursive) option.

WARNING: Files deleted via the `'rm'` command cannot be recovered through regular system tools.

- File Viewing Utilities

While text editors can display file contents, they risk accidental modifications and load large files slowly. For read-only viewing, use these dedicated utilities instead:

1. `'cat'` (concatenate): Prints the entire file contents directly to the terminal screen. For long files, use `'Shift + PageUp'` or `'Shift + PageDown'` to scroll

表 1.2: 常见的文件管理通配符

空白区域	功能意义	语法示例及含义
?	精确匹配任意字符	ζζζ (匹配文件名长度恰好为三个字符)
*	匹配零个或多个字符 (不包括开头的点)	ζ*.cζ (匹配所有以 ζ.cζ 为扩展名的文件)
[]	匹配字符 任何被单独包裹在内的物体.....	ζ[Aa]ζ (匹配以 ζAζ 或 ζaζ 结尾的名称)
	括号	
[-]	匹配一系列连续字符	ζ[A-Z]*ζ (匹配以大写字母开头的任何名称)
^ 或 !	匹配括号内未列出的任何字符	ζ[^0-9]*ζ (匹配不以数字开头的文件名)
{ }	匹配项在括号内 任何 细绳 以逗号分隔	ζ*.c.{[a-z]*}ζ (匹配目标扩展名或字符串块)

注意：移除写保护的普通空文件 “file2” ? y

要在自动化操作中跳过这些确认提示，请使用 ζ-fζ (force) 选项；若需删除目录及其所有嵌套内容，请使用 ζ-rζ (recursive) 选项。

警告：通过 ζrmζ 命令删除的文件无法使用常规系统工具恢复。

• 文件查看工具

文本编辑器虽可显示文件内容，但存在意外修改风险且加载大文件速度较慢。如需仅读取内容，请使用专用工具：

- 1. ζcatζ (串联)：将整个文件内容直接打印到终端屏幕。对于长文件，请使用 ζShift + PageUpζ 或 ζShift + PageDownζ 进行滚动显示。

through the terminal buffer. Note that terminal screens have fixed buffers (typically 1,000 to 2,000 lines), meaning older lines will be dropped.

2. `'more'/'less'`(paging utilities): Display file contents one screenful at a time, pausing for keyboard input to advance or scroll. `'less'` is generally preferred as it allows both forward and backward navigation.
3. `'head'/'tail'`(boundary viewers): By default, `'head'` outputs the first 10 lines of a file, while `'tail'` outputs the final 10 lines. Use the `'-n'` option followed by a number to change the line count. Combined with a Linux pipeline, these tools can extract a specific section from the middle of a file:

```
$ head -n 20 car.py | tail -n 5
```

The vertical bar `|` is the **pipe operator**. It redirects the standard output streams of the left-hand command into the standard input stream of the right-hand command. The example above pipes the first 20 lines of `'car.py'` into `'tail'`, which prints the final 5 lines, effectively isolating lines 16 through 20.

4. `'tac'`(reverse cat): Operates exactly like `'cat'`, but reverses the output line order to print from bottom to top. It is commonly used to inspect system logs, as recent entries are appended to the end of the file.
- `'chmod'`(change mode): Modifies file permissions. The permission attributes shown by `'ls -l'` (e.g., `'-rwxr-xr-x'`) can be modified using octal values or symbolic notation, except for the first character which defines the file type.

The three permission blocks (User, Group, Others) can be represented as a three-digit octal number, where Read=4, Write=2, and Execute=1. For example, to give the owner full permissions (7) and everyone else read and execute permissions (5), use:

```
$ chmod 755 hello.py
```

Alternatively, you can use symbolic notation with target groups (`'u'` for user, `'g'` for group, `'o'` for others, `'a'` for all), operators (`'+'` to add, `'-'` to remove, `'='` to set explicitly), and permission flags (`'r'`, `'w'`, `'x'`). To make a script executable for everyone, use:

```
$ chmod a+x hello.py
```

通过终端缓冲区传输。请注意，终端屏幕具有固定的缓冲区容量（通常为1,000至2,000行），因此较旧的行会被舍弃。

2. “more” / “less”（分页功能）：每次显示一个屏幕范围的文件内容，需暂停等待键盘输入以进行翻页或滚动。“less”通常更受推荐，因为它支持前后双向导航。
3. “头” / “尾”（边界查看器）：默认情况下，“头”输出文件的前10行，而“尾”输出最后10行。可使用“-n”选项后跟数字来更改输出行数。结合Linux管道使用时，这些工具能够从文件中间提取特定部分：

```
$ 头部 -n 20车辆.py | 尾部 -n 5
```

竖线「|」是**管道运算符**。它会将左侧命令的标准输出流重定向到右侧命令的标准输入流中。上述示例将「car.py」文件的前20行内容通过管道传递给「tail」命令，后者则打印最后5行内容，从而有效隔离了第16至20行。

4. `tac`（反向猫形命令）：其运行方式与“cat”完全相同，但会反转输出顺序，从底部开始打印至顶部。该命令常用于查看系统日志，因为最新记录会被追加到文件末尾。
- “.mod”（更改模式）：修改文件权限。通过「ls -l」命令显示的权限属性（例如「-rwxr-xr-x」），可使用八进制值或符号表示法进行修改，但用于定义文件类型的首个字符除外。

三个权限类别（用户、组、其他）可用三位八进制数表示，其中读取权限对应4，写入权限对应2，执行权限对应1。例如，若需授予所有者完全权限(7)，而其他所有人仅具备读取和执行权限(5)，则使用：

```
$ chmod 755 hello.py
```

或者，您也可以使用针对目标组的符号表示法（“u”代表用户，“g”代表组）。

其中，“o”代表组成员，“a”代表所有用户；运算符包括“+”（加法）、“-”（减法）和“=”（显式赋值）；权限标志位为“r”、“w”和“x”。若要使脚本对所有人可执行，请使用以下命令：

```
$ chmod a+x hello.py
```

Unlike Windows, which relies on file extensions (like ‘.exe’ or ‘.com’) to identify executables, Linux relies strictly on these permission bits to determine whether a file can be executed as a program.

- ‘tar’(tape archive): Bundles multiple files into a single archive for storage or transfer, often compressing the output at the same time. Table 1.3 lists its common options.

Table 1.3: Common ‘tar’ Command Options

Option	Functional Meaning	Option	Functional Meaning
-A	Append tariff archives to an existing archive	-t	List the contents of an archive file
-c	Create a brand new archive file	-x	Extract files out from an archive bundle
-d	Diff check archive against active file system	-X	Exclude files listed in a separate file
-u	Update older files within an existing archive	-f	Specify the target archive filename
-delete	Delete specific member files from an archive	-j	Compress or decompress using the ‘bzip2’ algorithm
-r	Append regular files to the tail of an archive	-J	Compress or decompress using the ‘xz’ algorithm
-v	Verbose: display processing status filenames	-z	Compress or decompress using the ‘gzip’ algorithm
-C	Change to a specified directory before running		

Note: In traditional UNIX syntax, the leading dash ‘-’ for ‘tar’ options can be omitted. A standard operation must include exactly one primary action flag (‘c’, ‘x’, ‘t’, ‘A’, ‘d’, ‘r’, or ‘u’).

To compress and bundle a folder named ‘Desktop’ using the ‘bzip2’ algorithm into an archive named ‘desktop.tar.bz2’, run:

```
$ tar cjf desktop.tar.bz2 Desktop/
```

When extracting archives, ‘tar’ detects the compression format automatically. Specifying an incorrect decompression flag manually will cause the command to

与依赖文件扩展名（如 `ζ.exeζ` 或 `ζ.com ζ`）来识别可执行文件的Windows系统不同，Linux完全依靠这些权限位来判断文件能否作为程序运行。

- `ζtar ζ`（磁带归档）：将多个文件打包成单一归档文件以供存储或传输，通常同时对输出数据进行压缩。表1.3列出了其常见选项。

表 1.3：常见的 `ζtarζ` 命令选项

选项 -A	功能含义：将附录档案添	选项 -t	功能意义
	加到现有档案中		列出归档文件的内容
-c	创建全新归档文件	-x	从.....中提取文件存档文件包
-d	将归档文件与活动文件系统进行差异检查	-X	排除单独文件中列出的文件
-u	更新现有归档中的旧文件	-f	指定目标归档文件名
-删除	从归档文件中删除特定成员文件	-j	使用 <code>ζbzip2ζ</code> 算法进行压缩或解压缩
-r	将常规文件附加到归档文件的末尾	-J	使用 <code>ζxzζ</code> 算法进行压缩或解压缩
-v	详细信息：显示文件名的处理状态	-z	使用 <code>ζgzipζ</code> 算法进行压缩或解压缩
-C	运行前切换至指定目录		

注意：在传统UNIX语法中，`ζtarζ` 选项前的连字符 `ζ-ζ` 可省略。标准操作必须包含恰好一个主要动作标志（`ζcζ`、`ζxζ`、`ζtζ`、`ζAζ`、`ζdζ`、`ζrζ` 或 `ζuζ`）。

要使用 `ζbzip2ζ` 算法将名为 `ζDesktopζ` 的文件夹压缩打包成名为 `ζdesktop.tar bz2 ζ` 的归档文件，请运行以下命令：

```
$ tar cjf桌面.tar.bz2桌面/
```

在提取归档文件时，`ζtar ζ` 会自动识别压缩格式。若手动指定错误的解压标志，将导致命令无法正常运行。

fail. To extract an archive into a specific directory, use:

```
$ tar xvf desktop.tar.bz2 -C Downloads/
```

This extracts the contents of ‘desktop.tar.bz2’ into the ‘Downloads/’ folder while listing the files on screen. You can also use standalone compression utilities like ‘gzip’/‘gunzip’ or ‘zip’/‘unzip’. Different compression algorithms offer varying trade-offs between compression ratio, compression speed, and decompression times.

1.3.5 Process Control

A **process** is an active, running instance of a program managed by the operating system as a single unit of work. While a simple tool runs as a single process, complex applications often spawn multiple processes.

To view active system processes, use the ‘ps’ (process status) command. The ‘ax’ options list all running processes across the system:

```
$ ps ax
  PID TTY          STAT TIME  COMMAND
    1 ?           Ss   1:45  /sbin/init splash
    2 ?           S    0:00  [kthreadd]
    4 ?           S<   0:00  [kworker/0:0H]
20755 ?           Ssl   0:00  /usr/lib/evince/evince
21326 ?           Ss   0:00  /lib/systemd/systemd --user
21327 ?           S    0:00  (sd-pam)
23376 pts/0        Sl   0:47  evince LinuxBasis.pdf
23885 pts/2        S    0:01  bash
25034 pts/0        S+   3:20  vim text/commandline.tex
```

PID (Process ID): A unique 16-bit positive integer assigned sequentially by the kernel to track each process. Process ‘1’ is the system initialization daemon (‘init’ or ‘systemd’). If the maximum PID limit (32,767 by default) is reached, the counter wraps around to assign available lower numbers.

TTY: Specifies the terminal controlling the process. Core system daemons and graphical interfaces lack an associated terminal and display a ‘.’.

STAT: Indicates the current execution state of the process. A process exists in one of four primary states: Running (‘R’), Sleeping/Waiting (‘S’), Stopped (‘T’), or Zombie/Defunct (‘Z’). These states can include sub-flags (e.g., ‘s’ for session leader, ‘+’ for foreground process).

失败。要将归档文件解压到指定目录, 请使用:

```
$ tar xvf desktop.tar.bz2 -C 下载目录 /
```

该命令会将`desktop.tar.bz2`文件中的内容解压至`Downloads/`目录, 并同时在屏幕上列出所有文件。您也可使用独立的压缩工具, 例如`gzip`、`gunzip`或`zip`、`unzip`。不同的压缩算法在压缩比、压缩速度和解压时间之间存在不同的权衡关系。

1.3.5 过程控制

进程是操作系统作为单一工作单元管理的、正在运行的程序实例。简单工具以单个进程形式运行, 而复杂应用程序通常会创建多个进程。

要查看活动的系统进程, 请使用`ps` (进程状态) 命令。该`ax`可列出系统中所有运行中的进程:

```
$ ps ax
```

PID	TTY	统计: 数据	时间: 时刻	命令
1	?	Ss	1:45	/sbin/Splash 启动画面
2	?	S	0:00	[kthreadd]
4	?	S<	0:00	[kworker/0:0H]
20755	?	Ssl	0:00	/usr/lib/evince/evince
21326	?	Ss	0:00	/lib/systemd/systemd --user
21327	?	S	0:00	(sd-pam)
23376	pts/0	Sl	0:47	返回 LinuxBasis.pdf
23885	pts/2	S	0:01	巴什
25034	pts/0	s+	3:20	VMX 文本/命令行 .tex

PID (进程ID): 由内核依次分配的唯一16位正整数, 用于标识每个进程。进程1为系统初始化守护进程 (init 或 systemd)。当达到最大PID限制值 (默认为32,767) 时, 计数器会循环回起始值以分配可用的较低编号。

TTY: 指定控制该进程的终端。核心系统守护进程及图形界面均不配备对应终端, 显示“?”符号。

STAT: 表示进程的当前执行状态。进程存在四种主要状态之一: 运行 (R)、休眠/等待 (S)、停止 (T) 或僵尸/失效 (Z)。这些状态可包含子标志 (例如, 用于会话领导者的s、用于前台进程的+或f)。

1.3.6 Modifying Process States

You can terminate or signal running processes using the ‘kill’, ‘pkill’, or ‘killall’ commands. The ‘kill’ command targets processes by their numeric PID, whereas ‘pkill’ and ‘killall’ target processes by their alphanumeric “program name”:

```
$ kill 23376          # Targets process by numeric PID
$ pkill evince        # Targets process by executable name
```

Despite its name, the ‘kill’ utility does not destroy processes directly; instead, it sends an asynchronous **signal** to them. Signals are a core mechanism for inter-process communication (IPC) in Linux. By default, ‘kill’ sends a ‘SIGTERM’ (15, terminate) signal, which requests the target process to save its state and exit cleanly.

Table 1.4 lists common Linux signals. You can specify these signals using either their numeric value or symbolic name as an option to the ‘kill’ command.

Some processes may hang or ignore a standard ‘SIGTERM’ signal. In these scenarios, you can force termination by sending a non-catchable ‘SIGKILL’ signal using the ‘-9’ option:

```
$ kill -9 23376
```

Signals can also control execution states without terminating the process. For example, you can pause and resume a process using ‘SIGSTOP’ and ‘SIGCONT’:

```
$ kill -STOP 23376    # Pauses the target process
$ kill -CONT 23376    # Resumes the paused process
```

1.3.7 Foreground and Background Execution

When launching graphical applications or long-running tasks from a terminal, you can append an ampersand symbol ‘&’ to the end of the command line. This runs the program as a **background process**, leaving the terminal prompt free to accept subsequent commands immediately:

```
$ idle3 &
```

You can manage these background tasks using the shell’s built-in ‘fg’ (foreground) and ‘bg’ (background) utilities. Running ‘fg’ brings the most recent background task back into the foreground.

If multiple jobs are running concurrently in the background, use the built-in ‘jobs’ command to list them along with their index numbers. You can pass these index numbers to ‘fg’ to target a specific job:

1.3.6 修改流程状态

可使用 `kill`、`kill` 或 `killall` 命令终止或向运行中的进程发送信号。`kill` 命令通过进程的数值PID进行定位，而 `kill` 和 `killall` 则通过程序名称（由字母和数字组成）进行定位。

```
$ 杀死编号为 23376 的进程（通过其数字 PID）  
$ pkill evince #按可执行文件名查找目标进程
```

尽管名称如此，`kill` 工具并不会直接终止进程；它会向进程发送异步信号。信号是Linux系统中进程间通信（IPC）的核心机制。默认情况下，`kill` 会发送 `SIGTERM`（15，`terminate`）信号，要求目标进程保存其状态并干净退出。

表1.4列出了常见的Linux信号。您可使用这些信号的数值或符号名称作为 `kill` 命令的选项进行指定。

某些进程可能会卡死或忽略标准 `SIGTERM` 信号。在此情况下，可使用 `-9` 选项发送不可捕获的 `SIGKILL` 信号来强制终止进程：

```
$ kill -9 23376
```

信号还能在不终止进程的情况下控制其执行状态。例如，可通过 `SIGSTOP` 和 `SIGCONT` 命令暂停或恢复进程。

```
$ kill -STOP 23376 # 暂停目标进程  
$ kill -CONT 23376 # 恢复已暂停的进程
```

1.3.7 前台与后台执行

从终端启动图形应用程序或长期运行的任务时，可在命令行末尾添加竖线符号 `&`。这将使程序以后台进程模式运行，从而使终端提示符可立即接收后续命令：

```
$ idle3 &
```

您可使用Shell内置的 `fg`（前台）和 `bg`（后台）工具来管理这些后台任务。运行 `fg` 命令可将最新的后台任务重新移至前台。

若后台存在多个并发运行的任务，请使用内置的 `jobs` 命令列出所有任务及其索引编号。您可将这些索引编号传递给 `fg` 以定位特定任务：

Table 1.4: Standard Linux System Signals

Signal Name	Numeric Value	Description/Operational Behavior
SIGHUP	1	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Interrupt from keyboard (triggered via ‘Ctrl + C’)
SIGQUIT	3	Quit from keyboard; forces a core dump execution
SIGABRT	6	Abort signal; indicates abnormal terminal execution
SIGKILL	9	Kill signal; forces immediate termination (cannot be blocked or caught)
SIGUSR1	10	User-defined signal 1; available for custom application actions
SIGSEGV	11	Segmentation fault; invalid or restricted memory access reference
SIGUSR2	12	User-defined signal 2; available for custom application actions
SIGPIPE	13	Broken pipe; write executed to a pipe with no readers
SIGALRM	14	Timer signal emitted by the kernel alert clock
SIGTERM	15	Termination signal; requests a clean program exit (default action)
SIGCHLD	17	Child process stopped, terminated, or changed state
SIGCONT	18	Continue execution if currently stopped
SIGSTOP	19	Stop process execution immediately (cannot be blocked or caught)

```
$ fg %1
```

Pressing ‘Ctrl+Z’ in the terminal sends a ‘SIGSTOP’ signal to the active foreground process, pausing it and returning you to the shell prompt. You can then run ‘bg’ to let the paused process resume execution in the background, or ‘fg’ to resume it in the foreground.

表 1.4：标准 Linux 系统信号

信号名称	数值	描述/操作行为
SIGHUP	1	在控制终端检测到挂机或控制进程已终止
信号情报；信息情报	2	来自键盘的中断（通过 “Ctrl + C” 触发）
SIGQUIT	3	退出键盘输入；强制执行核心转储
SIGABRT	6	中断信号；表示终端执行异常
SIGKILL	9	终止信号；强制立即终止（不可被阻断或捕获）
SIGUSR1	10	用户自定义信号1；可用于定制应用程序操作
SIGSEGV	11	分段错误；无效或受限的内存访问引用
SIGUSR2	12	用户自定义信号2；可用于定制应用程序操作
SIGPIPE	13	管道已损坏；写操作已执行到没有读取器的管道中
SIGALRM	14	由内核警报时钟发出的定时信号
SIGTERM	15	终止信号；请求程序干净退出（默认操作）
SIGCHLD	17	子进程已停止、终止或状态发生变化
SIGCONT	18	如果当前已停止，则继续执行
SIGSTOP	19	立即停止进程执行（无法被阻断或捕获）

```
$ fg %1
```

在终端中按下 ‘Ctrl+Z’ 可向当前活动的前台进程发送 ‘sigstop’ 信号，使其暂停并返回命令行界面。随后可运行 ‘bg’ 使暂停的进程在后台恢复执行，或运行 ‘fg’ 将其恢复至前台。

1.3.8 I/O Redirection and Pipelines

Standard Input/Output (I/O) redirection is a powerful feature of the Linux file system architecture. At the system level, all user interactions map to file reads and writes. Redirection allows you to change where a program reads its input from or writes its output to.

Table 1.5 lists the standard redirection operators.

Table 1.5: Standard Shell I/O Redirection Operators

Operator Symbol	Functional Redirection Meaning
>	Redirects standard output (stdout) to a file (overwrites existing file contents)
>>	Redirects standard output (stdout) to a file (appends to existing file contents)
<	Redirects standard input (stdin) from a file
<< Delimiter	Here-doc: Feeds input into a command until the specified delimiter string is met
<<< string	Here-string: Passes an explicit inline text string directly to standard input

To extract lines 16 through 20 from ‘hello.py’ and write them into a new file named ‘hello_bak.py’, combine a pipeline with output redirection:

```
$ head -n 20 hello.py | tail -n 5 > hello_bak.py
```

To append a line of text to the end of an existing file without modifying its current contents, use the append operator ‘>>’:

```
$ echo "hello, world" >> hello.py
```

IMPORTANT: The single ‘>’ operator creates the target file if it does not exist, but **overwrites** it silently if it does. The double ‘>>’ operator appends data safely to the end of the file.

If a script requires automated keyboard input, you can pre-write the responses into a text file and pass it to the script using input redirection (‘<’), avoiding the need for manual interaction:

```
$ head < hello.py
```

1.3.8 输入/输出重定向与管道

标准输入/输出（I/O）重定向是Linux文件系统架构的一项强大功能。在系统层面，所有用户操作均可映射为文件的读取与写入操作。该功能可让您更改程序读取输入数据或写入输出数据的位置。

表1.5列出了标准重定向操作符。

表 1.5：标准 Shell I/O 转向操作符

操作符符号	功能重定向的含义
>	将标准输出（stdout）重定向到文件（覆盖现有文件内容）
>>	将标准输出（stdout）重定向到文件（追加到现有文件内容中）
<	将标准输入（stdin）从文件重定向
<< 分隔符	Here-doc：将输入内容逐条传递给命令，直至遇到指定的分隔符字符串为止。
<<< 细绳	Here-string：将明确的内联文本字符串直接传递给标准输入

要从 ‘hello.py’ 中提取第16至20行代码并将其写入名为 ‘hello_bak.py’ 的新文件，请结合输出重定向的管道流程：

```
$ head -n 20 hello.py | tail -n 5 > hello_bak.py
```

若要在现有文件末尾追加一行文本而不修改其当前内容，请使用追加运算符「>>」：

```
$ echo "你好，世界" >>hello.py
```

重要：单引号 “ ” 运算符在目标文件不存在时创建该文件；若文件已存在，则会静默地覆盖该文件。双引号 ‘>>’ 运算符可安全地将数据追加到文件末尾。

若脚本需要自动键盘输入，可将响应内容预先写入文本文件，并通过输入重定向（ ‘<’ ）将其传递给脚本，从而避免手动操作：

```
$ 头部<你好.py
```

1.4 Text Editors

Writing code requires editing plain-text files without hidden formatting codes. In the Linux ecosystem, **VIM** (Vi Improved) is one of the most widely used and powerful command-line text editors.

VIM runs directly inside the terminal without standard menus or mouse interactions, relying entirely on keyboard shortcuts. It operates using three primary modes, as illustrated in Figure 1.3:

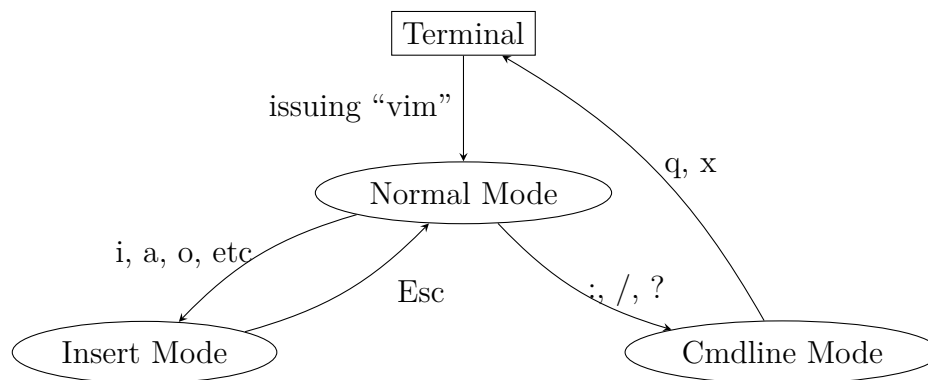


Figure 1.3: VIM Mode Transitions

1. **Normal Mode (Command Mode):** The default landing mode when opening a file. Keypresses are interpreted as navigation or structural editing commands. Typing insertion keys (such as ‘a’ or ‘i’) switches the editor to Insert Mode. Typing punctuation keys (such as ‘:’, ‘/’, or ‘;’) switches it to Command-Line Mode.
2. **Insert Mode (Editing Mode):** Functions like a conventional text editor where typing inputs characters and arrow keys move the cursor. Pressing the ‘Esc’ key exits Insert Mode and returns you to Normal Mode.
3. **Command-Line Mode (Ex Mode):** Activates a prompt at the very bottom line of the screen to execute global operations, such as saving, quitting, or running advanced regex search-and-replace actions.

VIM Command Reference

To switch from Normal Mode into Insert Mode, press one of the following keys:

- i/I: Insert text before the cursor/at the beginning of the current line.
- a/A: Append text after the cursor/at the end of the current line.

1.4 文本编辑器

编写代码意味着编辑不含隐藏格式代码的纯文本文件。在Linux生态系统中，VIM（Vi改进版）是最广泛使用且功能最强大的命令行文本编辑器之一。

VIM直接运行于终端界面中，无需标准菜单或鼠标操作，完全依赖键盘快捷键。

其运行模式主要分为三种，如图1.3所示：

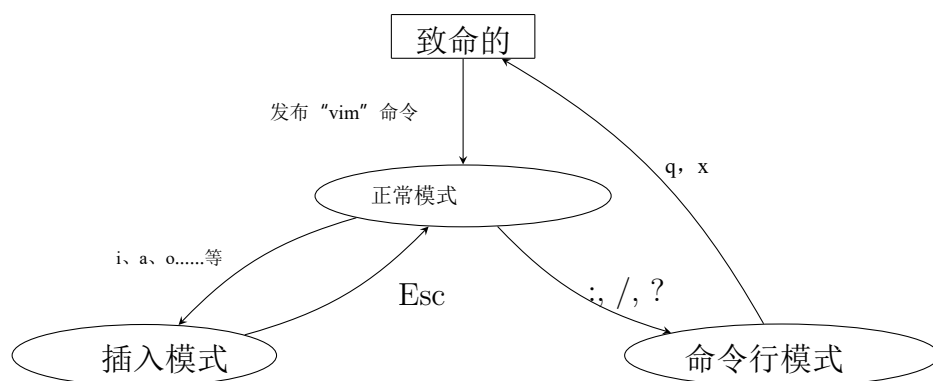


图 1.3: VIM 模式切换

1. **正常模式（命令模式）**：打开文件时的默认编辑模式。按键操作将被识别为导航或结构编辑指令；输入插入键（如 `ia` 或 `iz`）可切换至插入模式；输入标点键（如 `;`、`/` 或 `?`）则切换至命令行模式。
2. **插入模式（编辑模式）**：该模式类似传统文本编辑器，用户可通过输入字符或使用方向键移动光标；按下 `Esc` 键可退出插入模式并返回正常模式。
3. **命令行模式（扩展模式）**：在屏幕最底部显示提示符，用于执行全局操作，如保存、退出或运行高级正则表达式搜索与替换功能。

VIM命令参考

要从正常模式切换至插入模式，请按下以下任一按键：

- `i/I`：在光标位置或当前行开头插入文本。
- `a/A`：在光标处或当前行末尾添加文本。

- o/O: Open a new blank line below/above the current line.
- R: Enter replace mode to overwrite text characters.
- s/S: Substitute a single character/the entire current line.
- C: Delete text from the current cursor position to the end of the line and enter Insert Mode.

Common editing shortcuts in Normal Mode:

- ~: Toggle character capitalization between uppercase and lowercase.
- x/X: Delete the character directly under/before the cursor.
- dd/D: Delete (cut) the entire current line/from the cursor to the end of the line.
- dt.: Delete text from the current cursor position up to the first period ‘.’ character.
- J: Join the current line with the line directly below it.
- u/Ctrl+R: Undo the last action/Redo the undone action.
- .: Repeat the immediate preceding editing operation.
- y/yy: Copy (yank) the current visual selection/the entire current line into the buffer.
- p/P: Paste the buffer contents directly after/before the current cursor position.
- ZZ: Save all changes and exit VIM immediately.

Efficient cursor navigation in Normal Mode:

- 0/\$: Move the cursor to the exact start/end of the current line.
- ^: Move the cursor to the first non-whitespace character on the line.
- tx/fx: Jump the cursor right up to/onto the next occurrence of character ‘x’.
- 5h/5j/5k/5l: Move 5 characters left, 5 lines down, 5 lines up, or 5 characters right.
- 100G: Jump the cursor directly to line number 100.

Common actions in Command-Line Mode (prefixed with a colon ‘:’):

- :w file: Save (write) changes out to a specific file target.

- o/O: 在当前行下方/上方插入一条新的空白行。
- R: 进入替换模式以覆盖文本字符。
- s/S: 替换单个字符/整行内容。
- C: 从当前光标位置开始删除至行尾的所有文本，并进入插入模式。

正常模式下的常见编辑快捷键：

- : ~ 在大写和小写之间切换字符大小写。
- x/X: 删除光标正下方/前方的字符。
- dd/D: 删除（截断）从光标位置至行末的整个当前行。
- dt.: 从当前光标位置开始删除文本，直至第一个句号 `.` 字符为止。
- J: 将当前线条与下方的线条连接起来。
- u/Ctrl+R: 撤销上一步操作/重做已撤销的操作。
- : 重复前一步编辑操作。
- y/yy: 将当前视觉选区/整条当前线条复制到缓冲区。
- p/P: 将缓冲区内容直接粘贴到当前光标位置之后/之前。
- ZZ: 保存所有更改后立即退出 VIM。

正常模式下的高效光标导航：

- 0/\$: 将光标移动到当前行的确切起始/结束位置。
- 将光标移动到该行的第一个非空白字符处。
- tx/fx: 将光标直接跳转至下一个字符 `x` 处。
- 5h/5j/5k/5l: 向左移动5个字符，向下移动5行，向上移动5行，或向右移动5个字符。
- 100G: 直接将光标跳转到第100行。

命令行模式下的常用操作（以冒号开头 `:`）：

- -w文件: 将修改内容保存（写入）到指定文件中。

- `:q/:q!`: Quit the editor/Force-quit immediately, discarding all unsaved modifications.
- `:x`: Save changes and quit (equivalent to running `‘w’` followed by `‘q’`).
- `:1,20s/foo/bar/g`: Substitute every instance of string `‘foo’` with `‘bar’` between lines 1 and 20.
- `:%s/foo/bar/g`: Substitute every instance of string `‘foo’` with `‘bar’` across the entire file.
- `.,$-20s/foo/bar/g`: Substitute strings from the current line (`‘.’`) up to 20 lines from the end of the file (`‘$-20’`).
- `:1,20d`: Delete lines 1 through 20.
- `:5,10m3`: Move lines 5 through 10 and insert them directly after line 3.
- `:g/for/l`: Globally search and list every line containing the string match `‘for’`.
- `:v/for/d`: Invert match: globally delete all lines that **do not** contain the string `‘for’`.

Advanced VIM Layout Features:

- Window Splitting: In Normal Mode, press `‘Ctrl + W’` followed by `‘s’` to split the screen horizontally, or `‘v’` to split it vertically. This allows you to view different files simultaneously or look at two separate sections of the same file side-by-side.
- Visual Block Mode: Press `‘Ctrl + V’` to select text blocks by column rather than full rows. This allows you to delete columns of text, move blocks, or insert characters across multiple rows simultaneously (by typing `‘Ctrl + V’`, selecting rows, pressing `‘I’`, typing text, and hitting `‘Esc’`).

For beginners, VIM includes a built-in interactive tutorial called `‘vimtutor’`. You can access the complete documentation at any time inside VIM by typing `‘:help’`.

1.5 Network Connectivity

1.5.1 Remote Terminal Access

In embedded applications, target single-board computers like the Raspberry Pi are typically deployed as headless units without dedicated keyboards, mice, or monitors.

- `:q!q!` : 退出编辑器/立即强制退出, 放弃所有未保存的修改。
- `:x` : 保存更改并退出 (相当于运行 `':w'` 后跟 `':q'`)。
- `:1,20s.foo/bar/g` : 在第1行至第20行之间, 将字符串 `'foo'` 的所有出现位置均替换为 `'bar'` 。
- `:%s/foo/bar/g` : 在整个文件中将所有 `"foo"` 字符串实例替换为 `"bar"` 。
- `., $-20s.foo/bar/g` : 替换从当前行 (`'.'`) 开始至文件末尾后20行 (`'-20'`) 内的字符串。
- `:1,20d` : 删除第1行至第20行。
- `: 5,10m3` : 将第5行至第10行的内容移动, 并直接插入到第3行之后。
- `/g/for/l` : 在全球范围内搜索并列出所有包含字符串 `"for"` 匹配项的行。
- `/v:for/d` : 反向匹配: 全局删除所有不包含字符串 `'for'` 的行。**不包括**该字符串的情况。

VIM高级布局功能:

- 窗口分割: 在正常模式下, 按 `"Ctrl + W"` 后接 `"s"` 可水平分割屏幕, 按 `"v"` 可垂直分割屏幕。这样您可以同时查看不同文件, 或并排查看同一文件的两个独立部分。
- 视觉块模式: 按下 `"Ctrl + V"` 按列而非整行选择文本块。此功能可让您同时删除文本列、移动文本块或在多行中插入字符 (操作步骤为: 按下 `"Ctrl + V"` , 选择行, 按下 `"I"` , 输入文本, 再按下 `"Esc"`)。

对于初学者, VIM内置了一个名为 `"vimtutor"` 的交互式教程。您可随时在VIM中输入 `":help"` 来查看完整的文档说明。

1.5 网络连接

1.5.1 远程终端访问

在嵌入式应用中, 诸如Raspberry Pi之类的靶向单板计算机通常以无头单元形式部署, 无需配备专用键盘、鼠标或显示器。

While the Raspberry Pi natively supports standard USB devices and HDMI outputs, a dedicated workstation setup is usually unnecessary during development. Instead, developers can manage the hardware remotely over a local network.

Most Linux distributions run a background **SSH** (Secure Shell) service to provide secure command-line access over a network. Suppose your Raspberry Pi has an IP address of ‘192.168.2.103’.³ When it runs the SSH daemon (‘sshd’) and is connected to your local network, you can log in remotely from your personal computer’s terminal:

```
$ ssh pi@192.168.2.103
The authenticity of host '192.168.2.103 (192.168.2.103)' can't be established.
ECDSA key fingerprint is SHA256:rrFsuRrEhzNaGea15HNvGRCcYDJmaCbAwMYmja+fyD0.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.2.103' (ECDSA) to the list of known hosts.
pi@192.168.2.103's password:
```

During your first connection, SSH displays a security warning verifying the remote host’s fingerprint. Type ‘yes’ to save the host key to your local configuration. For subsequent connections, you will only be prompted for the user account password. Once authenticated, the terminal displays the remote login banner:

```
#####
# Raspberry Pi3 (aarch64)                #
# Desktop                                #
# WELCOME!                               #
#####
pi@raspberrypi:~$
```

In this example, the Raspberry Pi’s network IP address is ‘192.168.2.103’ and the login user is ‘pi’. For security reasons, most Linux distributions block remote root logins by default. Once logged in, any commands you type run directly on the Raspberry Pi. This terminal environment functions exactly like a local shell, missing only the graphical desktop.

The SSH suite also includes ‘scp’ (secure copy) for secure file transfers over the network:

```
$ scp file1 user@192.168.200.170:file2
```

³An Internet Protocol address (IP address) is a numerical label, such as 192.168.1.1 (IPv4) or 2001:0db8:0000:0000:8a2e:0370:7334 (IPv6), that is assigned to a device connected to a computer network that uses the Internet Protocol for communication. IP addresses serve two main functions: network interface identification, and location addressing.

虽然树莓派原生支持标准USB设备和HDMI输出接口，但在开发过程中通常无需专门配置工作站。开发者可通过本地网络远程管理硬件设备。

大多数Linux发行版都会在后台运行SSH（安全外壳）服务，以提供通过网络的安全命令行访问功能。假设您的树莓派设备IP地址为 ζ 192.168.2.103 ζ^3 ，当其运行SSH守护进程（ ζ sshd ζ ）并连接至本地网络时，您即可从个人电脑终端远程登录：

```
$ ssh pi@192.168.2.103
无法验证主机 '192.168.2.103 (192.168.2.103)' 的真实性。ECDSA 关键指纹为 SHA256: rrF suRrEhzNaG
ea15HNvGRC cYDJma CbAwMYm ja+fyD0。您确定要继续连接吗（是/否）？ 是
警告：已将 "192.168.2.103"（ECDSA）永久添加到已知主机列表中。
pi@192.168.2.103 的密码：
```

首次连接时，SSH会显示安全警告以验证远程主机的指纹。输入 ζ yes ζ 可将主机密钥保存至本地配置中。后续连接仅需输入用户账户密码即可。完成身份验证后，终端将显示远程登录界面：

```
#####
# Raspberry Pi 3（基于 Arch64 架构）          #
# 桌面                                          #
# 欢迎您！      #      #####                  pi@
raspberrypi:~$
```

在本示例中，树莓派的网络IP地址为 ζ 192.168.2.103 ζ ，登录用户为 ζ pi ζ 。出于安全考虑，大多数Linux发行版默认会阻止远程root账户登录。登录后，您输入的所有命令均直接在树莓派上执行。该终端环境的功能与本地shell完全相同，仅缺少图形界面。

SSH套件还包含 ζ scp ζ （安全复制功能），可实现网络上的安全文件传输：

```
使用命令 `scp file1 user@192.168.200.170: file2`
```

³ 互联网协议地址（IP地址）是一种数字标识，例如 192.168.1.1（IPv4）或 2001:0db8:0000:0000:0000:8a2e:0370:7334（IPv6），用于分配给连接到使用互联网协议进行通信的计算机网络中的设备。IP地址主要承担两项功能：网络接口识别和位置定位。

This copies the local file ‘file1’ to the home directory of the user ‘user’ on the remote machine ‘192.168.200.170’, renaming it ‘file2’. You will be prompted to enter the remote user’s password. If you want to keep the original filename, omit the target filename but keep the trailing colon:

```
$ scp user@192.168.200.170:file1 .
```

The dot ‘.’ represents the current local directory. This copies ‘file1’ from the remote machine directly into your local working directory under its original name.

Configuring Passwordless SSH Key Authentication

Entering passwords for every command or file transfer can slow down development. You can configure key-based authentication to log in securely without typing passwords:

1. Generate an SSH key pair on your development machine:

```
$ ssh-keygen -t rsa -b 4096
```

The ‘-t’ option specifies the encryption algorithm, and ‘-b’ defines the key length in bits. The utility generates a private key (‘id_rsa’) and a public key (‘id_rsa.pub’) inside your ‘~/ .ssh/’ folder. You can add an optional passphrase to encrypt the private key for extra security.

SECURITY RULE: The public key can be shared freely, but your private key must remain secure on your host machine.

2. Copy the public key to the remote Raspberry Pi:

```
$ ssh-copy-id pi@192.168.2.103
```

This automated tool uploads your local public key (‘id_rsa.pub’) and appends it to the ‘~/ .ssh/authorized_keys’ file on the remote machine, performing an operation equivalent to:

```
pi@remote:~$ cat id_rsa.pub >> ~/.ssh/authorized_keys
```

Once configured, you can run remote logins and ‘scp’ transfers instantly without password prompts.

1.5.2 Remote Desktop Access

If your development work requires a graphical interface, you can pass the ‘-X’ option through SSH to enable **X11 Forwarding**. This forwards graphical windows from the remote server to your local display:

此操作会将本地文件 `file1` 复制到远程计算机 `192.168.200.170` 上用户 `user` 的主目录，并将其重命名为 `file2`。系统将提示您输入远程用户的密码。如需保留原始文件名，请省略目标文件名但保留末尾的冒号：

```
使用命令 `scp user@192.168.200.170:file1`:
```

点符号 `.` 表示当前本地目录。该文件会将 `file1` 从远程机器直接复制到您的本地工作目录中，并保留其原始名称。

配置无密码 SSH 密钥认证

为每个命令或文件传输都手动输入密码会拖慢开发进度。您可以配置基于密钥的身份验证方式，无需输入密码即可安全登录。

1. 在开发机器上生成SSH密钥对：

```
$ ssh-keygen -t rsa -b 4096
```

`-t` 选项用于指定加密算法，而 `-b` 则定义密钥长度（以比特为单位）。该工具会在您的 `~/.ssh/` 目录下生成私钥（`id_rsa`）和公钥（`id_rsa.pub`）。

您可添加可选的密码短语对私钥进行加密，以增强安全性。

安全规则： 公钥可自由共享，但私钥必须在主机上保持安全。

2. 将公钥复制到远程树莓派设备：

```
$ ssh-copy-id pi@192.168.2.103
```

该自动化工具会上传您的本地公钥（`id_rsa.pub`），并将其添加到远程机器上的 `~/.ssh/authorized_keys` 文件中，执行的操作等同于：

```
pi@remote :~$ cat id_rsa.pub >> ~/.ssh/authorizedKeys
```

配置完成后，您即可立即执行远程登录和 `scp` 传输操作，无需输入密码。

1.5.2 远程桌面访问

若您的开发工作需要图形界面，可通过SSH传输 `-X` 选项以启用X11转发功能。

该功能可将远程服务器的图形窗口转发至本地显示器：


```
$ ssh pi@192.168.2.103 -X
```

When you launch a GUI application from this session, its interface renders inside a local window on your desktop. This requires X11 forwarding to be enabled in the remote server's configuration file (`/etc/ssh/sshd_config`):

```
X11Forwarding yes
```

Alternatively, you can use **VNC** (Virtual Network Computing) for a full remote desktop experience. To start a VNC server on the Raspberry Pi, use `'x11vnc'`:

```
# x11vnc -display :0 -shared -forever >/dev/null 2>/dev/null &
```

- `-display :0`: Connects to the primary local desktop display session.
- `-shared`: Allows multiple client connections simultaneously.
- `-forever`: Keeps the VNC server running after a client disconnects.
- `>/dev/null 2>/dev/null&`: Suppresses output messages and runs the server in the background.

Once the VNC server is running, connect from your host machine using a client viewer like `'vncviewer'`:

```
$ vncviewer 192.168.2.103
```

This opens a window displaying the active remote desktop environment (such as the lightweight XFCE4 desktop shown in Figure 1.4).



Figure 1.4: XFCE4 Graphical Desktop Environment Session on a Raspberry Pi

```
$ ssh pi@192.168.2.103 -X
```

从本次会话中启动图形用户界面应用程序时，其界面将显示在桌面的本地窗口中。这需要在远程服务器的配置文件（`/etc/ssh/sshd_config`）中启用X11转发功能。

X11转发：是

或者，您可使用VNC（虚拟网络计算）来实现完整的远程桌面体验。要在树莓派上启动 VNC 服务器，请运行 `x11vnc`：

```
# x11vnc -display :0 -共享 -永久 >/dev/null 2>/dev/null &
```

- 显示：0：连接至主本地桌面显示会话。
- 共享：支持同时建立多个客户端连接。
- -forever：在客户端断开连接后仍保持 VNC 服务器持续运行。
- /dev/null 2>/dev/null&：隐藏输出信息并在后台运行服务器。

VNC 服务器启动后，请使用 `vncviewer` 等客户端查看器从您的主机连接。

```
$ vncviewer 192.168.2.103
```

此时将弹出一个窗口，显示当前激活的远程桌面环境（例如图1.4所示的轻量级XFCE4桌面）。



图 1.4：Raspberry Pi 上的 XFCE4 图形桌面环境会话

1.5.3 Remote Network Storage (NFS)

Embedded devices often have limited onboard storage. You can expand available space using a **Network File System (NFS)** to share directories from your development PC with the Raspberry Pi.

First, configure your PC as an NFS server by adding the shared directory paths and access permissions to `/etc/exports`:

```
/srv/nfs4 192.168.*.*(rw,sync,subtree_check,no_root_squash)
```

Assuming your PC's network IP address is `192.168.200.170`, log into the Raspberry Pi as root and mount the shared directory onto a local folder path:

```
# mount 192.168.200.170:/srv/nfs4 /mnt -o nolock,proto=tcp
```

This mounts the PC's `/srv/nfs4` directory directly onto the Raspberry Pi's `/mnt` folder. Both machines now access the exact same files simultaneously, while the physical data remains stored on the PC's hard drive. The `-o proto=tcp` option forces the connection to use the TCP protocol, avoiding the data corruption or performance drops that can occur with NFS's default UDP transport when copying large files.

Using network connections during embedded development eliminates the need for extra hardware peripherals like spare keyboards and monitors. It also lets you leverage your host computer's storage capacity and processing power to streamline your development workflow.

1.6 Chapter Summary

This chapter covered fundamental operating system architecture concepts, core features of the Linux operating system, and standard terminal workflows. Review and master the essential commands summarized in Table 1.6 below.

1.5.3 远程网络存储（NFS）

嵌入式设备通常内置存储空间有限。您可以使用**网络文件系统（NFS）**将开发电脑中的目录共享给树莓派，从而扩展可用存储空间。

首先，通过在 `/etc/exports` 目录下添加共享目录路径及访问权限，将您的电脑配置为NFS服务器：

```
/srv/nfs4 192.168.*(rw, sync, subtree_check, no_root_squash)
```

假设您的电脑网络IP地址为 `192.168.200.170`，请以root用户身份登录树莓派，并将共享目录挂载到本地文件夹路径中：

```
# 目录挂载: 192.168.200.170:/srv/nfs4/mnt -o no lock, proto=tcp
```

该操作将PC的 `/srv/nfs4` 目录直接映射至Raspberry Pi的 `/mnt` 文件夹。两台设备可同时访问完全相同的文件，而实际数据仍存储在PC的硬盘中。通过 `-o proto=tcp` 参数设置，连接必须采用TCP协议，从而避免在复制大文件时因NFS默认UDP传输方式可能导致的数据损坏或性能下降问题。

在嵌入式开发过程中使用网络连接，可避免使用备用键盘、显示器等额外硬件外设。同时还能充分利用主机计算机的存储容量和处理能力，从而优化开发工作流程。

1.6 章节摘要

本章涵盖了操作系统架构的基本概念、Linux操作系统的核心功能以及标准终端工作流程。请回顾并熟练掌握下表1.6中总结的关键命令。

Table 1.6: Summary of Core Linux Commands

Command Name	Core System Function	Common Practice Option Flags
ls	Lists files and subdirectory entries	'-l' (long format), '-a' (show hidden files), '-i' (show inodes)
more/less	Displays text file contents page by page	Navigational keys, text search parameters
cp	Copies target files or folder pathways	'-r' (recursive directory copy), '-i' (interactive prompt)
cd	Changes the shell's active working directory	Target path argument (defaults to home directory '~')
mv	Moves or renames files and directories	'-i' (interactive alert prompts)
mkdir	Generates brand new directory structures	'-p' (automatically creates missing parent folders)
rmdir	Deletes empty target directories	Only executes successfully if the directory is completely empty
rm	Deletes target files permanently	'-i' (interactive prompt), '-f' (force), '-r' (recursive purge)
ps	Displays active system execution processes	'ax' (lists all system processes), 'aux' (includes user details)
tar	Bundles, compresses, and extracts archives	'-c' (create), '-x' (extract), '-v' (verbose), '-f' (archive file)

表 1.6: 核心 Linux 命令概览

命令名称	核心系统功能	常见操作选项标志
ls	列出文件和子目录 条目	ζ-l ζ (长格式), ζ-aζ (显示 隐藏文件)、ζ-iζ (显示文件标识
或多或少; 大致上	逐页显示文本文件内容	符)、导航键、文本搜索参数
cp	复制目标文件或文件夹路径	ζ-rζ (递归目录复制), ζ-iζ (交互 式提示)
cd	更改 Shell 的活动工作目录	目标路径参数 (默认为家目录 "~")
mv	移动或重命名文件和目录	ζ-i ζ (交互式提示)
命令	生成全新的目录结构	ζ-pζ (自动创建缺失的父级文件夹)
目录重命名	删除空的目标目录	仅在目录完全为空时才能成功执行
rm	永久删除目标文件	ζ-i ζ (交互提示)、ζ-fζ (强制执 行)、ζ-rζ (递归清除)
ps	显示活动的系统执行进程	ζax ζ (列出所有系统进程), ζaux ζ (包含用户详细信息)
焦油	打包、压缩和解压归档文件	ζ-c ζ (创建)、ζ-x ζ (提取)、ζ-v ζ (详细)、ζ-fζ (归档文件)

Chapter 2

Very Short Python Introduction

2.1 Introduction

Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Van Rossum developed the language while working as a researcher at CWI (Centrum Wiskunde & Informatica, English: Center for Mathematics and Computer Science) in the Netherlands. In December 1989, van Rossum was looking for a “hobby programming project that would keep him occupied during the week around Christmas” when his office was closed.

Python’s name is derived from the British comedy group *Monty Python’s Flying Circus*, which Guido van Rossum enjoyed watching while developing the language.



Guido van Rossum

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are natively supported, and many of its features facilitate functional programming and aspect-oriented programming. Many other development paradigms are supported via ecosystem extensions, including design-by-contract and logic programming.

Large-scale organizations that rely heavily on Python include Wikipedia, Google, Yahoo!, CERN (French abbreviation for the European Organization for Nuclear Research), NASA (National Aeronautics and Space Administration), Facebook, Amazon, Instagram, and Spotify. The social news and aggregation platform Reddit is written almost entirely in Python. Additionally, Python has been successfully embedded into numerous commercial and open-source software products as a robust scripting runtime.

第 2 章

《极简版Python入门指南》

2.1 引言

Python是一种解释型、高级、通用的编程语言。

该语言由Guido van Rossum创建，于1991年首次发布。van Rossum在荷兰CWI（数学与计算机科学中心）担任研究员期间开发了此语言。1989年12月，当他的办公室关闭时，van Rossum正在寻找一个“能在圣诞节前后的一周内让他忙个不停”的业余编程项目。

Python这个名字来源于英国喜剧团体蒙提·派森飞行马戏团，

Guido van Rossum在开发这种语言时喜欢观看他们的表演。



圭多·范·罗苏姆

Python是一种支持多种编程范式的语言。它原生支持面向对象编程和结构化编程，其众多特性也有利于函数式编程和面向切面编程的实现。通过生态系统扩展，Python还支持包括契约式设计和逻辑编程在内的多种其他开发范式。

众多大型组织都高度依赖Python语言，包括维基百科、谷歌、雅虎、欧洲核子研究组织（CERN，法语缩写）、美国国家航空航天局（NASA）、脸书、亚马逊、Instagram以及Spotify。社交新闻聚合平台Reddit几乎完全采用Python编写。此外，Python还作为一款功能强大的脚本运行环境，成功被集成到众多商业软件和开源软件产品中。

In modern Linux environments, a large portion of system administration tools and application software packages are written entirely in Python. Reflecting this massive growth, Python received the prestigious TIOBE Programming Language of the Year award in 2007, 2010, 2018, and 2020—an honor granted to the language demonstrating the highest annual popularity growth momentum.

2.2 Python Environment

The official Python website (<https://www.python.org/downloads>) provides pre-compiled installation binaries for all mainstream operating systems. Alternatively, developers can download the complete source code to configure, modify, or compile the interpreter locally. Building from source can be a resource-intensive and time-consuming process; it is generally unnecessary unless you are tailoring the runtime for a custom, highly constrained embedded system. Because Python is fundamentally cross-platform software, its operational behavior is consistent regardless of the underlying operating system architecture.

Most program development requires specialized tooling, and Python is no exception. However, because Python’s syntax design is highly minimal and clean, a well-configured text editor can easily serve as an Integrated Development Environment (IDE). Python is an essential dependency within modern Linux ecosystems; consequently, the vast majority of Linux distributions ship with a Python interpreter installed by default.

The standard development environment bundled with Python on Linux is IDLE (*Integrated Development and Learning Environment*), which is itself written entirely in Python using the `tkinter` GUI toolkit. Its core wrapper script contains only a few initialization lines:

Listing 2.1: `idle3`, Python’s Default IDE Wrapper

```
#!/usr/bin/python3

from idlelib.PyShell import main
if __name__ == '__main__':
    main()
```

This environment includes both a Python interactive shell and a script editor that features full Python syntax highlighting. Even without installing IDLE, developers can invoke the interactive interpreter directly from any standard Linux command shell by executing the `python3` binary. This interactive shell is ideal for learning the language

在现代Linux环境中，绝大多数系统管理工具和应用程序软件包均完全采用Python编写。这一迅猛发展势头使得Python在2007年、2010年、2018年及2020年连续荣获享有盛誉的 TIOBE 年度编程语言大奖——该奖项专门授予年度流行度增长势头最为显著的语言。

2.2 Python环境

Python官方网站 (<https://www.python.org/downloads>) 为所有主流操作系统提供了预编译的安装二进制文件。开发者也可下载完整的源代码，在本地进行配置、修改或编译解释器。从源代码构建程序通常需要大量资源且耗时较长；除非您需要为定制化、硬件条件极为受限的嵌入式系统定制运行环境，否则这一过程通常并非必要。由于Python本质上是一款跨平台软件，其运行行为在不同底层操作系统架构下均保持一致。

大多数程序开发都需要专用工具支持，Python也不例外。不过，由于Python的语法设计极为简洁明了，配置得当的文本编辑器完全可以充当集成开发环境（IDE）。Python是现代Linux生态系统中不可或缺的核心组件；因此，绝大多数Linux发行版都会默认预装Python解释器。

Linux系统中随Python预装的标准开发环境是IDL（集成开发与学习环境），该环境完全采用Python语言并基于 tkinter图形用户界面工具包编写。其核心封装脚本仅包含少量初始化代码：

列表 2.1: idle3——Python 的默认集成开发环境封装器

```
/或使用 `r/bin/pythons3`  
  
从空闲库文件。PyShell 导入 main if __name__ ==  
    ' __main__ ' :  
        主要的 ()
```

该环境同时包含一个Python交互式外壳和一个具备完整Python语法高亮功能的脚本编辑器。即使未安装IDL，开发者也可以通过运行python3二进制文件，直接从任意标准Linux命令行外壳调用该交互式解释器。此交互式外壳非常适合用于学习Python语言。

syntax, inspecting APIs, and conducting rapid micro-benchmarks. However, it is not practical for building complex applications, as code modifications are cumbersome and the entered lines cannot be saved directly to a persistent file for later execution.

2.3 Python Basics

2.3.1 Running Your First Python Program

The traditional entry point for learning Python is the classic “Hello, world!” script:

```
#!/usr/bin/python3
print("Hello, world!")
```

This minimal script contains only two lines. When writing scripts, ensure that no accidental leading whitespaces exist at the beginning of these execution lines.

To execute this source code on a computer, it must first be saved into a file, such as `hello.py`. The `.py` extension explicitly designates it as a Python source file. On Windows systems, double-clicking this file tells the operating system shell to pass it to the registered Python interpreter binary. On Linux and other UNIX-like operating systems (such as macOS), there are three primary methodologies for running a Python program:

1. **Explicit Interpreter Invocation via the Command Line:**

```
$ python3 hello.py
```

The interpreter parses the script, executes the commands sequentially, and prints `Hello, world!` directly to the standard output stream of the terminal.

2. **Direct Script Execution via Executable Permissions:**

First, modify the file’s access permissions to grant execute attributes using the system utility:

```
$ chmod +x hello.py
```

Once the executable bit is set, the script can be invoked directly by specifying its relative path under the current working directory:

```
$ ./hello.py
```

The prefix `./` explicitly denotes the current directory. When executed this way, the Linux program loader reads the first line of the script. The special `#!` character

支持语法分析、API检查以及快速微基准测试。然而，该方法不适用于构建复杂应用程序，因为代码修改过程繁琐，且输入的代码行无法直接保存至持久化文件以供后续执行。

2.3 Python基础

2.3.1 运行您的第一个 Python 程序

学习Python的传统入门方式是经典的“Hello, world!”脚本：

```
/或使用 `r/bin/python3`  
print( "Hello , world ! " )
```

此最小脚本仅包含两行代码。编写脚本时，请确保执行行开头不存在意外的空格。

要在计算机上运行这段源代码，首先需要将其保存为文件（例如hello.py）。.py扩展名明确表明该文件是Python源文件。在Windows系统中，双击该文件即可让操作系统外壳将其传递给已注册的Python解释器二进制文件；而在Linux及其他类UNIX操作系统（如macOS）中，运行Python程序主要有三种方法：

1. 通过命令行显式调用解释器：

```
$ Python 3 hello.py
```

解释器会解析脚本，按顺序执行命令，并将“Hello, world!”输出至终端的标准输出流。

2. 通过可执行权限直接运行脚本：

首先，使用系统工具修改文件的访问权限，授予执行属性：

```
$ chmod +x hello.py
```

一旦可执行位被设置，即可通过指定当前工作目录下的相对路径直接调用该脚本：

```
$ ./你好.py
```

前缀 ./ 明确表示当前目录。以这种方式运行时，Linux 程序加载器会读取脚本的第一行。特殊的 #! 字符

sequence (known as a **shebang**) indicates that the rest of the line specifies the path to the interpreter binary responsible for parsing the file.

3. **Graphical Execution:** If a desktop environment is active on the Linux target, the file can be executed by clicking its icon within the file manager, provided the file manager is configured to run Python scripts automatically.

In Python source files, a single physical line can host multiple independent statements if they are explicitly separated by a semicolon (;). The trailing semicolon at the absolute end of a line is optional. However, placing multiple statements on a single line is discouraged by Python style guidelines unless the individual operations are exceptionally concise.

Unlike many popular programming languages that use curly braces ({}) or keyword blocks to isolate execution scopes, Python relies strictly on leading whitespace indentation to delimit block structures. An increase in leading indentation denotes the beginning of a nested block, whereas a decrease back to the matching column alignment signifies the closure of the current scope block. Consequently, a Python program's visual arrangement directly represents its logical and semantic execution structure.

2.3.2 Interactive Mode

Executing `python3` or appending an ampersand to launch IDLE in the background (`idle3 &`) opens an interactive terminal session. When the standard primary prompt (`>>>`) appears, you can enter commands to be executed immediately:

```
Python 3.8.2 (default, Jul 20 2020, 22:30:15)
[GCC 9.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, Python!")
Hello, Python!
>>>
```

While the interactive shell does not replace a structured script development environment, it serves as an excellent utility for testing small logic snippets, exploring module exports, and reading documentation. The built-in `help()` function allows developers to inspect the interactive reference manuals for any core keyword, function, or loaded software module.

IDLE acts as a lightweight graphical wrapper for this loop. Developers can open a fresh, dedicated script file editor via the menu navigation path **File** → **New File**, write their

该序列（称为**shebang**）表示后续代码行用于指定负责解析文件的解释器二进制程序的路径。

3. **图形化执行**：若Linux目标系统上存在活跃的桌面环境，则可通过在文件管理器中点击该文件图标来运行该文件，前提是文件管理器已配置为自动运行Python脚本。

在Python源文件中，单行代码若使用分号（；）明确分隔，即可容纳多个独立语句。行末绝对位置的分号可选；但根据Python风格指南，除非具体操作极其简洁，否则不建议在同一行编写多个语句。

与许多使用大括号（{ }）或关键字块来划分执行范围的流行编程语言不同，Python完全依赖前导空格缩进来界定代码块结构：缩进程度增加表示嵌套代码块的开始；缩进程度减小至与列对齐时，则标志着当前代码块的结束。因此，Python程序的视觉布局直接反映了其逻辑与语义上的执行结构。

2.3.2 交互模式

运行Python 3或在命令末尾添加&符号（即idle3&）可在后台启动IDLE，从而打开交互式终端会话。当标准提示符（>>>）出现时，即可输入立即执行的命令：

```
Python 3.8.2 (默认版本, 2020年7月20日, 22:30:15)
[GCC 9.2.0] 在 Linux 上

如需更多信息, 请输入 "help" 、 "copyright" 、 "credits" 或 "license" 。>>> print( "你好, Python! " )

你好, Python!
>>>
```

虽然交互式外壳无法替代结构化的脚本开发环境，但它却是测试小型逻辑片段、探索模块导出内容以及阅读文档的绝佳工具。内置的**帮助()**功能可让开发者查阅交互式参考手册中关于任何核心关键词、函数或已加载软件模块的信息。

IDLE可作为该循环的轻量级图形化封装工具。开发者可通过菜单路径“文件→新建文件”打开专用的脚本文件编辑器，并编写所需代码。

source code, and execute the complete file within the shell by pressing the F5 shortcut or choosing Run → Run Module.

2.3.3 Numerical Variables

Constants

Python handles three distinct classifications of numeric constants: integers (**int**), floating-point decimals (**float**), and complex mathematical structures (**complex**). For example, 2 is processed natively as an integer; 2.5 and 2. are interpreted as floating-point numbers; and 2j or 2.5J represent imaginary numerical constants.

- Arithmetic operations (addition, subtraction, and multiplication) executed entirely between integers produce an integer result. However, division behavior is split into two specialized operators: floating-point division (/), which always evaluates to a **float**, and floor division (//), which truncates the fractional portion to return an integer value.
- The imaginary unit suffix symbol (j or J) must immediately follow the numeric coefficient. No multiplication operator (*) can be placed between the coefficient and the suffix, because an isolated j is interpreted by the parser as a standard variable name identifier.
- Python uses the double asterisk symbol (**) as its native exponentiation operator.

Consider the following interactive command stream examples:

```
>>> 12345678 + 87654321
99999999
>>> 1000 / 330
3.0303030303030303
>>> (1 + 2j) * (3 - 2j)
(7+4j)
>>> 2.8 // 0.03
93.0
>>> (1 + j) * (1 - j)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'j' is not defined
>>> (-2) ** 0.5
(8.659560562354934e-17+1.4142135623730951j)
```

打开源代码文件，通过按下F5快捷键或选择运行→运行模块，在shell中执行整个文件。

2.3.3 数值变量

常量

Python 对数值常量分为三种不同的类型：整数（**int**）、浮点数（**float**）以及复数结构（**complex**）。例如，2 被原生处理为整数；2.5 和 2.0 被解释为浮点数；而 2j 或 2.5J 则表示虚数常量。

- 完全在整数之间执行的算术运算（加法、减法和乘法）均会产生整数值结果。然而，除法运算则分为两种专用操作符：浮点除法（/），其结果始终为浮点数；以及取整除法（//），它会截断小数部分以返回整数值。
- 虚数单位后缀符号（j 或 J）必须紧接在数值系数之后。系数与后缀之间不得插入乘法运算符（*），因为解析器会将孤立的 j 视为标准变量名称标识符。
- Python将双星号符号（**）作为其内置的幂运算符。请参考以下交互式命令流示例：

```
>>> 12345678 + 87654321
99999999
>>> 1000 / 330
3.0303030303030303
>>> (1 + 2j) * (3 - 2j)
(7+4j)
>>> 2.8 // 0.03
93.0
>>> (1 + j) * (1 - j)
回溯（最近一次调用时间）：
  文件 "<stdin>", 第 1 行, 位于 <module> 中"
名称错误: 名称 "j" 未定义
>>> (-2) ** 0.5
(8.659560562354934e-17+1.4142135623730951j)
```


Python also natively converts alternative numerical radix bases. Constants prefixed with `0b`, `0o`, or `0x` (case-insensitive) represent binary, octal, and hexadecimal values, respectively:

```
>>> 0xff - 0b1111
240
```

Variables

A variable is a symbolic name or reference pointing to an underlying data object. Python does not require explicit variable type declarations; variables are bound dynamically when an assignment statement (the single equals sign operator, `=`) is evaluated. Accessing an unassigned variable identifier will halt execution and raise a `NameError`:

```
>>> a1 = 10
>>> a2 = a2 + a1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a2' is not defined
>>> a2 = 20
>>> a2 = a2 + a1
>>> a1, a2
(10, 30)
```

Variable identifiers can consist of uppercase and lowercase letters, numeric digits, and underscores (`_`). However, an identifier cannot begin with a numeric digit; hence, `Zhang3` is a valid variable name, whereas `3Zhang` is syntactically invalid. Python 3 permits the use of arbitrary Unicode characters (such as Chinese characters) for variable identifiers, though this practice is generally discouraged in professional software engineering.

```
>>> 张三的年龄 = 32
>>> 李四 = 张三的年龄 + 3
>>> print("李四的年龄是 ", 李四)
李四的年龄是  35
```

Python also supports elegant sequence unpacking syntax, making object state swapping highly intuitive without requiring temporary variables:

```
>>> a, b = 200, 'hello'
>>> b, a = a, b
>>> print('a=%s,' % a, 'b=%d' % b)
```

Python还能原生转换不同的数值进制。以0b、0o或0x（不区分大小写）开头的常量分别表示二进制、八进制和十六进制值：

```
>>> 0xff - 0b1111
240
```

变量

变量是一个指向底层数据对象的符号名称或引用。Python无需显式声明变量类型；变量会在赋值语句（即单个等号运算符=）执行时动态绑定。尝试访问未赋值的变量标识符将导致程序执行中断并抛出 `NameError` 错误。

```
>>> a1 = 10
>>> a2 = a2 + a1
回溯（最近一次调用时间）：
  文件 "<stdin>", 第 1 行, 位于 <module> 中"
名称错误: 名称 'a2' 未定义
>>> a2 = 20
>>> a2 = a2 + a1
>>> a1, a2
(10, 30)
```

变量标识符可由大写字母、小写字母、数字以及下划线（`_`）组成。但标识符不能以数字开头；因此，`Zhang3` 是有效的变量名称，而 `3Zhang` 在语法上是无效的。Python 3 允许使用任意Unicode字符（如中文字符）作为变量标识符，不过这种做法在专业软件开发中通常不被推荐。

```
>>> 张三的年龄 = 32
>>> 李四 = 张三的年龄 + 3
>>> print("李四的年龄是", 李四)
李四的年龄是 35
```

Python还支持优雅的序列解包语法，使得对象状态交换无需使用临时变量即可实现高度直观的操作。

```
>>> a, b = 200, "你好"
>>> b, a = a, b
>>> 打印 ('a=%s' , %a; 'b=%d' , %b)
```

```
a=hello, b=200
```

In Python, the percent token (%) serves multiple contextual syntactic duties. When placed between string templates and matching element data variables, it performs classic C-style printf string interpolation formatting:

```
>>> "%4x" % 123
' 7b'          # Hexadecimal string, right-aligned with space padding
>>> "%04X" % 456
'01C8'         # Hexadecimal string, padded with leading zeros
```

Table 2.1: String Format Specifiers in Python

Specifier	Target Data Type Representation
%s	String conversion representation
%d, %i, %u	Signed or unsigned decimal integer formats
%o	Octal integer format
%x, %X	Hexadecimal integer format (lowercase/uppercase)
%e, %E	Scientific exponential floating-point representation
%f, %F	Standard decimal floating-point formatting

When applied directly between two numerical values, the % operator calculates the arithmetic modulo remainder (e.g., 5 % 3 returns 2).

2.3.4 Strings

String variables store arbitrary textual data and sequence characters.

```
>>> greeting = "Hello, Python!"
>>> print(greeting)
Hello, Python!
>>> number = 35
>>> number + number
70
>>> string = "35"
>>> string * 3
'353535'
```

Strings can be wrapped interchangeably in single quotes (' approach) or double quotes (" approach). Triple quotes (""" or ''') are used to handle multi-line string

```
a = 你好, b = 200
```

在Python中，百分号（%）具有多种上下文相关的语法功能。当置于字符串模板与对应的元素数据变量之间时，它可实现经典的C语言风格printf字符串插值格式化效果：

```
>>> "%4x" % 123
'7b' # 十六进制字符串，右对齐并使用空格填充 >>> "%04X" % 456
'01C8' # 十六进制字符串，以前导零填充
```

表 2.1: Python 中的字符串格式指定符

限定词	目标数据类型的表示方式
%s	字符串转换表示法
%d, %i, %u	带符号与不带符号的十进制整数格式
%o	八进制整数格式
%x, %X	十六进制整数格式（小写/大写）
%e, %E	科学指数浮点表示法
%f, %F	标准十进制浮点格式

当直接应用于两个数值之间时，%运算符可计算算术模余（例如：5 % 3 的结果为 2）。

2.3.4 字符串

String variables store arbitrary textual data and sequence characters.

```
>>> greeting = "你好, Python!"
>>> 打印（问候语）
Hello, Python!
>>> number = 35
>>> number + number
70
>>> string = "35"
>>> string * 3
'353535'
```

Strings can be wrapped interchangeably in single quotes (' approach) or double quotes (" approach). Triple quotes (""" or ''') are used to handle multi-line string

blocks or text sections that span multiple document line breaks. Special control characters embedded inside strings can be escaped using a leading backslash (`\`) character:

```
>>> "\"Hello, world!\" she said"
'"Hello, world!" she said'
```

Python provides specialized string literals using string prefixes, including raw strings (`r`), legacy Unicode strings (`u`), and byte arrays (`b`):

```
>>> rStr = r"Python is easy."
>>> uStr = u'Python isn\'t hard to learn.'
>>> bStr = b'Hello!'
```

When dealing with plain ASCII text characters, standard strings and explicit Unicode representations are practically identical. However, when building systems that use non-English character sets (such as Chinese), developers must carefully manage text encodings. Modern Python 3 handles all standard string symbols as internal Unicode. To prevent configuration or decoding errors when script files are processed across different operating systems, we highly recommend inserting the following two header lines at the very top of your Python source files:

```
#!/usr/bin/python3
# -*- coding: utf-8 -*-
```

While the standard hash character (`#`) denotes a code comment, these specific opening lines tell the operating system loader which interpreter binary to use, and tell the Python compiler to parse the source file using the UTF-8 character encoding format. Mixed or mismatched character encodings will cause data corruption or lead to runtime errors during text output.

Crucially, Python strings are fundamentally **immutable**. Once created, individual character positions inside a string object cannot be modified directly; modifying a string requires generating a new string object through re-assignment or manipulation functions:

```
>>> my_str = 'hello'
>>> print(my_str[0])
h
>>> my_str[0] = 'H'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

跨越多个文档换行的段落或文本区域。嵌入字符串中的特殊控制字符可通过前置反斜杠（\）进行转义：

```
"你好，世界！" 她说。  
"你好，世界！" 她说。
```

Python通过字符串前缀提供了专门的字符串字面量，包括原始字符串(r)、传统Unicode字符串(u)以及字节数组(b)：

```
>>> rStr = r "Python非常简单。"  
>>> uStr = u '学习 Python 并不难。'  
>>> bStr = b '你好！'
```

在处理纯ASCII文本字符时，标准字符串与显式Unicode表示形式几乎完全一致。然而，在开发使用非英语字符集（如中文）的系统时，开发者必须谨慎管理文本编码。现代Python 3会将所有标准字符串符号均视为内部Unicode编码。为避免脚本文件在不同操作系统中处理时出现配置或解码错误，我们强烈建议在Python源文件的最顶部插入以下两条头文件声明：

```
/或使用 `r/bin/pythons3`  
#-*- 编码: utf-8 -*-
```

虽然标准哈希字符（#）表示代码注释，但这些特定的开头行会告知操作系统加载器应使用哪个解释器二进制文件，并指示Python编译器采用 UTF -8字符编码格式解析源文件。字符编码混用或不匹配将导致数据损坏或在文本输出时引发运行时错误。

关键在于，Python字符串本质上是**不可变的**。一旦创建，字符串对象中的各个字符位置都无法直接修改；若需修改字符串，必须通过重新赋值或调用相关操作函数来生成新的字符串对象：

```
>>> my_str = '你好'  
>>> 打印 ( my_str[0]  
h  
>>> my_str[0]= 'H'  
回溯（最近一次调用时间）：  
  文件 "<stdin>"，第 1 行，位于 <module> 中"  
错误类型： 'str' 对象不支持项目赋值
```

2.3.5 Boolean Logic

The Boolean primitive data type handles logical states and contains only two distinct values: `True` and `False`. In conditional evaluations, Python interprets the following values as falsy (evaluating to `False`):

1. Numerical zero structures of any numeric type (0, 0.0, 0j).
2. Strings with a length of zero ("" or '').
3. Empty collection sequences (empty dictionaries {}, sets `set()`, lists [], or tuples ()).
4. The special empty single-instance placeholder object: `None`.
5. The primitive constant value `False` itself.

All other structural instances or valid objects evaluate to `True`. Both logical states belong explicitly to the foundational core class `bool`:

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

The special value `None` represents the absence of a value or a null pointer state. When a function completes execution without encountering an explicit `return` statement, it returns `None` to the caller. Boolean expressions are primarily used in comparison operations, logical combinations, and conditional control flows:

```
>>> x = 123
>>> if x > 0:
...     print("x is greater than zero.")
x is greater than zero.
>>>
```

Python provides a complete set of relational and comparison operators:

2.3.5 布尔逻辑

布尔基本数据类型用于表示逻辑状态，仅包含两个不同的值：**True**（真）和**False**（假）。在条件判断中，Python会将以下值视为假值（即计算结果为False）：

1. 任意数值类型的数值零结构（0、0.0、0j）。
2. 长度为零的字符串（`""` 或 `''`）。
3. 空收集序列（空字典 `{}`、集合`set()`、列表`[]`或元组`()`）。
4. 特殊的空单实例占位符对象：无。
5. 原始常量值 **False** 本身。

所有其他结构实例或有效对象均评估为 **True**。这两种逻辑状态均明确属于基础核心类 **bool**：

```
>>> 类型（真）  
类 '布尔值' >  
>>> 类型（False）  
类 '布尔值' >
```

特殊值 **None** 表示不存在值或处于空指针状态。当函数在未遇到显式**return**语句的情况下完成执行时，会向调用方返回 **None**。布尔表达式主要用于比较运算、逻辑组合以及条件控制流程中：

```
>>> x = 123  
>>> 如果 x > 0:  
...print( "x 大于零。" )  
x 大于零。  
>>>
```

Python 提供了一整套关系运算符和比较运算符：

Operator Flag	Computational Condition and Operational Purpose
<code>==</code>	Evaluates whether the values of two objects are equal
<code>!=</code>	Evaluates whether the values of two objects are unequal
<code>></code>	Strict greater-than comparison
<code><</code>	Strict less-than comparison
<code>>=</code>	Greater-than-or-equal-to comparison
<code><=</code>	Less-than-or-equal-to comparison
<code>is</code>	Evaluates object identity (checks if both variables reference the exact same memory address)
<code>is not</code>	Evaluates if two variables reference distinct underlying objects

Complex conditional logic can be constructed using the three text-based operators: **and**, **or**, and **not**. Their evaluation behaviors follow standard mathematical logic rules. For example, the statement:

```
x > 0 and x < 10
```

evaluates to **True** if and only if **x** is simultaneously greater than zero and less than ten.

Python also natively supports chained comparison expressions, matching standard mathematical notation:

```
0 < x < 10
```

This chained expression functions exactly like the combined logical statement above. When evaluated within numeric contexts or arithmetic equations, Boolean states are treated as numbers, where **True** maps to 1 and **False** maps to 0.

2.3.6 Functions

A function is a reusable block of structured code designed to perform a specific computational task. Python includes an extensive collection of globally accessible, pre-configured tools known as “built-in” functions. In addition to these built-in utilities, developers can define custom functions to handle specialized tasks.

For example, consider the built-in mathematical function **pow()**, which duplicates the behavior of the native exponentiation operator (******):

```
>>> pow(-1, 0.5)
(6.123233995736766e-17+1j)
```

操作员标志	计算条件与操作目的
<code>==</code>	评估两个对象的值是否相等
<code>!=</code>	评估两个对象的值是否不相等
<code>></code>	严格的大于比较
<code><</code>	严格的小于比较
<code>>=</code>	大于或等于比较
<code><=</code>	小于或等于比较
是	评估对象身份（检查两个变量是否引用完全相同的内存地址）
并非如此。	评估两个变量是否引用不同的底层对象

可使用三种基于文本的运算符构建复杂的条件逻辑：**和**、**或**以及**非**。这些运算符的求值行为遵循标准数学逻辑规则。例如，以下语句：

```
x > 0 且 x < 10
```

当且仅当 `x` 同时大于零且小于十时，结果评估为 `True`。

Python还原生支持链式比较表达式，其语法与标准数学符号完全一致：

```
0 < x < 10
```

该链式表达式的功能与上述组合逻辑语句完全一致。在数值上下文或算术方程中评估时，布尔状态被视为数值：真值对应1，假值对应0。

2.3.6 功能

函数是一种可重复使用的结构化代码块，旨在执行特定的计算任务。Python 提供了大量全局可访问、预先配置好的工具，称为“内置”函数。除这些内置工具外，开发者还可定义自定义函数来处理专门任务。

例如，考虑内置数学函数`pow()`，其功能与原生幂运算符（`**`）完全一致：

```
>>> pow(-1, 0.5)
(6.123233995736766e-17+1j)
```

We pass parameters (arguments) into the function call (in this instance, the arguments are `-1` and `0.5`); the function executes its internal instructions and passes a computed result back to the caller. Individual functions accept varying numbers or types of arguments and return different data types depending on their purpose.

2.3.7 Order of Operations

When an expression contains multiple operators, Python determines the execution order using established operator precedence rules. For standard mathematical operators, Python follows conventional algebraic rules. The common mnemonic acronym **PEMDAS** (**P**arentheses, **E**xponentiation, **M**ultiplication/**D**ivision, **A**ddition/**S**ubtraction) summarizes these precedence levels:

- **Parentheses** possess the highest priority. They can enforce a specific execution order regardless of standard precedence rules, or clarify complex equations to improve code readability (e.g., `(minute * 100) / 60`).
- **Exponentiation** has the next highest priority level. Consequently, an expression like `2**1+1` evaluates to 3 (not 4), and `3*1**3` evaluates to 3 (not 27).
- **Multiplication and Division** share identical precedence levels, which are lower than exponentiation but higher than **Addition and Subtraction**.
- Operators that share the exact same precedence level are evaluated sequentially from left to right.

When writing complex equations, we highly recommend using explicit grouping parentheses to ensure that operations execute in the intended order and to avoid subtle logic bugs.

Bitwise Operators

Bitwise operators manipulate integer values at the level of individual raw bits. Instead of treating an integer as a single numeric scalar value, these operators treat the value as a sequential string of bits encoded in two's-complement binary format.

Compound Assignment Operators

In addition to the standard assignment operator (`=`), Python provides compound assignment shortcuts that combine an arithmetic or bitwise operation with an assignment step, making the code more concise.

我们将参数（即参数值）传递给函数调用（本例中参数为 -1 和 0.5）；函数执行其内部指令后，将计算结果返回给调用者。不同函数接受的参数数量或类型各不相同，并根据其用途返回不同的数据类型。

2.3.7 作战顺序

当表达式包含多个运算符时，Python会根据既定的运算符优先级规则来确定执行顺序。对于标准数学运算符，Python遵循传统的代数规则。常用的助记缩写**PEMD-AS**（**括**号、**幂**运算、**乘**法/除**法**、**加**法/减**法**）概括了这些优先级等级：

- **括号**具有最高优先级。它们能够无视标准优先级规则强制执行特定的执行顺序，或用于简化复杂表达式以提升代码可读性（例如：(分钟 * 100) / 60）。
- **指数运算**的优先级次之。因此，表达式 $2^{**}1+1$ 的计算结果为 3（而非 4），而 $3*1^{**}3$ 的计算结果也为 3（而非 27）。
- **乘法与除法**具有相同的运算优先级，其优先级低于幂运算，但高于**加法与减法**。
- 具有完全相同优先级的运算符将从左至右依次进行评估。

在书写复杂方程时，我们强烈建议使用显式括号分组，以确保运算按预期顺序执行并避免出现细微的逻辑错误。

位运算符

位运算符在单个原始比特层面操作整数值。这些运算符并非将整数视为单一数值标量，而是将其视为以二补码二进制格式编码的连续比特序列。

复合赋值运算符

除标准赋值运算符（=）外，Python还提供了复合赋值快捷方式，可将算术运算或位运算与赋值步骤结合使用，从而使代码更加简洁。

Table 2.2: Operator Precedence Hierarchy in Python Expressions

Operator Group	Computational Operational Meaning	Precedence Hierarchy Level
**	Exponentiation power scaling	Highest priority
*, /, //, %	Multiplicative, division, floor division, modulo calculations	
+, -	Basic arithmetic addition and subtraction	
==, !=	Value equality and inequality evaluations	
>, <, >=, <=, is, in	Relational comparison and collection inclusion testing	
not, and, or	Logical Boolean conjunctions and operators	Lowest priority

Table 2.3: Bitwise Operators and Expressions

Operator	Functional Meaning	Syntax Example and Practical Use Case
&	Bitwise AND operation	x & 1 (Isolates the Least Significant Bit)
 	Bitwise OR operation	x 1 (Forces the LSB to state 1)
^	Bitwise Exclusive OR (XOR)	x ^ 0xf (Inverts the final 4 bits of the variable)
~	Bitwise Unary Complement (NOT)	y = ~x (Inverts all bits of the target integer)
<<	Left logical bit shift	x << 2 (Shifts bits left twice, padding vacant LSB positions with zeros)
>>	Right arithmetic bit shift	x >> 2 (Shifts bits right twice, preserving the original sign bit)

These compound assignments are single, atomic lexical tokens; therefore, no

表 2.2: Python 表达式中的运算符优先级层次结构

操作员组	计算操作的意义	优先级层级
**	指数运算的幂次缩放：乘法、除法、	最高优先级
*, /, //, %	取整除法、模运算	
+, -	基本算术加法与减法	
==, !=	值相等与不等的判断	
>, <, >=, <=表示:	关系比较与集合包含性检验	
不; 或者; 或	逻辑布尔合取运算符与操作符	最低优先级

表 2.3: 位运算符与表达式

经营者	功能意义	语法示例及实际应用场景
&	按位与运算	x & 1 (分离出最小值)重要位)
	按位或运算	x 1 (强制 LSB 设置为状态1)
^	按位异或运算 (XOR)	x ^ 0xf (反转变量的最后4位)
~	位元一元补码 (NOT)	y ≈ x (将目标整数的所有位进行反转)
<<	左逻辑位移	x << 2 (将位向左移动两次, 并用零填充空 LSB 位置)
>>	正确的算术位移	x >> 2 (将位向右移动两次, 同时保留原始符号位)

这些化合物指代均为单一的原子级词汇标记；因此，无需进一步处理。

Table 2.4: Compound Assignment Statements

Operator Shortcut	Explicit Structural Breakdown	Equivalent Verbose Expression
<code>+=</code>	<code>a += b</code>	<code>a = a + b</code>
<code>-=</code>	<code>a -= b</code>	<code>a = a - b</code>
<code>*=</code>	<code>a *= b</code>	<code>a = a * b</code>
<code>/=</code>	<code>a /= b</code>	<code>a = a / b</code>
<code>%=</code>	<code>a %= b</code>	<code>a = a % b</code>
<code>//=</code>	<code>a //= b</code>	<code>a = a // b</code>
<code>**=</code>	<code>a **= b</code>	<code>a = a ** b</code>
<code>&=</code>	<code>a &= b</code>	<code>a = a & b</code>
<code> =</code>	<code>a = b</code>	<code>a = a b</code>
<code>^=</code>	<code>a ^= b</code>	<code>a = a ^ b</code>
<code><<=</code>	<code>a <<= b</code>	<code>a = a << b</code>
<code>>>=</code>	<code>a >>= b</code>	<code>a = a >> b</code>

whitespaces or characters are permitted between the operator symbol and the equals sign.

Advanced Membership and Identity Operators

Python includes two specialized operators: **is** and **in**. The identity operator (**is**) tests whether two variables point to the exact same underlying object in memory. In contrast, the comparison operator (**==**) evaluates whether the values stored inside those two objects are equal:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
True
>>> a is b
False
>>> c = a
>>> a is c
True
>>> a[0] = 9
>>> print(c)
[9, 2, 3]
```

表 2.4：化合物赋值语句

操作员快捷键	明确的结构分解	等效的动词表达方式
<code>+=</code>	<code>a += b</code>	<code>a = a + b</code>
<code>-=</code>	<code>a -= b</code>	<code>a = a - b</code>
<code>*=</code>	<code>a *= b</code>	<code>a = a * b</code>
<code>/=</code>	<code>a /= b</code>	<code>a = a / b</code>
<code>%=</code>	<code>a %= b</code>	<code>a = a % b</code>
<code>//=</code>	<code>a //= b</code>	<code>a = a // b</code>
<code>**=</code>	<code>a **= b</code>	<code>a = a ** b</code>
<code>&=</code>	<code>a &= b</code>	<code>a = a & b</code>
<code> =</code>	<code>a = b</code>	<code>a = a b</code>
<code>^=</code>	<code>a ^= b</code>	<code>a = a ^ b</code>
<code><<=</code>	<code>a <<= b</code>	<code>a = a << b</code>
<code>>>=</code>	<code>a >>= b</code>	<code>a = a >> b</code>

运算符符号与等号之间允许存在空格或字符。

高级成员与身份运算符

Python 提供了两个专用运算符：**is**和**in**。恒等运算符（**is**）用于判断两个变量是否指向内存中完全相同的对象；而比较运算符（**==**）则用于判断这两个对象内部存储的值是否相等：

```
>>> a=[1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
真
>>> a 是 b
假的
>>> c = a
>>> a is c
真
>>> a[0] = 9
>>> 打印(c)
[9, 2, 3]
```


In this snippet, `a` and `b` contain identical values, so `a == b` evaluates to `True`. However, they reference separate, independently allocated list objects in memory, which is why `a is b` returns `False`. Because we assigned `c = a`, variable `c` points directly to the same object address as `a`; thus, modifying a value through `a` dynamically alters the data seen by `c`. The negative counterpart of the identity operator is `is not`.

The membership operator (`in`) tests whether a collection sequence (such as a list, tuple, dictionary, or string) contains a specific target value:

```
>>> s = 'Hello'
>>> 'e' in s
True
```

The negative counterpart of the membership operator is `not in`.

2.4 Data Structures

2.4.1 Lists

A list is a mutable, ordered sequence of elements. The items stored inside a list can be of any data type, and individual lists can hold elements of mixed data types simultaneously. Elements are accessed using a zero-based index enclosed in square brackets (`[]`):

```
>>> lst = [-2, 'alpha', 3.14, [1, 2, 3]]
>>> print(lst[0] + lst[2])
1.14
>>> print(lst[3][2])
3
```

Python also supports negative indexing, which accesses items relative to the end of the list. An index of `-1` points to the absolute final item, `-2` points to the second-to-last item, and so forth:

```
>>> print(lst[-1])
[1, 2, 3]
```

To extract a sub-range of elements, you slice the list using the colon (`:`) operator. Slicing syntax accepts up to three parameters: `L[start:stop:step]`. This returns a fresh sub-list containing elements from the `start` index up to, but not including, the `stop` index, moving at the specified increment (`step`). If omitted, `start` defaults to the beginning of the list, `stop` defaults to the end of the list, and `step` defaults to 1:

在这段代码中，a和b包含相同的值，因此a == b的判断结果为True。然而，它们实际上指向内存中独立分配的列表对象，这就是为什么a是b的判断结果为False的原因。由于我们将c赋值为a，变量c直接指向与a相同的对象地址；因此，通过a修改a的值会动态改变c所看到的数据。恒等运算符的反向操作符是不是。

成员运算符（中的）用于检测集合序列（如列表、元组、字典或字符串）是否包含特定目标值：

```
>>> s = '你好' ;  
"e" 在 s 中  
真
```

成员运算符的负对应元素不在中。

2.4 数据结构

2.4.1 列表

列表是一种可修改、有序的元素序列。列表中存储的元素可以是任意数据类型，且不同列表可同时包含混合数据类型的元素。访问元素时需使用方括号（[]）内的零基索引：

```
>>> l st = [-2, 'alpha', 3.14, [1, 2, 3]]>>> print (l st[  
0]+l st[2])  
1.14  
>>> 打印 (l st[3][2]) 3
```

Python还支持负索引功能，可根据列表末尾位置来访问元素。索引值为-1时指向列表的最后一个元素，-2时指向倒数第二个元素，依此类推：

```
>>> 打印 (最后[-1]  
[1, 2, 3])
```

要提取特定元素子集，可使用冒号（:）运算符对列表进行切片。切片语法最多接受三个参数：L[起始索引:终止索引:步长]。该操作将返回一个新的子列表，其中包含从起始索引开始、至终止索引结束（不包括终止索引）范围内、以指定步长（step）递增排列的元素；若省略参数，则起始索引默认为列表首部，终止索引默认为列表末尾，步长默认为1。

```
>>> a = list(range(10)) # Generates a mutable list: [0, 1, ..., 9]
>>> b = a[:]           # Generates a shallow copy of the entire list
>>> c = a[::-1]        # Reverses the entire list order
>>> print(c[::2])      # Extracts every second element
[9, 7, 5, 3, 1]
```

Remember that the **start** boundary index is inclusive, whereas the **stop** boundary index is strictly exclusive.

The following script demonstrates several common built-in methods for list manipulation:

```
#!/usr/bin/python3
# -*- coding: utf-8 -*-

season = ['立春', '立夏 ', '立秋', '立冬']
season.insert(1, '雨水')           # Inserts element at index 1
season.extend(('小雪', '大雪'))    # Appends multiple elements
target_idx = season.index('立秋')  # Locates index of target string
deleted_item = season.pop(target_idx) # Deletes and returns item at index
season[-1] = '大寒'                # Modifies element at final index
season.sort()                      # Sorts items alphanumerically
print('清明' in season)            # Evaluates list membership
```

Calling the `.copy()` method on a list generates a new top-level list container object. However, if any element inside that list is a mutable object (such as a nested sub-list), Python copies references to those nested objects rather than duplicating them. This behavior is known as a **shallow copy**:

```
>>> lst1 = [['中秋', '重阳'], 8.15, 9.9]
>>> lst2 = lst1.copy()
>>> lst2[0][1] = 'PureBright'
>>> lst2[1] = 4.5
>>> print(lst1)
[['中秋', '清明'], 8.15, 9.9]
```

2.4.2 Tuples

A tuple is an ordered sequence of values that behaves similarly to a list. It can store elements of any data type, and items are accessed using standard integer indexing. However, the critical architectural difference is that tuples are completely **immutable**.

```
>>> a = list(range(10)) # Generates a mutable list: [0, 1, ..., 9]
>>> b = a[:]            # Generates a shallow copy of the entire list
>>> c = a[::-1]         # Reverses the entire list order
>>> print(c[::2])       # Extracts every second element
[9, 7, 5, 3, 1]
```

需注意，起始边界索引为包含型，而终止边界索引则严格为排他型。

以下脚本展示了多种常用的内置列表操作方法：

```
/或使用`r/bin/python3`
#-*- 编码: utf-8 -*-

季节=['立春' , '立夏' , '立秋' , '立冬' ]
季节。插入(1, '雨水')      # 在索引1处插入元素 season。扩展(( '小雪' , '大雪' )
                             # 向字符串末尾追加多个元素 目标_idx = season.index('立秋')      # 查找目标字符串的索引
删除项 = season.pop(target_idx) # 删除该字符并在索引/season[-1]='大寒'处返回该项
                             # 在最终索引处修改元素
季节。排序()                # 按字母数字顺序对项目排序
print('清明' in season)     # 检查列表成员资格
```

对列表调用`.copy()`方法会生成一个新的顶级列表容器对象。然而，如果该列表中的任何元素是可修改对象（例如嵌套子列表），Python 会复制这些嵌套对象的引用而非直接复制它们。

这种行为被称为**浅层复制**：

```
>>> lst1 = [['中秋' , '重阳' ], 8.15, 9.9] >>>
lst2 = lst1.copy()
>>> lst2[0][1] = 'PureBright'
>>> lst2[1] = 4.5
打印(lst1)
[['中秋' , '清明' ], 8.15, 9.9]
```

2.4.2 元组

元组是一种有序值序列，其行为类似于列表。它可以存储任何数据类型的数据元素，并通过标准整数索引进行访问。但关键的架构差异在于：元组是**完全不可变的**。

Once a tuple is instantiated, you cannot modify its values, append new elements, or shrink its size.

Syntactically, a tuple is defined as a comma-separated sequence of values; however, wrapping tuples in parentheses `()` is recommended to clearly distinguish them from other structures:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> t = 'a', 'b', 'c', 'd', 'e'
```

To instantiate a tuple containing only a single element, you must include a trailing comma; otherwise, the parser will treat the expression as a standard string or integer enclosed in grouping parentheses:

```
>>> t1 = ('a')
>>> type(t1)
<class 'str'>
>>> t2 = ('a',)
>>> type(t2)
<class 'tuple'>
```

2.4.3 Sets

A Python set is an unordered collection of unique elements. Sets do not permit duplicate entries. They are commonly used to filter duplicate data out of sequences or to perform mathematical set operations such as unions, intersections, differences, and symmetric differences.

The primary architectural properties of sets include:

- **Unordered Structure:** Elements are not maintained in any specific sequence or index arrangement. Because sets are unordered, they do not support indexing or slicing; individual items cannot be accessed using an integer index key.
- **Guaranteed Uniqueness:** Duplicate entries are automatically dropped when adding items to a set.
- **Mutable Containers:** While the elements inside a set must be immutable types, the set container itself is mutable. It can dynamically grow or shrink as items are added or removed.

一旦元组被实例化，便无法修改其值、添加新元素或缩小其规模。

从句法角度而言，元组被定义为由逗号分隔的值序列；然而，建议使用括号（`()`）包裹元组以明确将其与其他结构区分开来。

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> t = 'a', 'b', 'c', 'd', 'e'
```

若要实例化仅包含单个元素的元组，必须在末尾添加逗号；否则解析器会将该表达式视为标准字符串或用括号括起来的整数。

```
>>> t1 = ('a')
>>> 类型(t1)
类名: "str"
>>> t2 = ('a',)
>>> 类型(t2)
类 '元组' >
```

2.4.3 集合

Python集合是一种由唯一元素组成的无序集合，不允许存在重复项。它常用于从序列中过滤重复数据，或执行并集、交集、差集及对称差等数学集合运算。

集合的主要结构特性包括：

- **无序结构**：元素不按特定顺序或索引方式排列。由于集合是无序的，因此不支持索引或切片操作；无法使用整数索引键访问单个元素。
- **唯一性保证**：向集合中添加项目时，重复项将被自动删除。
- **可变容器**：虽然集合内的元素必须是不可变类型，但集合容器本身却是可变的。随着项的添加或删除，其容量可以动态扩展或缩减。

2.4.4 Dictionaries

Dictionaries represent Python's optimization of associative arrays, mapping tables, or hash maps. A dictionary manages a collection of **key:value** pairs, where each unique key maps directly to a corresponding data value. Keys must be immutable data types (such as strings, integers, or tuples):

The primary architectural properties of dictionaries include:

- **Unordered Association:** Items are accessed by their lookup keys rather than by an integer position or insertion sequence.
- **Mutable Interface:** Dictionaries can be altered in place, and they can expand or contract dynamically as pairs are modified.

```
zhang3 = {}                # Instantiates an empty dictionary
                           #   (equivalent to dict())
zhang3['name'] = 'Zhang3'   # Inserts a key-value pair
                           #   ('name' maps to 'Zhang3')
li4 = zhang3.copy()        # Generates a shallow copy object
li4['age'] = 23             # Appends a new item into the copied
                           #   dictionary
print(zhang3, li4)
```

2.5 Python Program Architecture

2.5.1 Program Structure

Consider the following standalone script that determines whether an integer is a prime number:

Listing 2.2: prime.py, Script to Verify Prime Numbers

```
1  #!/usr/bin/python3
2  # -*- coding: utf-8 -*-
3
4  x = 127
5
6  def is_prime(val):
7      for i in range(2, val):
8          if val % i == 0:
```

2.4.4 词典

字典是Python对关联数组、映射表或哈希表的优化实现。字典管理一组键值 `value` 对，每个唯一键均直接对应一个数据值。键必须为不可变数据类型（如字符串、整数或元组）。

词典的主要架构特性包括：

- **非顺序关联：**项目通过其查找键而非整数位置或插入序列进行访问。
- **可修改接口：**词典可直接在原位置进行修改，并且随着配对项的变更，词典内容可动态扩展或收缩。

```
张3 = {}; # `Instantiate` 是一个空字典。
# (相当于 dict())
zhang3[ 'name' ] = '张3' # 插入键值对
# ( 'name' 对应 'Zhang3' )
li4 = zhang3; 复制 ( ) li4 # 生成一个浅拷贝对象
[ 'age' ] = 23 # 在复制的内容中添加新项目
# 词典
print(zhang3 , li4)
```

2.5 Python程序架构

2.5.1 程序结构

请考虑以下独立脚本，该脚本用于判断一个整数是否为质数：

列表 2.2: prime.py, 验证质数的脚本

```
1 位于 /sp_1> us r/bin/python3
2  #-*-编码: utf-8 -*-      3
4  x = 127
5
6  定义为 _prime (val):
7      对于i在范围内(2, val):
8          if val % i == 0:
```



```

9         return False
10    return True
11
12 if is_prime(x) is True:
13     print(x, 'is a prime.')
14 else:
15     print(x, 'is not a prime.')

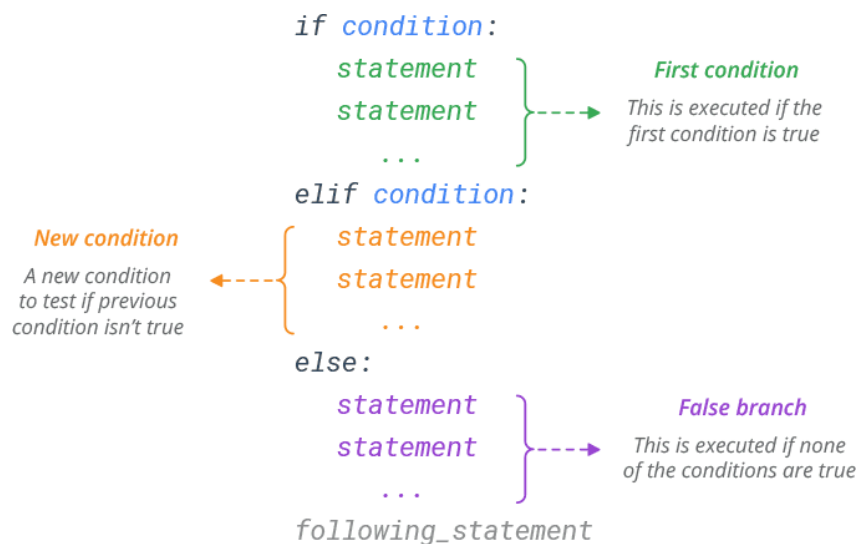
```

The Python interpreter processes script files sequentially, line-by-line, starting from the first executable top-level statement. Function and class declarations can be placed anywhere throughout this execution flow. The **def** keyword defines a named function block, and all internal statements belonging to that function must be indented uniformly.

To maintain clean code formatting, avoid mixing tabs and raw spaces within the same execution block, as different text editors interpret tab widths inconsistently. The official Python style guide strongly recommends using four spaces per indentation level.

2.5.2 Conditional Execution

To implement meaningful program logic, scripts must evaluate conditions and alter execution flows accordingly. Python handles conditional execution using the keywords **if**, **elif** (else if), and **else**.



- **Basic Conditional Branching:**

```

a = '1234'
if isinstance(a, int):
    print('a is an integer.')

```

```

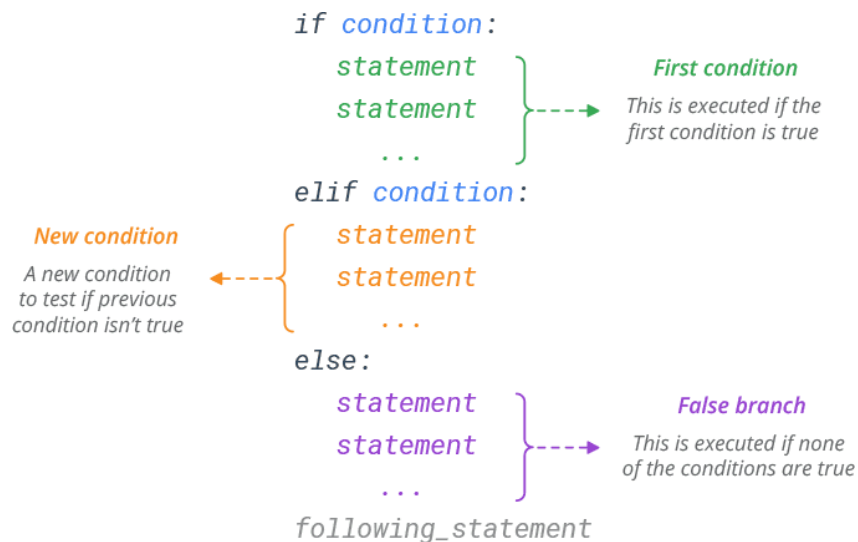
9         返回False
10    返回True
11
12如果 _prime(x)为True:
13打印(x, '是质数。' )
14否则:
15打印(x, '不是质数。' )

```

Python解释器会从第一个可执行的顶级语句开始，逐行顺序处理脚本文件。函数和类声明可以放置在执行流程的任何位置。**def**关键字用于定义命名函数块，该函数块内的所有内部语句必须保持统一的缩进格式。为确保代码格式整洁，请避免在同一执行块内混用制表符与空格——不同文本编辑器对制表宽度的解析存在差异。Python官方风格指南明确建议每个缩进层级使用四个空格。

2.5.2 条件执行

为实现有意义的程序逻辑，脚本必须评估条件并相应地调整执行流程。Python使用**if**、**elif**（else if）和**else**等关键字来处理条件执行。



- 基本条件分支:

```

a = '1234'
if isinstance(a, int):
    print('a is an integer.')

```

The Boolean expression evaluated immediately following the **if** keyword is the condition. The condition statement must end with a trailing colon (:), and the code lines that execute if the condition is true must be indented.

- **Alternative Binary Execution Paths:**

```
a = 2018
if a > 0:
    print('a is a positive number.')
else:
    print('a is not a positive number.')
```

Because the evaluated condition must resolve to either true or false, exactly one of the two code blocks will execute.

- **Chained Multiple Branch Conditionals:**

```
if a > 0:
    print('a is a positive number.')
elif a < 0:
    print('a is a negative number.')
else:
    print('a is zero.')
```

The interpreter evaluates each condition sequentially. If a condition returns false, control flows to the next branch. When a condition evaluates to true, its corresponding code block executes and the entire conditional structure terminates. Even if multiple conditions are true, only the first valid branch will execute.

- **Inline Conditional Expressions:** Python supports inline conditional expressions (ternary operations) to assign values or select concise actions based on a condition:

```
status_str = 'positive' if a > 0 else 'non-positive'
```

2.5.3 Loops and Iterations

Computers are routinely deployed to automate repetitive operations. Python provides two primary looping structures to handle iterations: **for...in** loops and **while** loops. Loop execution flows can be controlled using the keywords **continue**, **break**, and **pass**.

紧接在**if**关键字之后评估的布尔表达式即为条件。条件语句必须以冒号（:）结尾，且当条件为真时执行的代码行需缩进。

- 替代的二进制执行路径：

```
a = 2018
如果a > 0:
    print( 'a is a positive number . ' )else:
    print( 'a is not a positive number . ' )
```

由于评估结果必须为真或假，因此两个代码块中仅有一个会被执行。

- 链式多分支条件句：

```
如果a > 0:
    print( 'a is a positive number . ' )elif a < 0:
    print( 'a is a negative number . ' )else:
    print( 'a is zero . ' )
```

解释器会依次评估每个条件。若某个条件返回**false**，则控制流转至下一个分支；若条件返回**true**，则执行对应的代码块并终止整个条件结构。即使多个条件均为真，仅第一个有效的分支会被执行。

- 内联条件表达式：Python支持内联条件表达式（三元运算），可根据条件赋值或选择简洁的操作：

```
状态_str = '阳性' 若a > 0否则 '非阳性'
```

2.5.3 循环与迭代

计算机常被用于自动化重复性操作。Python提供了两种主要的循环结构来处理迭代：**for...in**循环和**while**循环。可通过**continue**、**break**和**pass**关键字控制循环执行流程。

The **break** statement terminates the innermost active loop, passing execution control to the first line of code immediately following the loop block. The **continue** statement stops the current loop iteration and immediately jumps execution back to the top of the loop block to evaluate the next item or loop condition. The **pass** statement is a null operation; it performs no action and serves strictly as a structural placeholder when Python syntax requires an indented code block.

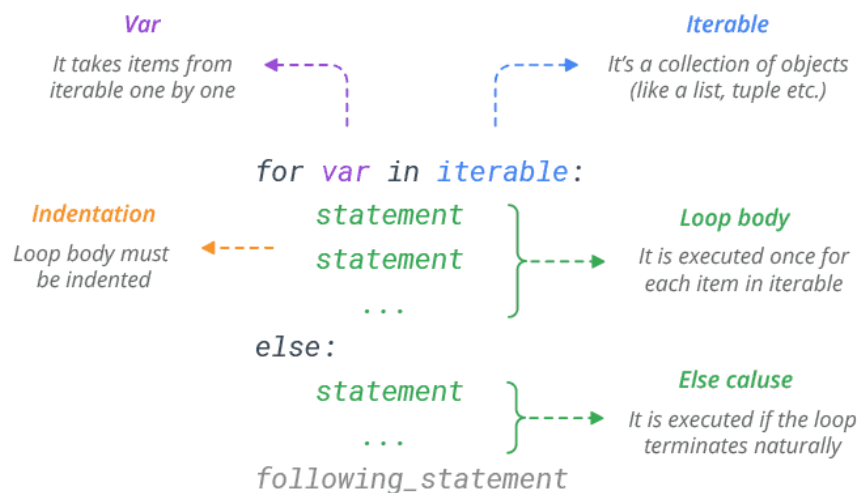
- **The for...in Collection Loop:**

```
days = [('Jan', 31), ('Feb', 28), ('Mar', 31), ('Apr', 30)]
for month, count in days:
    print('There are {} days in {}'.format(count, month))
```

- **The Integrated for...else Structure:**

```
numbers_list = [3, 9, 4, 2, 0, 5]
for x in numbers_list:
    if x <= 0:
        print('Found a non-positive number.')
        break
else:
    print('All numbers in the list are positive.')
```

Note that the terminal **else** block belongs to the loop structure itself, rather than the internal **if** statement. The code inside the loop's **else** block executes if and only if the loop runs to natural completion without encountering a **break** statement.



- **The Conditional while Loop:**

break语句用于终止最内层的活动循环，将执行控制权传递给循环块之后紧接的第一行代码。**continue**语句会停止当前循环迭代，并立即跳转回循环块开头以评估下一个条件或循环项。**pass**语句属于空操作，不执行任何动作，仅在Python语法要求缩进代码块时作为结构上的占位符使用。

- 在集合循环中的 `for ...` 语句:

天数 = [('1月' , 31日), ('2月' , 28日), ('3月' , 31日), ('4月' , 30日)]对于月份，按天数计算：

```
print( 'There are {} days in {} .' .format(count , month ))
```

- 集成的 “else” 结构:

数字列表 `_list=[3, 9, 4, 2, 0, 5]`对于数字列表

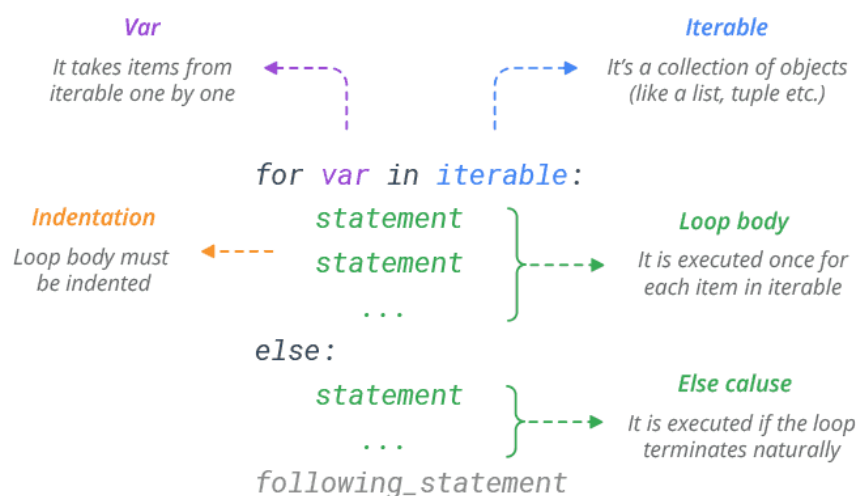
表 `_list` 中的 `x`:

```
if x <= 0:
    print( 'Found a non -positive number .' )
    破碎
```

其他的:

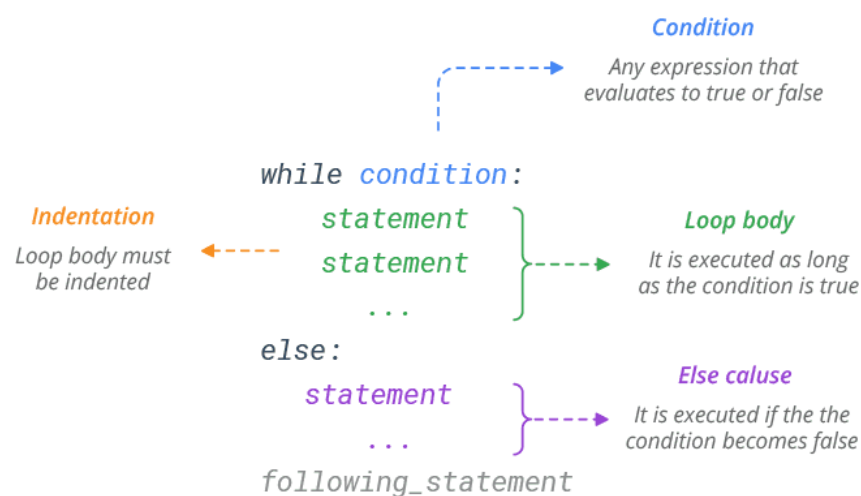
```
print( 'All numbers in the list are positive .' )
```

请注意，**else**块属于循环结构本身的一部分，而非内部的**if**语句。循环的**else**块内的代码仅当循环正常执行完毕且未遇到**break**语句时才会被执行。



- 条件循环结构:

```
import time
countdown = 10
while True:
    print(countdown)
    countdown -= 1
    if countdown <= 0:
        break
    time.sleep(1)
print('Time is up!')
```



For concise list initialization, looping constructs can be nested directly within bracket declarations to perform **list comprehensions**:

```
# Efficiently populates a list with squared values: [0, 1, 4, ..., 81]
squares_table = [x * x for x in range(10)]
```

2.5.4 Functions and Scope

Functions maximize code reuse, minimize logic duplication, and break complex programs into manageable blocks. The `def` statement creates a function object and binds it to an identifier:

```
def function_name(arg1, arg2, argN):
    # Function body statements
```

Function bodies often include a `return` statement, which can be placed anywhere within the block. Executing a `return` statement immediately exits the function and sends a calculated value back to the caller.

导入时间倒计时 = 10

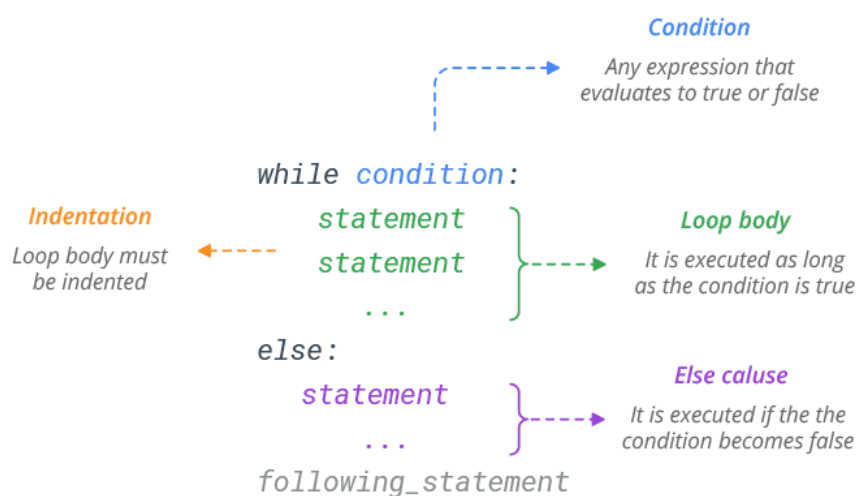
当True时:

```
print(countdown)
countdown -= 1
if countdown <= 0:
```

破碎

时间。睡眠(1)打印 ('时

间到了! ')



对于简洁的列表初始化，可以在括号声明中直接嵌套循环结构以执行**列表推导**：

```
# 使用平方值高效填充列表: squares_table = [x * x 对于 x 在 范围内(10)]
```

2.5.4 功能与范围

函数能够最大化代码复用性、最小化逻辑重复，并将复杂程序拆分为易于管理的模块。**def**语句用于创建函数对象并将其绑定到标识符。

定义函数名称 (参数1, 参数2, 参数N):

```
# 函数体语句
```

函数体通常包含一个**return**语句，该语句可放置在代码块内的任何位置。执行**return**语句会立即退出函数并将计算结果返回给调用者。

Functions also support default argument values, which make specific parameters optional during invocation. If a caller omits an optional argument, Python assigns it its default value before executing the function:

```
>>> def display_coordinates(x, y, label='Point'): print(x, y, label)
...
>>> display_coordinates(1, 2, 'Vertex')
1 2 Vertex
>>> display_coordinates(y=3, x=5)
5 3 Point
```

To accept an arbitrary number of inputs, functions can declare parameters prefixed with single (*) or double (**) asterisks. A single asterisk (***args**) captures excess positional inputs into an immutable tuple, whereas double asterisks (****kwargs**) capture excess keyword arguments into a mutable dictionary:

```
>>> def sum_all(*args): print(sum(args))
...
>>> sum_all(1, 3, 5, 7, 9)
25
>>> def print_metadata(**kwargs): print(kwargs.keys(), kwargs.values())
...
>>> print_metadata(name='Zhang', age=35)
dict_keys(['name', 'age']) dict_values(['Zhang', 35])
```

Anonymous, single-expression utility functions can be created using the inline **lambda** expression syntax:

```
>>> compute_sum = lambda x, y, z: x + y + z
>>> compute_sum(2, 3, 4)
9
```

2.5.5 Modules and Package Management

Python is an extensible language. Complex programs can be organized across separate files called **modules**. Each individual script file operates as an independent module that can export functions, variables, and classes to be used by other files. To access names defined inside an external module, you must first import it using the **import** keyword:

```
import math
print(math.sin(math.pi * 0.5))
```

函数还支持默认参数值，使得调用时特定参数可选。若调用者省略了可选参数，Python将在执行函数前为其赋值默认值：

```
>>> def display_coordinates(x, y, label = '点'): print(x, y, label);  
...  
>>> 显示坐标(1,2, '顶点')  
1 2 顶点  
>>> 显示坐标(y=3, x=5)  
5 3 观点
```

为接受任意数量的输入参数，函数可声明以单个(*)或双个(**)星号开头的参数。单个星号(*args)将多余的位置参数封装到不可变元组中，而双星号(**kwargs)则将多余的关键字参数封装到可变字典中：

```
>>> def sum_all(*args): print(sum(args));  
...  
>>> sum_all(1, 3, 5, 7, 9)  
25  
>>> def print_metadata(**kwargs): print(kwargs.keys(), (kwargs.values()))  
...  
>>> 打印元数据(名称='张', 年龄=35)  
字典键: ('name', 'age'); 字典值: ('Zhang', 35)
```

可以使用内联**lambda**表达式语法创建匿名单表达式实用函数：

```
>>> compute_sum = lambda x, y, z: x + y + z >>>  
compute_sum(2,3,4)  
9
```

2.5.5 模块与包管理

Python是一种可扩展的语言。复杂的程序可以组织成称为**模块**的独立文件。每个单独的脚本文件都作为一个独立的模块，能够导出函数、变量和类供其他文件使用。

若要访问外部模块中定义的名称，必须先使用**import**关键字导入该模块：

```
导入数学  
print(math.sin(math.pi * 0.5))
```

By default, functions imported via a standard `import module` statement must be prefixed with the module's name to prevent namespace clashes. Alternative import styles let you omit this prefix or rename modules:

```
from math import sin
print(sin(1))           # Executable
print(sin(pi / 2))      # Raises a NameError because 'pi' was
                        # not explicitly imported
...
from math import *
print(cos(pi / 3))      # Executable; imports all symbols
                        # from the module
...
import math as m
print(m.exp(1.0))       # Executable; renames the module namespace
                        # shortcut to 'm'
```

The broader Python developer ecosystem shares thousands of open-source third-party modules via the official PyPI (*Python Package Index*) repository (<https://pypi.org>). Because many packages have cascading dependencies, Python provides an automated command-line tool called `pip` (*Package Installer for Python*) to search, download, compile, and manage these libraries:

```
# pip install numpy      # Resolves dependencies when installing
# pip list               # Lists all installed packages
# pip uninstall numpy    # Safely removes the package
```

2.6 Summary

Python is a powerful, high-level language used extensively across data analysis, machine learning, and hardware automation. This section reviewed the core fundamentals of Python programming. For rapid lookup, Table 2.5 and Table 2.6 compile all reserved system keywords and globally available built-in functions.

You can inspect the reserved keyword list inside an interactive shell by running:

```
>>> import keyword
>>> print(keyword.kwlist)
```

默认情况下，通过标准**import**模块语句导入的函数必须以该模块名称作为前缀，以避免命名空间冲突。其他导入方式允许您省略此前缀或重命名模块：

```
from math import sin
print(sin(1))           # 可执行的
print(sin(pi / 2))      # 由于未明确导入 'pi'，因此抛出了 NameError 错
                        # 误。
...
from math import *
print(cos(pi / 3))      # 可执行文件；从该模块导入的 所有符号
                        # 内容
...
import math as m
print(m.exp(1.0))       # 可执行文件；重命名 模块命名空间 e
                        # 前往 "m" 的快捷方式
```

更广泛的Python开发者生态系统通过官方PyPI（Python软件包索引）仓库（<https://pypi.org>）提供了数千个开源第三方模块。由于许多软件包存在级联依赖关系，Python提供了一个名为pip（Python软件包安装器）的自动化命令行工具，用于搜索、下载、编译和管理这些库：

```
# 使用 pip 安装 numpy           # 安装时解决依赖关系
# 排列列表                     # 列出所有已安装的软件包
# 使用pip卸载numpy             # 安全地移除该软件包
```

2.6 摘要

Python是一种功能强大、面向高级编程的语言，在数据分析、机器学习和硬件自动化领域得到广泛应用。本节回顾了Python编程的核心基础知识。为便于快速查阅，表2.5和表2.6汇总了所有保留的系统关键字及全局可用的内置函数。

您可以通过运行以下命令，在交互式外壳中查看已保留的关键词列表：

```
>>> 导入关键词
>>> 打印（关键词.kw 列表）
```

Table 2.5: Reserved Keywords in Python Programming

<code>False</code>	<code>class</code>	<code>finally</code>	<code>is</code>	<code>return</code>
<code>None</code>	<code>continue</code>	<code>for</code>	<code>lambda</code>	<code>try</code>
<code>True</code>	<code>def</code>	<code>from</code>	<code>nonlocal</code>	<code>while</code>
<code>and</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>as</code>	<code>elif</code>	<code>if</code>	<code>or</code>	<code>yield</code>
<code>assert</code>	<code>else</code>	<code>import</code>	<code>pass</code>	
<code>break</code>	<code>except</code>	<code>in</code>	<code>raise</code>	

Table 2.6: Globally Available Built-in Interpreter Functions

<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

表 2.5: Python 编程中保留的关键词

假的	课	最后	是	返回
没有一个	持续	为了	λ	尝试
真	def	从...	非局部	在...期间
和	del	全球的	不	和
作为	如果; 倘若	如果	或	产量; 收益
断言; 主张	其他的	进口	使经过	
破碎	除了...之外	在	提高; 升高	

表 2.6: 全球通用的内置解释器函数

abs()	delattr()	哈希 (函数)	内存视图()	设置()
所有 ()	dict()	帮助()	min()	setattr()
任何 ()	dir()	hex()	下一个的 ()	薄片 ()
ascii()	divmod()	id()	目标()	分类的 ()
垃圾桶 ()	枚举 () 输入 ()		oct()	staticmethod()
bool()	eval()	int()	open()	str()
断裂点()	执行()	isinstance()	ord()	总数 ()
bytearray()	过滤器 ()	issubclass()	pow()	极好的 ()
字节()	使漂浮 ()	迭代 ()	打印 ()	元组()
可随时支取的 ()	版式 ()	len()	财产 ()	类型 ()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	本地 ()	repr()	zip()
编写()	全局; 全球	地图 ()	反转 ()	导入()
复杂的()	hasattr()	max()	轮 ()	

Chapter 3

I/O Interfaces on Raspberry Pi

3.1 Raspberry Pi

3.1.1 Short Introduction

The Raspberry Pi is a series of small single-board computers developed by a non-profit organization (the Raspberry Pi Foundation) in the United Kingdom in association with Broadcom Inc. The original Raspberry Pi project leaned towards promoting the teaching of basic computer science in schools and developing countries. Due to its low cost, modularity, and open design, it is now widely used across a diverse range of technical application fields. By December 2019, a total of 30 million Raspberry Pi units had been sold globally.¹

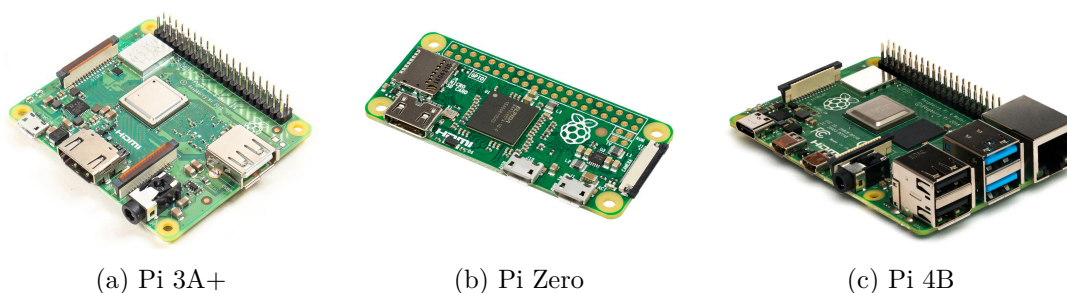


Figure 3.1: Different Models of Raspberry Pi

A single-board computer is a complete computer built on a single circuit board, containing the microprocessor(s), memory, input/output (I/O) facilities, and other fundamental features required of a fully functional computer. Single-board computers

¹Tweet by Eben Upton (CEO of Raspberry Pi Trading), December 14, 2019.

第 3 章

树莓派上的输入/输出接口

3.1 树莓派

3.1.1 简要介绍

树莓派是由英国非营利组织（树莓派基金会）与博通公司联合开发的一系列小型单板计算机。最初的树莓派项目旨在推动学校及发展中国家的基础计算机科学教育。凭借其低成本、模块化设计和开放架构的特点，该产品现已广泛应用于众多技术领域。截至2019年12月，全球累计售出3000万台树莓派设备。¹

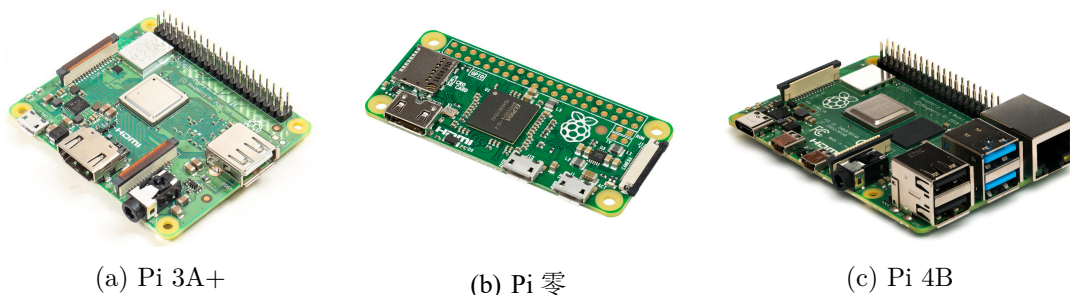


图 3.1：树莓派的不同型号

单板计算机是一种基于单一电路板构建的完整计算机，包含微处理器、存储器、输入/输出（I/O）接口以及功能完备的计算机所需的所有基本组件。

¹埃本·阿普顿（Raspberry Pi Trading公司首席执行官）于2019年12月14日发布的推文。

are commonly manufactured as demonstration or evaluation platforms, for educational instruction, or for deployment as embedded hardware controllers.

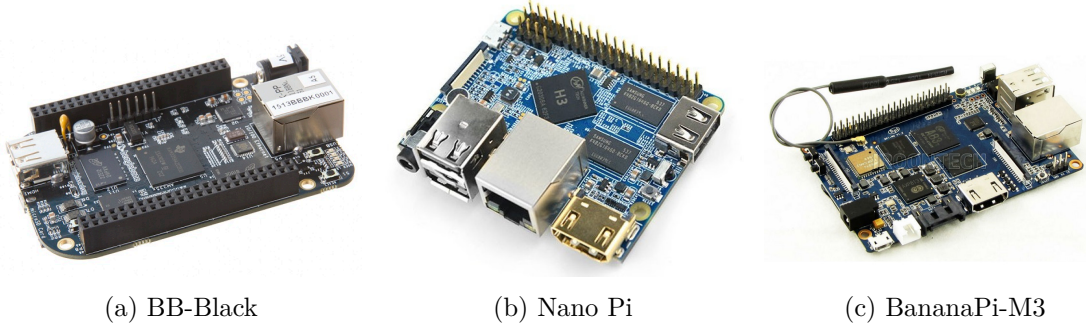


Figure 3.2: Alternative Single-Board Computer Configurations

All modern Raspberry Pi models feature a Broadcom system on a chip (SoC) with an integrated ARM-compatible central processing unit (CPU) and an on-chip graphics processing unit (GPU). The processing speeds across the product line range from 700 MHz to over 1.5 GHz, paired with onboard memory allocations scaling from 256 MB up to 8 GB of RAM. Secure Digital (SD) cards in MicroSD form factors are utilized to host the operating system and provide persistent non-volatile storage.

The boards feature up to four integrated USB ports, composite video outputs, and HDMI ports for digital display connectivity. Low-level hardware interfacing is supported via a row of general-purpose input/output (GPIO) pins, which natively expose standard serial communication protocols such as I²C, SPI, and UART.

3.2 Low-Level Hardware Expansion Interfacing

The primary physical feature distinguishing single-board embedded computers from traditional desktop workstations is the row of hardware pins exposed along the edge of the PCB. This pin array is designated as the general-purpose input/output interface connector. Through these pins, the system can interact directly with peripheral hardware components such as switches, sensors, solid-state relays, display modules, and operational drivers.

The physical layout and pin allocation scheme of the Raspberry Pi hardware connector is illustrated in Figure 3.1. This header contains 40 distinct pins spaced at a standard 2.54mm pitch. The pins are organized into three primary functional groups:

1. **Power Supply Rails:** Direct power outputs connected to the system's power traces. These include two 5V power pins (Pins 2 and 4), two 3.3V power pins

通常作为演示或评估平台、用于教育教学，或作为嵌入式硬件控制器部署使用。



图 3.2: 其他单板计算机配置方案

所有现代树莓派型号均采用博通系统级芯片（SoC），内置ARM兼容的中央处理器（CPU）和片上图形处理器（GPU）。该产品系列的处理速度范围从700 MHz至超过1.5 GHz，配套内存容量则从256 MB到8 GB不等。系统采用MicroSD格式的安全数字（SD）卡来存储操作系统并提供持久非易失性存储空间。

该板卡配备多达四个集成USB端口、复合视频输出接口以及用于数字显示连接的HDMI端口。通过一排通用输入/输出（GPIO）引脚实现低级硬件接口支持，这些引脚原生支持I²C、SPI和UART等标准串行通信协议。

3.2 低级硬件扩展接口

单板嵌入式计算机与传统桌面工作站的主要物理区别在于：其PCB边缘排列着一排硬件引脚，该引脚阵列被称为通用输入/输出接口连接器。通过这些引脚，系统可直接与开关、传感器、固态继电器、显示模块及操作驱动程序等外围硬件组件进行交互。

树莓派硬件连接器的物理布局及引脚分配方案如图3.1所示。该接口包含40个独立引脚，间距均为标准的2.54毫米。这些引脚被划分为三个主要功能组：

1. **电源排线：**直接连接至系统电源线路的电源输出端口，包括两个5V电源引脚（引脚2和4）以及两个3.3V电源引脚。

Table 3.1: Physical GPIO Pin Mapping of the Raspberry Pi

GPIO Connector					
BCM GPIO	Name	Header		Name	BCM GPIO
	3.3v	1	2	5v	
GPIO2	SDA1	3	4	5v	
GPIO3	SCL1	5	6	GND	
GPIO4	GCLK	7	8	TxD	GPIO14
	GND	9	10	RxD	GPIO15
GPIO17	GEN0	11	12	GEN1	GPIO18
GPIO27	GEN2	13	14	GND	
GPIO22	GEN3	15	16	GEN4	GPIO23
	3.3v	17	18	GEN5	GPIO24
GPIO10	MOSI	19	20	GND	
GPIO9	MISO	21	22	GEN6	GPIO25
GPIO11	SCLK	23	24	CE0	GPIO8
	GND	25	26	CE1	GPIO7
GPIO0	ID-SD	27	28	ID-SC	GPIO1
GPIO5		29	30	GND	
GPIO6		31	32		GPIO12
GPIO13		33	34	GND	
GPIO19		35	36		GPIO16
GPIO26		37	38		GPIO20
	GND	39	40		GPIO21
BCM GPIO	Name	Header		Name	BCM GPIO

note: GPIO18 and GPIO19 can be used as hardware PWM function.

(Pins 1 and 17), and multiple Ground references (GND). These pins supply physical voltage to drive low-current external hardware peripherals.

2. **General-Purpose I/O (GPIO) Pins:** Software-configurable pins that can be independently toggled via user applications to serve as digital inputs or digital outputs.
3. **Alternative Protocol Pins:** Multi-functional pins that can be multiplexed internally to handle hardware-driven serial communication protocols, including I²C,

表3.1：树莓派物理 GPIO 引脚对应关系表

GPIO 连接器					
BCM GPIO	姓名	标题		姓名	BCM GPIO
	3.3v	1	2	5v	
GPIO2	SDA1	3	4	5v	
GPIO3	SCL1	5	6	GND	
GPIO4	GCLK	7	8	TxD	GPIO14
	GND	9	10	RxD	GPIO15
GPIO17	GEN0	11	12	GEN1	GPIO18
GPIO27	GEN2	13	14	GND	
GPIO22	GEN3	15	16	GEN4	GPIO23
	3.3v	17	18	GEN5	GPIO24
GPIO10	MOSI	19	20	GND	
GPIO9	味精；盐	21	22	GEN6	GPIO25
GPIO11	SCLK	23	24	CE0	GPIO8
	GND	25	26	CE1	GPIO7
GPIO0	身份识别 SD	27	28	身份识别 SC	GPIO1
GPIO5		29	30	GND	
GPIO6		31	32		GPIO12
GPIO13		33	34	GND	
GPIO19		35	36		GPIO16
GPIO26		37	38		GPIO20
	GND	39	40		GPIO21
BCM GPIO	姓名	标题		姓名	BCM GPIO

注意：GPIO18和GPIO19可作为硬件 PWM 功能使用。

（引脚1和17）以及多个接地参考点（GND）。这些引脚提供物理电压以驱动低电流的外部硬件外设。

2. 通用输入/输出（GPIO）引脚：可通过软件配置的引脚，用户可独立通过应用程序切换其功能以实现数字输入或数字输出。
3. 替代协议引脚：多功能引脚，可内部复用以支持硬件驱动的串行通信协议（包括I²C）。

SPI, and UART interfaces.

3.3 I/O System Usages

A GPIO is an uncommitted digital signal pin on an integrated circuit or electronic circuit board whose behavior – including whether it acts as input or output – is controllable by the user at run time. To access these resources in Linux user space, let's change our working directory to `/sys/class/gpio`:

```
# cd /sys/class/gpio
# ls -l
```

We find following files and directories:

```
/sys/class/gpio
|-- export      (to create a GPIO node)
|-- unexport    (to delete a GPIO node)
`-- gpiochip0   (symbolic link)
```

The file `export` and `unexport` are used to create or delete GPIO nodes. With these nodes we can read or write GPIO levels. To access a pin (GPIO18 or pin 12 for instance), we need to create a node in user space related with it. With this node, we can read or change GPIO levels.

To create a node is to write a digital number to the file `export` by using redirection technique:

```
# echo 18 > export
```

Now a new symbolic link file (actually it is a directory) is created. Let's enter this directory and list files again:

```
# cd gpio18
# ls
active_low  direction  power      uevent
device     edge       subsystem  value
```

The file `direction` is used to set GPIO mode: “in” for input, “out” for output. The file `value` is used to control or read pin level. When GPIO is set to input, the file `value` is for reading; when GPIO is set to output, writing 0 or 1 to file `value` sets the pin level low or high:

```
# echo "in" > direction
# more value                # print pin level ("0" or "1")
# echo "out" > direction
```

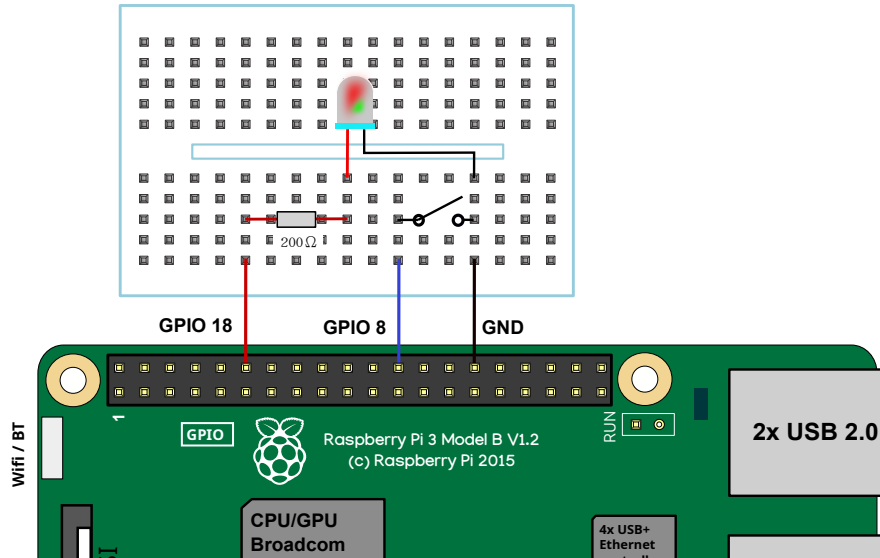



Figure 3.3: Connecting an LED and a Switch to GPIOs

```
# echo 1 > value          # output HIGH
# echo 0 > value          # Output LOW
```

When an LED (Light-Emitting Diode) is connected to this GPIO, you may see it on and off by writing “1” or “0” to file `value`.

There is PWM modules under the directory `/sys/class/pwm/pwmchip0`. To generate a PWM output, one can write a 0 or 1 to create a PWM node (just like GPIO node), enter the subdirectory, set duty cycle and period to the appropriate file.

When nodes are no longer to be used, it's better to remove the node directories from filesystem. The way to remove a node directory is to write a appropriate digit to the file `unexport`, which was written to `export` before.

The on-board LEDs are also controlled through GPIO. On Raspberry Pi 3, Active indicator is connected to GPIO47 and power status indicator is connected GPIO35). Users may enter directory `/sys/class/leds` to access them.

3.4 GPIO Modules for Python

3.4.1 Install a GPIO Module

There are several Python modules to manipulate GPIOs of Raspberry Pi. A flexible module which is written by Ben Croston is preferable and is also suggested by Raspberry community. A simplest way to install this module is using pip on Raspberry Pi:

```
# pip install RPi.GPIO
```

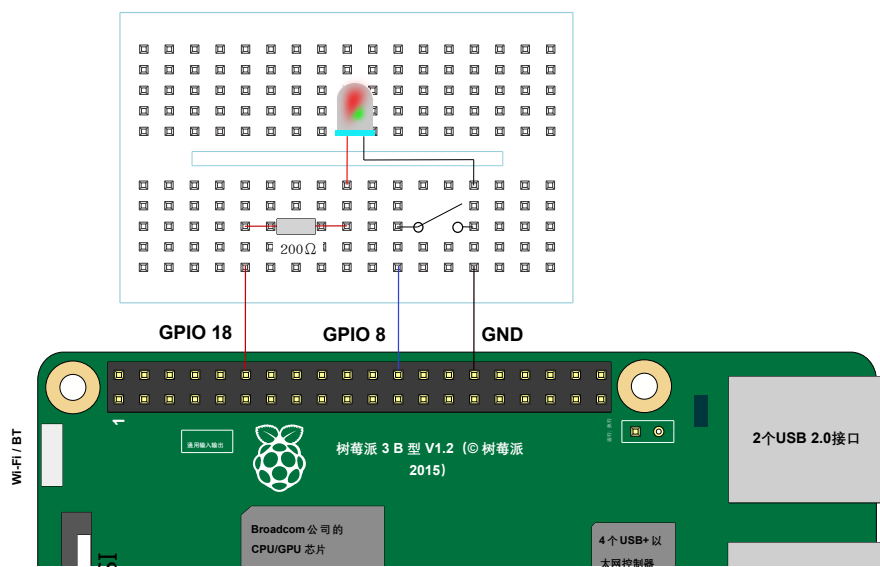


图 3.3: 将 LED 和开关连接到 GPIO 端口

```
# 回声: 回声效应
# 回声: 回声效应 1 > 值 0 > 值
```

输出 高: 高度

输出 低的

当LED（发光二极管）连接至该 GPIO 时，可通过向文件值写入“1”或“0”来控制其亮灭状态。

在目录 `/sys/class/pwm/pwmchip0` 下存在 PWM 模块。要生成 PWM 输出，可写入0或1以创建 PWM 节点（与 GPIO 节点类似），进入该子目录，并将占空比和周期设置为相应值。

当节点不再需要使用时，最好将该节点目录从文件系统中删除。删除节点目录的方法是：在先前用于导出的文件“unexport”中写入一个相应的数字。

板载LED灯亦通过 GPIO 进行控制。在Raspberry Pi 3上，活动指示灯连接至GPIO47，电源状态指示灯连接至GPIO35。用户可进入目录 `/sys/class/leds` 以访问这些指示灯状态信息。

3.4 Python 的 GPIO 模块

3.4.1 安装 GPIO 模块

有多个Python模块可用于操作树莓派的GPIO接口。由Ben Croston开发的灵活模块尤为推荐，这也是树莓派社区的建议方案。安装该模块最简便的方法是在树莓派上使用pip工具。

```
# 使用 pip 安装 RPi.GPIO
```


The official released module in PYPI is lack of hardware PWM support (till version 0.7). A patched version, which is hosted on github, can be downloaded and installed by following steps:

1. Get source code from github:

```
# git clone https://github.com/yfang1644/RPi.GPIO
```

2. Enter the source code directory to compile and install:

```
# cd RPi.GPIO
# python3 setup.py build
# python3 setup.py install
```

3. Now the module `RPi.GPIO` is available. Try following operations in Python shell to ensure the module is installed correctly. Note that the module name is **case-sensitive**.

```
# import RPi.GPIO
```

After installation, source code is no longer needed and therefore can be deleted.

3.4.2 Pin Numbering

There are two numbering system the IO pins on a Raspberry Pi within `RPi.GPIO` (see Table 3.1). The first is using the BOARD numbering system. This refers to the pin numbers on the connector of the Raspberry Pi board. The advantage of using this numbering system is that your hardware will always work, regardless of the board revision of the RPi. You will not need to rewire your connector or change your code.

The second numbering system is the BCM numbers. This is a lower level way of working – it refers to the channel numbers on the Broadcom SoC. You have to always work with a diagram of which channel number goes to which pin on the RPi board. Your script could break between revisions of Raspberry Pi boards.

To specify which you are using, call the function `setmode()`:

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
# or
GPIO.setmode(GPIO.BCM)
```

To detect which pin numbering system has been set (for example, by another Python module):

PYPI 中官方发布的模块存在硬件 PWM 支持缺失的问题（截至0.7版本）。用户可从GitHub下载并安装已修复的版本，具体步骤如下：

1. 从 GitHub 获取源代码：

```
# 使用git克隆https://github.com/yfang1644/RPi.GPIO
```

2. 输入用于编译和安装的源代码目录：

```
# CD RPi.GPIO
# 运行 `python3setup.py` 命令进行安装
# 在 Python 3 中运行以下命令：`setup.py install`
```

3. 目前模块RPi.GPIO 已可用。请在Python Shell中执行以下操作以确认该模块已正确安装。请注意，模块名称具有大小写敏感性。

```
# 导入 RPi.GPIO
```

安装完成后，源代码不再需要，因此可予以删除。

3.4.2 密码编号

树莓派（RPi）上的IO引脚采用两种编号系统，即 GPIO 系统（详见表3.1）。第一种是采用板级编号系统，即对应树莓派主板连接器上的引脚编号。使用该编号系统的优点在于：无论树莓派的板型版本如何变化，硬件都能稳定运行，无需重新布线或修改代码。

第二种编号系统是 BCM 编号。这是一种较低层级的标识方式，用于指代Broadcom系统级芯片（SoC）上的通道编号。使用时必须始终参考Raspberry Pi板上各通道编号与对应引脚的对应关系图；否则脚本在不同版本的Raspberry Pi板之间运行时可能出现问题。

如需指定所使用的模式，请调用 `setmode()` 函数：

```
导入RPi.GPIO as GPIO
GPIO.设置模式（GPIO.板卡）
# 或
GPIO.设置模式（GPIO.BCM）
```

用于检测已设置的引脚编号系统（例如由其他Python模块设定）：

```
mode = GPIO.getmode()
```

The mode will be `GPIO.BOARD`, `GPIO.BCM`. If the pin is not configured, the function returns `None`.

3.4.3 Set up Pins

The function of a pin must be set up before use – either as an input or as an output. To do so, you need to call function `setup()`:

```
GPIO.setup(channel, GPIO.IN)
# or
GPIO.setup(channel, GPIO.OUT)
```

where `channel` is the pin number or GPIO number based on the numbering system you have specified (`BOARD` or `BCM`).

You can also specify an initial value for your output channel, or pull-up/pull-down resistor for your input channel

```
GPIO.setup(channel1, GPIO.OUT, initial=GPIO.HIGH)
GPIO.setup(channel2, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

Multiple channels can be set up per call, when iterable object is used:

```
chan_list = [11,12]      # add as many channels as you want!
                        # you can tuples instead i.e.:
                        #   chan_list = (11,12)
GPIO.setup(chan_list, GPIO.OUT)
# or
GPIO.setup(chan_list, [GPIO.IN, GPIO.HIGH])
```

Similarly, you can set multiple pin levels by calling function `output()` once, when the pins are configured as output:

```
chan_list = [11,12]      # also works with tuples
GPIO.output(chan_list, GPIO.LOW)
GPIO.output(chan_list, (GPIO.HIGH, GPIO.LOW))
```

At the end of the program, it is good practice to clean up any resources you might have used. After `cleanup()`, all channels you have used back to inputs with no pull up/down. This will avoid accidental damage to Raspberry Pi by shorting out the pins.

```
GPIO.cleanup(channel)      # individual clean up
GPIO.cleanup([channel1, channel2]) # multiple clean up
GPIO.cleanup()             # clean up all pins
```

```
模式 = GPIO。获取模式 (getmode())
```

模式将设置为 GPIO.board、GPIO.BCM。若未配置该引脚，则函数返回None。

3.4.3 设置图钉

使用前必须设置引脚的功能（可设为输入或输出）。具体操作需调用functionsetup()函数：

```
GPIO.设置 (通道, GPIO.输入)
# 或
GPIO.设置 (通道, GPIO.输出)
```

其中channel表示根据您指定的编号系统（电路板或 BCM）对应的引脚编号或 GPIO 编号。

您还可以为输出通道指定初始值，或为输入通道设置上拉/下拉电阻。

```
GPIO.设置 (通道1, GPIO.OUT, 初始值= GPIO.HIGH)
GPIO.设置 (channel2, GPIO.In, pull_up_down= GPIO.PUD_UP)
```

当使用可迭代对象时，每次调用均可设置多个通道：

```
chan_list=[11,12]           # 可根据需要添加任意数量的频道！

                               # 您可以使用元组代替 `i`，例如：
                               # 查看列表 = (11,12)

GPIO.设置 (chan_list, GPIO.OUT)
# 或
GPIO.设置 (chan_list, [GPIO.IN, GPIO.HIGH])
```

同样地，当引脚配置为输出模式时，您只需调用一次function output()函数即可设置多个引脚电平。

```
chan_list=[11,12]           # 也适用于元组
GPIO.输出 (chan_list, GPIO.LOW)
GPIO.输出 (chan_list, (GPIO.HIGH, GPIO.LOW))
```

程序结束时，建议清理所有已使用的资源。调用 cleanup() 后，所有已使用的通道均需恢复为无拉高/拉低信号的输入状态，以防止引脚短路导致树莓派意外损坏。

```
GPIO.清理 (通道)           # 个人清洁工作      GPIO.清理 ([通道1, 通道2]) # 多重清理
GPIO.清理()                # 清理所有图钉
```

3.4.4 Use GPIO as Input

Polling

GPIO input reflects the level the pin connects to. you can use function `input()` to read the value of a GPIO pin (polling):

```
val = GPIO.input(channel)
if val == GPIO.LOW:
    ...
elif val == GPIO.HIGH:
    ...
```

Function `input()` returns either 0/GPIO.LOW/False or 1/GPIO.HIGH/True.

To wait for a button press by polling in a loop:

```
while GPIO.input(channel) == GPIO.LOW:
    time.sleep(0.01)
```

This assumes that pressing the button changes the input from LOW to HIGH. CPU reads the pin status every 10 ms.

Interrupts and Edge detection

As we can see from polling, CPU runs busily on reading the status of a pin, and may miss some state changes (when the duration of a button press is shorter than 10ms).

GPIO module can detect the edge. An edge is the change in state of an electrical signal from LOW to HIGH (rising edge) or from HIGH to LOW (falling edge). Quite often, we are more concerned by a change in state of an input than its value. This change in state is an event.

To avoid missing a button press while your program is busy doing something else, there are several ways to get round this:

- the `wait_for_edge()` function
- the `event_detected()` function
- a threaded callback function that is run when an edge is detected

The `wait_for_edge()` function is designed to **block** execution of your program until an edge is detected. In other words, the example above that waits for a button press could be rewritten as:

3.4.4 使用 GPIO 作为输入

投票

GPIO 输入值反映了引脚所连接的电平。您可以使用`input()`函数读取 GPIO 引脚的数值（轮询方式）。

```
值 = GPIO.输入（通道）
if val == GPIO.LOW:
    ...
如果值 == GPIO.HIGH:
    ...
```

函数`input()`返回值为0/ `GPIO.LOW/False`或1/ `GPIO.HIGH/True`。

通过循环轮询来等待用户按下按钮：

```
while GPIO.input(channel) == GPIO.LOW:
    时间：睡眠（0.01）
```

该假设认为按下按钮可将输入信号从低电平（LOW）切换至高电平（HIGH）。CPU每10毫秒读取一次引脚状态。

中断与边缘检测

如调查结果所示，CPU主要忙于读取引脚状态，因此可能无法检测到某些状态变化（当按钮按压持续时间短于10毫秒时）。

GPIO 模块能够检测信号边沿。信号边沿是指电信号状态从低电平变为高电平（上升沿）或从高电平变为低电平（下降沿）的变化。通常，我们更关注输入信号的状态变化而非其具体数值；这种状态变化本身即构成一个独立事件。

为避免在程序执行其他任务时错过按键操作，有多种方法可解决此问题：

- `wait_for_edge()` 函数
- `event Detected()` 函数
- 一个在检测到边时运行的带线程回调函数

`wait_for_edge()`函数的设计目的是在检测到边之前**阻断**程序的执行。换言之，上述等待按钮点击的示例可以重写为：

```
chn = GPIO.wait_for_edge(channel, GPIO.RISING, timeout=10)
if chn is None:
    print('Timeout occurred')
else:
    print('Edge detected on channel', channel)
```

You can set `RISING`, `FALLING` or `BOTH` event to be detected. The advantage of doing it this way is that it uses a negligible amount of CPU, so there is plenty left for other tasks.

The `event_detected()` function is designed to be used in a loop with other things, but unlike polling, it is not going to miss the change in state of an input while the CPU is busy working on other things. This could be useful when using something like GUI programs where there is a main loop listening and responding to GUI events in a timely basis.

```
GPIO.add_event_detect(channel, GPIO.RISING)
do_something()
if GPIO.event_detected(channel):
    print('Button pressed')
```

Note that you can detect events for `RISING`, `FALLING` or `BOTH`.

Callback Functions

RPi.GPIO runs a second thread for callback functions. This means that callback functions can be run at the same time as your main program, in immediate response to an edge. For example:

```
def my_callback(channel):
    print('This is a edge event callback function!')
    print('Edge detected on channel %s' % channel)
    print('The callback runs in a different thread to main program')

GPIO.add_event_detect(channel,
                      GPIO.RISING,
                      callback=my_callback)
...the rest of your program...
```

Callback functions are special functions. They are not called explicitly, since the events are unpredictable. The arguments of a callback function is determined by the designer, not user. And more often, callback function has no return value, because no caller can get the return value.

```
chn = GPIO.wait_for_edge(通道, GPIO.rising, timeout=10)如果chn为None:
```

```
    print( 'Time out occurred' )
```

其他的:

```
    print( 'Edge detected on channel' , channel )
```

您可以设置检测上升沿、下降沿或两者兼有。这种方式的优势在于占用的CPU资源极低，从而为其他任务腾出充足资源。

event Detected()函数专为与其他组件配合在循环中使用而设计；与轮询机制不同，它能在CPU忙于其他任务时也不会错过输入状态的变化。这一特性在使用图形用户界面（GUI）程序时尤为实用——这类程序通常包含一个主循环，能够实时监听并响应GUI事件。

```
GPIO.添加事件检测（通道, GPIO.升沿）
```

```
做某事（）
```

```
如果GPIO.event Detected（通道）:
```

```
    print( 'Button pressed' )
```

请注意，您可以检测到价格上涨、下跌或同时出现的两种情况。

回调函数

RPi.GPIO 会为回调函数运行第二个线程。这意味着回调函数可与主程序同时执行，即时响应事件触发。例如：

```
def my_callback(channel):
    print( 'This is a edge event callback function ! ' )
    print( 'Edge detected on channel %s' % channel )
    print( 'The callback runs in a different thread to main program ' )
```

```
GPIO.添加事件检测功能（通道，
```

```
GPIO.上升；增长。
```

```
回调函数 = my Callback )
```

```
. .....您程序的其余部分.....
```

回调函数是特殊函数。由于事件不可预测，它们不会被显式调用。回调函数的参数由设计者而非用户决定。此外，回调函数通常不返回值，因为调用方无法获取其返回结果。

If you want more than one callback function for an event:

```
def my_callback_one(channel):
    print('Callback one')

def my_callback_two(channel):
    print('Callback two')

GPIO.add_event_detect(channel, GPIO.RISING)
GPIO.add_event_callback(channel, my_callback_one)
GPIO.add_event_callback(channel, my_callback_two)
```

Note that in this case, the callback functions are run sequentially, not concurrently. This is because there is only one thread used for callbacks, in which every callback is run, in the order in which they have been defined. Therefore, it's possible that one channel has multiple callbacks, but **multiple channels cannot share a common callback function**.

Switch Debounce

You may notice that the callbacks are called more than once for each button press. This is as a result of what is known as “switch bounce”. There are two ways of dealing with switch bounce: hardware approach and software approach. Add a capacitor across the switch to filter the high frequency signal may improve debouncing. To debounce using software, add the `bouncetime` parameter in milliseconds to a function where you specify a callback function:

```
# add rising edge detection on a channel,
# ignoring further edges for 200ms for switch bounce handling
GPIO.add_event_detect(channel,
                      GPIO.RISING,
                      callback=my_callback,
                      bouncetime=200)

# or
GPIO.add_event_callback(channel,
                       my_callback,
                       bouncetime=200)
```

If for some reason, your program no longer wishes to detect edge events, it is possible to stop them:

如果您需要为同一事件配置多个回调函数：

定义my Callback_one（通道）：

```
print( 'Callback one ' )
```

定义my_callback_two（通道）：

```
print( 'Callback two ' )
```

GPIO . 添加事件检测（通道， GPIO . 升沿）

GPIO . 添加事件回调函数（通道， my Callback-one）

GPIO . 添加事件回调函数（通道， my Callback_two）

请注意，在这种情况下，回调函数是按顺序执行而非并行执行。这是由于仅使用一个线程处理所有回调函数，每个回调函数均按照其定义顺序依次运行。因此，单个通道可能包含多个回调函数，但不同通道无法共享同一个回调函数。

切换去抖

您可能会注意到，每次按下按钮时回调函数都会被调用多次。这种现象被称为“开关回弹”（switch bounce）。解决开关回弹问题主要有两种方法：硬件方案和软件方案。在开关两端串联电容器以滤除高频信号，可有效改善回弹现象；若采用软件方案，则需在定义回调函数的代码中添加一个以毫秒为单位的**bouncetime**参数。

```
# 在通道上添加上升沿检测功能，
# 在处理开关回弹时，将后续边线的检测延迟 200 毫秒          GPIO . 添加事件检测功能
（通道，
                                GPIO . 上升；增长。
                                回调函数 = my Callback；
                                bouncetime=200）

# 或
GPIO . 添加事件回调函数（通道，
                                我的回调函数：
                                bouncetime=200）
```

若因某种原因您的程序不再需要检测边缘事件，可随时停止该功能：

```
GPIO.remove_event_detect(channel)
```

3.4.5 Ultrasonic Ranging

Ultrasonic ranging entails transmitting a sound wave and measuring the time that it takes for the sound wave to reflect off of an object and back to the transducer. The reflection time is proportional to the distance between the object and the transducer. In this implementation, a blanking interval is required between signal transmission and reception.

The ultrasonic ranging module SR04 provides stable readings and accurately measures distance. This module can be used for a number of projects such as fluid level in a tank, distance to obstacles, and other sensing tasks.

The processor initiates IO trigger by supplying at least $10\mu\text{s}$ high level pulse to “trig” pin. The module will automatically send 8 40kHz square waves, drive “echo” pin to high, and detect the returning pulse signal. When returning signal is received, the device pulls the “echo” back to low. The processor should mark the time that the level changes of the “echo” pin (Figure 3.4).

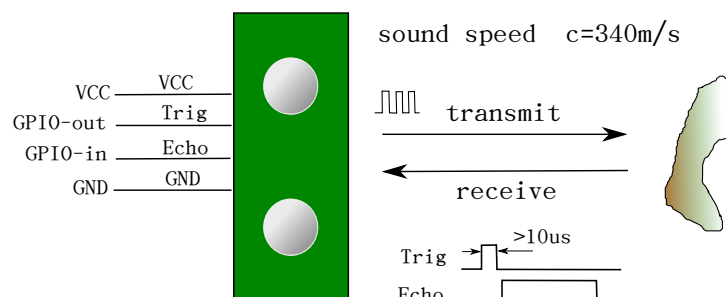


Figure 3.4: Ultrasonic Ranging

A test code for ultrasonic ranging is shown in List 3.1.

Listing 3.1: ranging.py, Ranging Using a Ultrasonic Module

```
TRIG, ECHO = ....

def ranging():
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.000012)
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == GPIO.LOW:
        pass
```

GPIO . 移除事件检测 (通道)

3.4.5 非超声测距

超声测距技术通过发射声波，并测量该声波从物体表面反射后返回换能器所需的时间。反射时间与物体与换能器之间的距离成正比。在此实现方案中，信号发射与接收之间需设置一个消隐间隔。

SR04超声波测距模块可提供稳定的读数并精确测量距离，适用于多种应用场景，如储罐液位监测、障碍物距离检测及其他传感任务。

处理器通过向“trig”引脚提供至少10个 μ 的高电平脉冲来触发输入/输出操作。该模块将自动发送8个40 kHz方波，使“echo”引脚保持高电平，并检测返回的脉冲信号；接收到返回信号后，设备会将“echo”引脚拉回低电平。处理器需记录“echo”引脚电平变化的时间点（图3.4）。

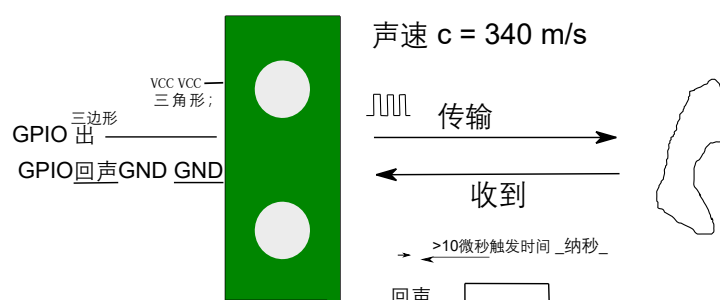


图 3.4: 超声测距

超声测距的测试代码如列表3.1所示。

列表 3.1: ranging.py, 使用超声波模块进行测距

```
TRIG、ECHO = ...

def ranging():
    GPIO 输出 (TRIG, GPIO 高电平)
    时间。睡眠 (0.000012)
    GPIO 输出 (TRIG, GPIO 低电平)
    while GPIO.input(ECHO) == GPIO.LOW:
        pass
```

```

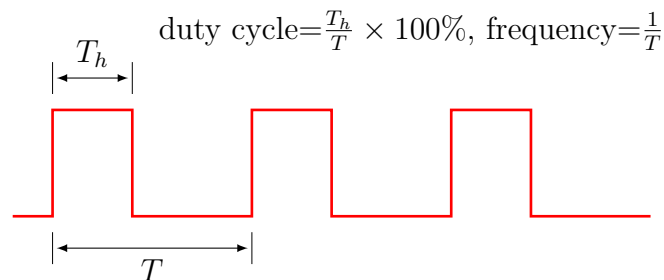
t1 = time.time()                # mark the rising edge
while GPIO.input(ECHO) == GPIO.HIGH:
    pass
t2 = time.time()                # mark the falling egde
return (t2 - t1)*34000/2        # distance in cm

```

3.5 Using PWM

3.5.1 What is a PWM

Pulse-width modulation (PWM), or pulse-duration modulation (PDM), is a modulation technique used to encode a message into a pulsing signal.



The term **duty cycle** describes the proportion of “on” time to the regular interval or “period” of time. Duty cycle is expressed in percent, 100% being fully on. A lower duty cycle corresponds to lower power.

3.5.2 Accessing PWM in Userspace

Under the directory of `/sys/class/pwm`, several subdirectory for pwmchips are listed, one of them looks like:

```

/sys/class/pwm/pwmchip0
| ...
|-- export          (to create PWM node)
|-- unexport        (to delete PWM node)
`-- npwm            (number of PWMs)

```

The file `npwm` describes how many pins this group has. Each group has maximum 2 PWM pins valid. we can write an index “0” or “1” to file `export` to create PWM node. Under each node directory, there are files for the user to set PWM property:

```

/sys/class/pwm/pwmchip0
| ...

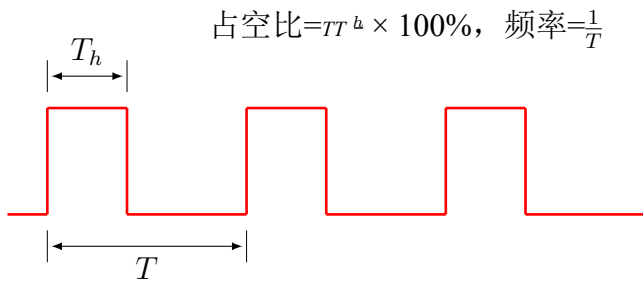
```

```
t1 = 时间。time()                                # 标记上升沿
在输入GPIO . (ECHO) 时      ==  GPIO . 高:
    使经过
t2 = 时间。time()                                #  标记: 标注   落下      例如: de ;
返回 (t2 - t1 )*34000/2)                        #      距离      以厘米为单位
```

3.5 使用 PWM

3.5.1 什么是 PWM ?

脉宽调制（PWM）或脉冲持续时间调制（PDM）是一种将信息编码为脉冲信号的调制技术。



占空比指“开启”时间与常规间隔或“周期”时间的比例。占空比以百分比表示，100%代表完全开启状态；占空比越低，功耗越低。

3.5.2 在用户空间中访问 PWM

在 /sys/class/pwm 目录下，列出了多个用于 pwmchips 的子目录，其中一个子目录结构如下：

```
系统类/泵程序/泵芯片0
| ...
|-- 导出                (用于创建 PWM 节点)
|-- unexport            (用于删除 PWM 节点)
`-- npwm                (PWM的数量)
```

文件npwm用于描述该组包含的引脚数量。每个组最多支持2个有效的 PWM 引脚。可通过向文件写入索引“0”或“1”来创建 PWM 节点。在每个节点目录下，均设有供用户设置 PWM 属性的文件：

```
系统/类/pwm/pwmchip0
| ...
```

```

|-- export          (to create PWM node)
|-- unexport        (to delete PWM node)
|-- npwm            (number of PWMs)
`-- pwm0
    | ...
    |-- enable       (0 or 1)
    |-- capture
    |-- period       (time in ns)
    |-- duty_cycle   (time in ns)
    `-- polarity     (normal or inversed)

```

Writing an integer number to file `period` will set the output waveform to a certain period in nano seconds. Writing an integer number to `duty_cycle` sets its duty cycle periods in nano seconds. The duty cycle must be less or equal to period.

3.5.3 PWM Controlling using RPi.GPIO

Python's PWM module within RPi.GPIO enables to access the PWM on board, to easily and accurately generate a PWM output signal of a given duty cycle and frequency.

To create a PWM instance:

```

GPIO.setup(channel, GPIO.OUT)
p = GPIO.PWM(channel, frequency)

```

A pin used as PWM must be set as output. Function `start()` starts PWM output:

```

p.start(dc)          # duty cycle (0.0 <= dc <= 100.0)

```

Once the PWM instance variable `p` is created, it's not suitable to create it again before removing it (`cleanup()`). Function `ChangeFrequency()` and `ChangeDutyCycle()` are used to change the output of the PWM:

```

p.ChangeFrequency(freq)  # set new frequency in Hz
p.ChangeDutyCycle(dc)    # 0.0 <= dc <= 100.0

```

Function `stop()` stops the PWM output, until next `start()` function call.

```

p.stop()

```

Note that PWM will also stop if the instance variable `p` goes out of scope.

Since software PWMs in RPi.GPIO module are programmed as thread, all GPIO pins can be configured as PWM. But only GPIO18 and GPIO19 are hardware PWM pins. Hardware PWM's clock are generated by crystal oscillators, which is far more accurate than thread in timeslice and required almost no CPU instructions.

```

|-- 导出                      (用于创建 PWM 节点)
|-- unexport                  (用于删除 PWM 节点)
|-- npwm                      (PWM的数量)
`-- pwm0
    | ...
    |-- enable                (0 or 1)
    |-- capture
    |-- period                (time in ns)
    |-- duty_cycle            (time in ns)
    `-- polarity              (normal or inversed)

```

将整数写入“周期”字段可将输出波形设置为特定的纳秒周期；将整数写入“占空比”字段则可设置其占空比值（单位：纳秒）。占空比必须小于或等于周期值。

3.5.3 PWM 使用RPI进行控制。GPIO

RPI中的Python PWM 模块。GPIO 可访问内置 PWM，轻松精准地生成具有指定占空比和频率的 PWM 输出信号。

要创建一个 PWM 实例：

```

GPIO . 设置 (通道, GPIO . 输出)
p = GPIO . PWM (通道, 频率)

```

用作 PWM 的引脚必须设置为输出模式。function start()函数可启动 PWM 输出：

```

第 p 页起始位置 (dc) # 工作周期 (0.0 <= dc <= 100.0)

```

一旦创建了 PWM 实例变量p，就不宜在删除该变量之前再次创建（清理操作）。可使用ChangeFrequency()和ChangeDutyCycle()函数来更改 PWM 的输出结果：

```

p.ChangeFrequency(freq)          # 设置新频率                Hz
p.ChangeDutyCycle(dc)             # 0.0 <= dc <= 100.0

```

停止()函数可暂停 PWM 输出，直至下一次调用start()函数时恢复。

```

p. 停止 ()

```

请注意，如果实例变量p超出作用域范围，PWM 也会终止运行。

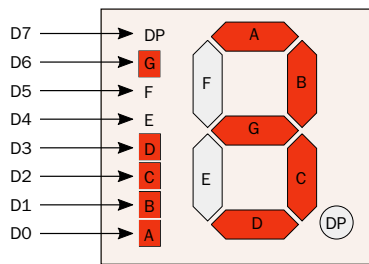
由于RPI GPIO 模块中的软件PWM模块以线程形式编程，所有 GPIO 引脚均可配置为 PWM 引脚；但仅GPIO18和GPIO19属于硬件 PWM 引脚。硬件 PWM 的时钟由晶体振荡器生成，其精度远高于基于时间片的线程机制，且几乎无需CPU指令支持。

3.6 I2C Module

3.6.1 Single Segments Display

A 7-segment display (also called 8-segment display, with a decimal point) is a form of electronic display device for displaying decimal numerals. Such displays are widely used in digital clocks, electronic meters, basic calculators, and other electronic devices that display numerical information.

The display is composed of 8 LEDs, with all of the cathodes (negative terminals) or anodes (positive terminals) connected together and brought out to a common pin; this is referred to as a “common cathode” or “common anode” device. When D7~D0 connect to data bus of a computer, if you want to display “3”, you can write a “0x4F” to the common cathode device, or “0xB0” to the common anode one (Figure 3.5a).



(a) 7-Segment Display



(b) 4 Displays in a Module

Figure 3.5: 7-Segment LED Display

3.6.2 Multiple Display

Each 7-segment display needs at least 9 pins to control. With this kind of structure, multiple displays will not be practical to use.

A multiple segment LED display with 4 digitals is shown in Figure 3.5b. In order to send multiple digital signal with less pins, serial communication techniques are introduced. This module is an I²C (Inter-Integrated Circuit, or I2C, IIC, pronounced “I square C” or “I two C”) communication peripheral.

The I²C bus is a synchronous, multi-master, multi-slave, packet-switched serial communication bus (See Figure 3.6). It is classified as a two-wire serial interface because it requires only two physical signal wires to transmit and receive data between devices:

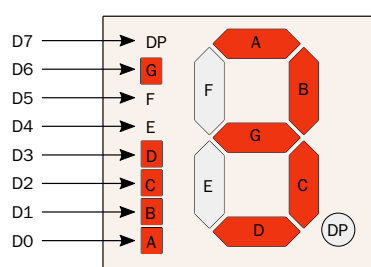
- **SDA (Serial Data):** The bidirectional line used to transmit data bits between masters and slaves.

3.6 I2C模块

3.6.1 单段显示

七段显示器（亦称带小数点的八段显示器）是一种用于显示十进制数字的电子显示装置。此类显示器广泛应用于数字时钟、电子仪表、基础计算器以及其他需要显示数值信息的电子设备中。

该显示装置由8个LED组成，所有阴极（负极）或阳极（正极）均相互连接并引出至同一引脚；这种结构称为“共阴极”或“共阳极”器件。当D7~D0连接至计算机的数据总线时，若需显示数字“3”，可向共阴极器件写入“0x4F”，或向共阳极器件写入“0xB0”（图3.5a）。



(a) 7段显示器



(b) 4 在模块中显示

图 3.5：7 管段 LED 显示屏

3.6.2 多显示器

每个7段显示器至少需要9个引脚进行控制。采用此种结构时，多块显示器难以实际应用。

图3.5b展示了一款具备四位数字显示功能的多段LED显示屏。为以较少引脚数量传输多个数字信号，本文采用了串行通信技术。该模块采用I²C（集成电路间通信，简称I2C，IIC发音为“I square C”或“I two C”）通信外设。

I²C总线是一种同步、多主多从、分组交换的串行通信总线（见图3.6）。它被归类为双线串行接口，因为仅需两条物理信号线即可在设备间传输和接收数据：

- **SDA（串行数据）**：用于主设备与从设备之间传输数据位的双向线路。

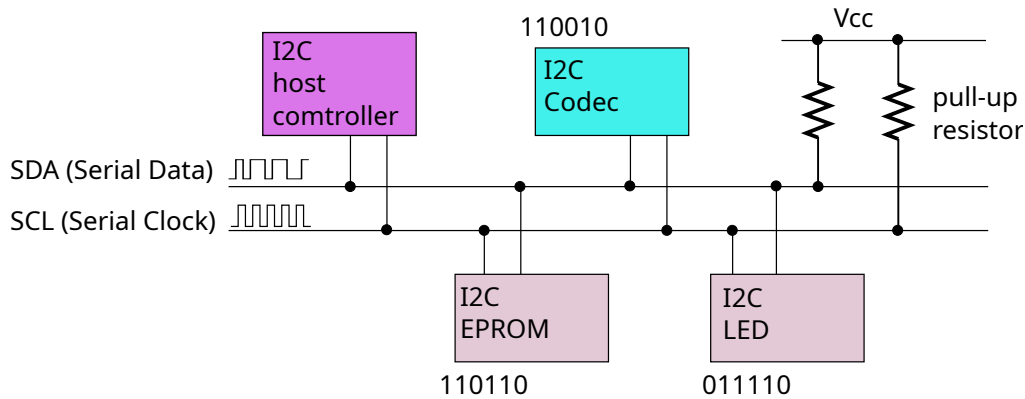


Figure 3.6: Generalized I2C Connection Diagram

- **SCL (Serial Clock):** The clock signal line generated by the active master device to synchronize data sampling along the bus.

The foundational timing characteristics and signaling constraints of the I²C protocol are illustrated in Figure 3.7.

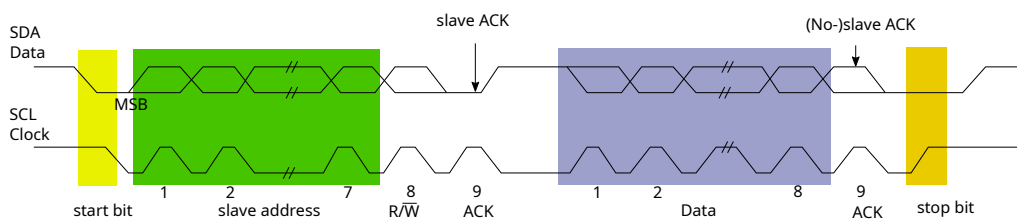


Figure 3.7: Standard I2C Bus Signaling and Timing Constraints

Depending on your distribution's default kernel configuration, you may occasionally need to manually insert the underlying hardware driver modules into the kernel space:

```
# modprobe i2c_dev
# modprobe i2c_bcm2835
```

The physical I²C bus interface exposed on the Raspberry Pi's external connector maps internally to the system device node `i2c-1`. On the physical header, Pin 3 (GPIO2) serves as the serial data line (SDA), and Pin 5 (GPIO3) serves as the serial clock line (SCL). You can verify the presence and permissions of the corresponding device driver file within the system device directory:

```
# ls -l /dev/i2c-1
crw----- 1 root  root  89,   1 Jan 24  2021 /dev/i2c-1
```

The `i2cdetect` program will probe all the addresses on a bus, and report whether any devices are present.

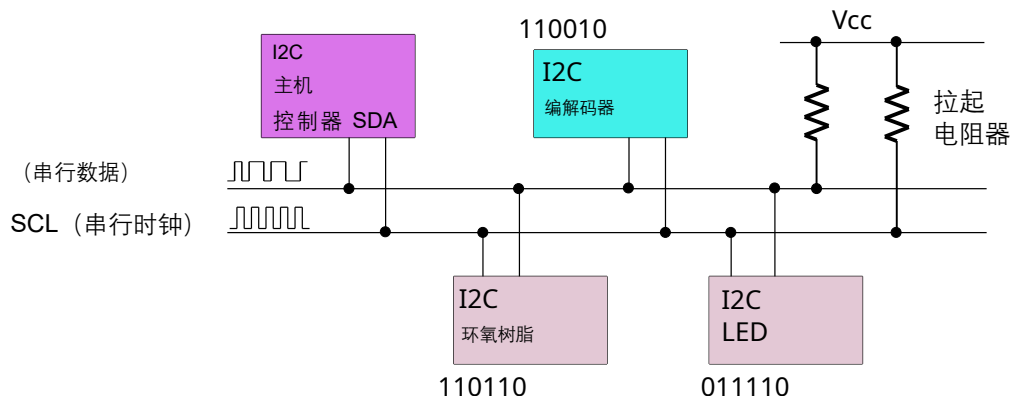


图 3.6: 广义 I2C 连接示意图

- **SCL（串行时钟）**：由主设备生成的时钟信号线，用于同步总线上的数据采样。

I²C 协议的基本时序特性及信号约束如图 3.7 所示。

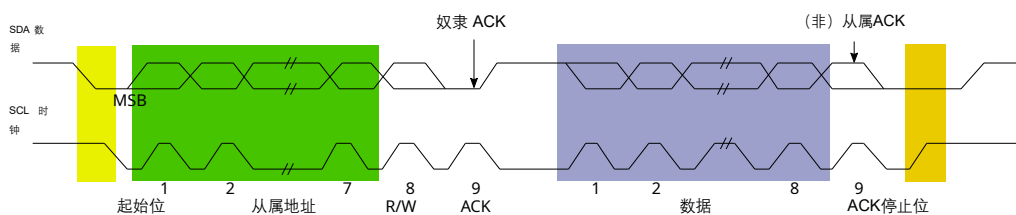


图 3.7: 标准 I2C 总线信号传输及时序约束

根据您所使用的发行版默认内核配置的不同，有时可能需要手动将底层硬件驱动模块插入内核空间：

```
# modprobe i2c_dev
# modprobe i2c_bcm2835
```

树莓派外部接口上暴露的物理 I²C 总线接口在系统内部对应设备节点 i2c-1。在物理引脚布局中，第 3 引脚（GPIO2）作为串行数据线（SDA），第 5 引脚（GPIO3）作为串行时钟线（SCL）。您可通过系统设备目录中的相应设备驱动程序文件来验证其存在状态及权限设置：

```
# ls -l /dev/i2c-1
crw----- 1 根目录 89,2021年1月24日 /dev/i2c-1
```

i2cdetect 程序将探测总线上的所有地址，并报告是否存在相关设备。

```
# i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- 24 25 26 27 -- -- -- -- -- -- -- -- --
30: -- -- -- -- 34 35 36 37 -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

In this setup, each connected 7-segment LED display module is configured with a unique hardware slave address on the I²C bus (typically mapping to addresses 0x34, 0x35, 0x36, and 0x37, respectively). By executing targeted block write operations to a specific slave address, user applications can selectively illuminate the display’s segments to output numbers or characters.

The option “-y” will disable interactive mode, the “1” indicates that we are scanning for I2C devices on I2C bus 1 (e.g. /dev/i2c-1).

You may write a byte 0x3f to I2C register 0x34 with `i2cset`:

```
# i2cset -y 1 0x34 0x3f
```

`i2cset`, `i2cget`, etc. are useful I2C commands that manipulate I2C devices. Examples:

- Write the value 0x5000 to a 16-bit register 0x20 to a device on at 7-bit I2C address 0x68 on bus 1 with no user confirmation:

```
# i2cset -y 1 0x68 0x20 0x5000 w
```

Warning: `i2cset` can be dangerous if used improperly. It can confuse your I2C bus, cause data loss, or have more serious side effects.

- Read the content of an I2C device at 7-bit address 0x68 on bus 1 from an 8-bit register at 0x20 after user confirmation:

```
# i2cget 1 0x68 0x20
```

Warning: `i2cget` can be dangerous if used improperly. Some chips can process a I2C read command as a write command and write incorrect data to a register.

- Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using the default read byte mode with a user confirmation:

```
# i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- 24 25 26 27 -- -- -- -- -- -- --
30: -- -- -- -- 34 35 36 37 -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

在此配置中，每个连接的7段LED显示模块均在I²C总线上被分配一个唯一的硬件从属地址（通常分别对应地址0x34、0x35、0x36和0x37）。通过向特定从属地址执行定向块写入操作，用户应用程序可选择性点亮显示屏的各段以显示数字或字符。

“-y”选项将禁用交互模式；“1”表示我们正在扫描I2C总线1上的I2C设备（例如/dev/i2c-1）。

您可使用 i2cset 命令将字节值 0x3f 写入 I2C 寄存器 0x34：

```
# i2c 设置 -y 1 0x34 0x3f
```

i2cset、i2cget等是用于操作I2C设备的实用命令。例如：

- 无需用户确认，即可将值0x5000写入总线1上、7位I2C地址为0x68的设备中16位寄存器0x20：

```
# i2cset -y 1 0x68 0x20 0x5000 w
```

警告：若使用不当，i2cset 可能存在安全隐患。它可能干扰您的 I2C 总线、导致数据丢失或引发更严重的副作用。

- 用户确认后，从总线1上地址为0x20的8位寄存器中读取I2C设备在地址0x68处的7位内容：

```
# i2c获取 1 0x68 0x20
```

警告：若使用不当，i2cget 可能存在安全隐患。某些芯片可能将 I2C 读取命令误判为写入命令，并向寄存器写入错误数据。

- 使用默认读取字节模式，并经用户确认，将总线1上7位地址0x68处的I2C设备内容进行写入。

```
# i2cdump 1 0x68
```

- Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using I2C block read transactions without user confirmation:

```
# i2cdump -y 1 0x68 i
```

Warning: `i2cdump` can be dangerous if used improperly. Most notably, the `c` mode starts with WRITING a byte to the chip. **Do not use `i2cdump` on random addresses.**

- On bus 0, from an EEPROM at address 0x50, read 8 bytes from offset 0x64 (first message writes one byte to set the memory pointer to 0x64, second message reads from the same chip):

```
# i2ctransfer 0 w1@0x50 0x64 r8
```

The description blocks are composed as `{r|wn}[address]`. `{r|w}` specifies if the message is read or write, `n` specifies the number of bytes read or written in this message. It is parsed as an unsigned 16-bit integer. `[address]` specifies the 7-bit address of the chip to be accessed for this message, and is an integer.

Warning: `i2ctransfer` can be dangerous if used improperly. Some chips can process a I2C read command as a write command and write incorrect data to a register.

One of the premier Python modules available for I2C to interact with this bus is `smbus2`. You can install this module on your system using the `pip` package manager:

```
# pip install smbus2
```

With `smbus2`, you can easily handle I2C devices as following:

```
from smbus2 import SMBus, i2c_msg
# Open a handle to "/dev/i2c-1", representing the I2C bus.
bus = SMBus(1)
# or
bus = SMBus();
bus.open(1)
# or
bus.open('/dev/i2c-1')

# Write a single byte to a device (addr)
```

```
# i2cdump 1 0x68
```

- 使用I2C块读取事务将总线1上7位地址0x68处的I2C设备内容直接写入，无需用户确认：

```
# i2cdump -y 1 0x68 i
```

警告：如果使用不当，i2cdump 可能存在安全隐患。尤其需要注意的是，“c”模式会先向芯片写入一个字节。**请勿在随机地址上使用 i2cdump。**

- 在总线0上，从地址0x50的 EEPROM 处读取偏移量0x64处的8个字节（第一条消息写入1个字节将内存指针设置为0x64，第二条消息从同一芯片读取数据）：

```
# i2transfer 0 w1@0x50 0x64 r8
```

描述块的格式为 {r|wn}[地址]。其中 {r|w} 表示消息是读取还是写入操作；*n*表示该消息中读取或写入的字节数（以无符号16位整数形式解析）；[地址] 则指定用于该消息访问的芯片的7位地址（为整数值）。

警告：若使用不当，i2ctransfer 可能存在安全隐患。部分芯片可能将 I2C 读取命令误判为写入命令，并向寄存器写入错误数据。

smbus2是目前可用于I2C总线通信的主流Python模块之一。您可通过pip包管理器将其安装到系统中：

```
# 安装 smbus2（使用 pip）
```

使用 smbus2，您可以轻松地对 I2C 设备进行如下操作：

```
from smbus2 import SMBus, i2c_msg

# 打开路径 "/dev/i2c - 1"，表示I2C总线。bus = SMBus(1)

# 或
总线 = SMBus ();
总线。开放(1)

# 或
总线。打开( '/dev/i2c -1 ' )

# 向设备写入单个字节（添加r）
```



```

bus.write_byte(addr, byte, force=None)

# Write a byte to a specified register
bus.write_byte_data(addr, register, value, force=None)

# Write a block of byte data to a specified register.
bus.write_block_data(addr, register, data, force=None)

# Write a block of byte data to a specified register.
bus.write_i2c_block_data(addr, register, data, force=None)

# Read a single byte from a device (addr)
byte = bus.read_byte(addr, force=None)

# Read a block of up to 32-bytes from a specified register.
data = bus.read_block_data(addr, register, force=None)

# Read a single byte from a designated register.
data = bus.read_byte_data(addr, register, force=None)

# Read a block of byte data from a specified register.
data = bus.read_i2c_block_data(addr, register, length, force=None)

'''
i2c_rdwr(*i2c_msgs) Combines a series of i2c read and write
operations in a single transaction (with repeated start bits
but no-stop bits in between). This method takes i2c_msg
instances as input, which must be created first with
i2c_msg.read() or i2c_msg.write()
'''
command = [0x11, 0x22, 0x33, 0x44, 0x55]
write = i2c_msg.write(address, command)
bus.i2c_rdwr(write)

write = i2c_msg.write(address, [0xA1, offset])
read = i2c_msg.read(address, length)
bus.i2c_rdwr(write, read)          # Data read in list 'read'

```

总线。写入字节（地址，字节值，强制参数=无）

将一个字节写入指定寄存器

总线：写入字节数据（地址、寄存器、值，强制设置=无）

将一段字节数据写入指定寄存器。

总线：写入数据块（地址、寄存器、数据，强制参数=无）

将一段字节数据写入指定寄存器。

总线：写入 `_i2c_block_data`（地址、寄存器、数据、强制设置=无）

从设备读取单个字节（添加 `r`）

字节 = 总线读取字节（地址，强制=无）

从指定寄存器读取最多32字节的数据。data = 总线。read_block_data(addr, register, force = None)

从指定寄存器读取单个字节。

数据 = 总线读取字节数据（地址，寄存器，强制设置 = 无）

从指定寄存器读取一段字节数据。

数据 = 总线。read_i2c_block_data（地址，寄存器，长度，强制参数=无）

'''

*i2c_rd(*i2c_msgs)* 将一系列i2c读写操作整合为单个事务（包含重复的起始位）。

（但中间不包含停顿位）。该方法以 i2c_msg 实例作为输入，这些实例必须

先通过 i2c_msg.read() 或 i2c_msg.write() 方法创建。

'''

命令 = [0x11, 0x22, 0x33, 0x44, 0x55]

写入 = i2c_msg.write（地址，命令）

总线。I2C_rdwr（写入）

写入 = i2c_msg.write(地址, [0xA1, 偏移量])

读取 = i2c_msg.read（地址，长度）

总线：I2C_rdwr（写入、读取）

在列表 "read" 中读取数据

3.7 Addressable LED Strip WS2812

The WS2812 (often referred to as NeoPixel in the Adafruit ecosystem) is an addressable, smart RGB LED source. Unlike traditional RGB LED strips where the entire strip changes color uniformly, every single LED on a WS2812 strip can be controlled individually and independently.

Key Technical Features:

- **Integrated Control Circuit:** Inside each 5050 (5.0mm × 5.0mm) SMD LED (Surface Mount Device Light Emitting Diode) package, there is a built-in constant-current driver IC along with high-brightness Red, Green, and Blue LED chips.
- **Single-Wire Communication:** It uses a specialized, time-critical single-line NRZ (Non-Return-to-Zero) serial protocol. Because data is passed sequentially from one pixel to the next, an entire strip of hundreds of LEDs can be driven using just one digital GPIO pin from a microcontroller or single-board computer (like a Raspberry Pi or Arduino).
- **Cascading Architecture:** Each pixel acts as a data repeater. When a pixel receives a stream of data, it strips off the first 24 bits (8 bits each for R, G, and B) for its own color configuration, reshapes the remaining data signals, and passes them along via its Data Out (DO) pin to the Data In (DI) pin of the next pixel in line.

A typical addressable LED strip implementation is illustrated in Figure 3.8. This type of LED strip exposes three physical interfacing terminals: Power (+5V), Ground (GND), and Serial Data Input (DI).

Data is clocked sequentially into the DI terminal of the primary LED pixel. The internal control logic of the first pixel intercepts the initial 24 bits of the incoming serial data stream to update its local RGB color state. It then automatically reshapes the remaining data stream and routes it via its Data Output (DO) pin to the DI terminal of the subsequent cascading pixel. Through this daisy-chained cascade architecture, an application can achieve independent color and intensity control over every individual LED on the strip.

The protocol's timing constraints are exceptionally stringent. For example, transmitting a logical bit '1' requires a high-level voltage pulse spanning 700ns followed by a low-level period of 600ns. Conversely, a logical bit '0' requires a high-level pulse spanning 350ns followed by a low-level period of 800ns. The permissible timing tolerance for these waveform windows is narrow ($\pm 150\text{ns}$).

3.7 可寻址LED灯带 WS2812

WS2812（在Adafruit生态系统中常被称为NeoPixel）是一款可寻址的智能RGB LED光源。与传统RGB LED灯带需整体均匀变色不同，WS2812灯带上的每个LED均可独立控制。

关键技术特性：

- 集成控制电路：每个5050（5.0毫米×5.0毫米）SMD LED（表面贴装发光二极管）封装内部均内置恒流驱动IC以及高亮度红、绿、蓝三色LED芯片。
- 单线通信：该技术采用一种专用的、对时间要求严格的单线 NRZ（非归零）串行协议。由于数据按顺序从一个像素传递至下一个像素，因此仅需微控制器或单板计算机（如RaspberryPi或Arduino）的一个数字 GPIO 引脚即可驱动整条由数百个LED组成的灯带。
- 级联架构：每个像素均作为数据中继器。当像素接收到数据流时，会提取前24位（红、绿、蓝各8位）用于确定自身颜色配置，对剩余数据信号进行重构处理，并通过其数据输出（DO）引脚将数据传递至相邻像素的数据输入（DI）引脚。

典型的可寻址LED灯带实施方案如图3.8所示。此类LED灯带配备三个物理接口端子：电源（+5V）、地线（GND）和串行数据输入（DI）。

数据以顺序方式输入至主LED像素的DI端口。首个像素的内部控制逻辑会截取传入串行数据流的前24位，用于更新其本地RGB颜色状态；随后自动重新处理剩余数据流，并通过数据输出（DO）引脚将其传输至后续级联像素的DI端口。借助这种菊花链式级联架构，应用程序可实现对灯带上每个独立LED的颜色与亮度进行独立控制。

该协议的时间约束条件极为严格。例如，传输逻辑位“1”需要一个持续时间为700纳秒的高电平脉冲，随后是600纳秒的低电平阶段；相反，传输逻辑位“0”则需要一个持续时间为350纳秒的高电平脉冲，随后是800纳秒的低电平阶段。这些波形窗口的允许时间容差非常狭窄（±150纳秒）。

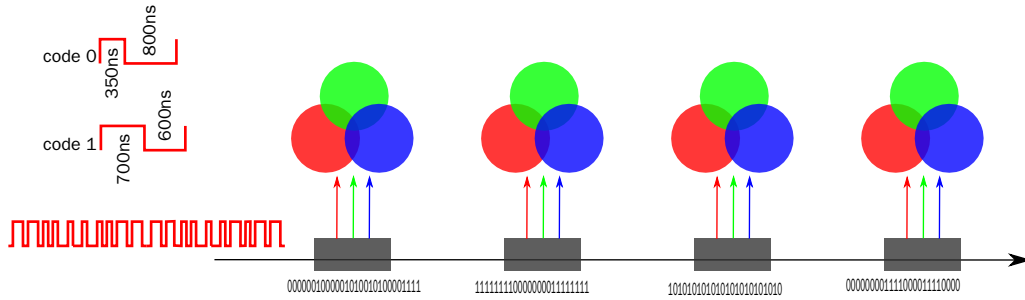


Figure 3.8: WS2812 Addressable LED Strip

Because the Linux operating system operates as a preemptive, multi-tasking time-sharing system, standard software-driven GPIO bit-banging cannot guarantee such high microsecond-level real-time deterministic accuracy. Consequently, specialized hardware blocks on the Broadcom SoC must be leveraged to synthesize the required waveforms. In practice, this is typically achieved by utilizing either the hardware **Serial Peripheral Interface** (SPI) bus serializer or the PWM controller backed by direct memory access (DMA) transfers.

To control the LED strip, we need to install `rpi_ws281x` module:

```
# git clone https://github.com/yfang1644/rpi_ws281x
# cd rpi_ws281x/python
# python3 setup.py build
# python3 setup.py install
```

This module provides both C and Python libraries. It supports the use of PWM, PCM, and SPI; however, only PWM allows for the simultaneous and independent control of two LED strips. The pin assignments are listed in Table 3.2. GPIO pins with numbers higher than 27 are not exposed on the Raspberry Pi board. Example code can be found in the `'python/examples/'` directory.

Table 3.2: Supported GPIO Pins on Raspberrypi

Function	GPIO
PWM 0	12, 18, 40, 52
PWM 1	13, 19, 41, 53, 45
PCM	21, 31
SPI	10, 38

```
1 from neopixel import *
```

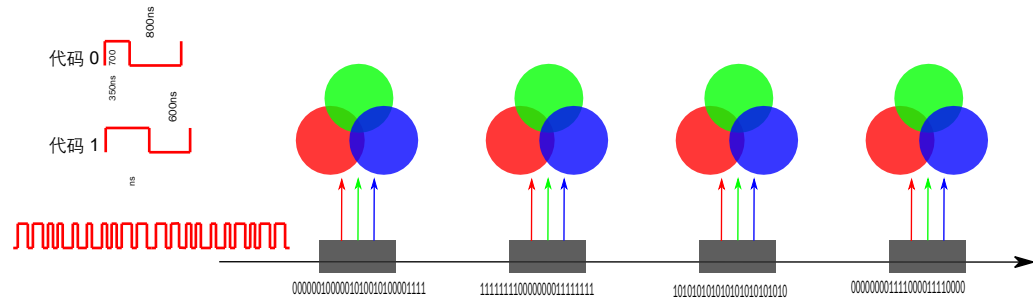


图 3.8: WS2812 可寻址 LED 灯带

由于Linux操作系统采用抢占式多任务分时系统架构，标准软件驱动的 GPIO 位同步技术无法保证如此高的微秒级实时确定性精度。因此，必须利用Broadcom系统级芯片（SoC）中的专用硬件模块来生成所需的波形信号。实际应用中，通常通过使用硬件串行外设接口（SPI）总线序列化器或基于直接内存访问（DMA）传输的 PWM 控制器来实现这一目标。

要控制LED灯带，需安装 rpi_ws281x 模块：

```
# 使用git克隆https://github.com/yfang1644/ rpi_ws281x
# CD rpi_ws281x /python
# 运行 `python3setup.py` 命令进行安装
# 在 Python 3 中运行以下命令： `setup.py install`
```

本模块同时提供C语言和Python语言的库支持。它兼容 PWM 、PCM及SPI接口，但仅 PWM 接口支持对两根LED灯带进行同步且独立的控制。引脚分配详见表3.2。树莓派板上未提供编号大于27的 GPIO 引脚。示例代码可参见 `python /examples/` 目录。

表 3.2: Raspberrypi 上支持的 GPIO 引脚

函数	通用输入输出
PWM 0	12、18、40、52
PWM 1	13、19、41、53、45
PCM	21、31
SPI	10、38

```
2
3 # LED strip configuration:
4 LED_COUNT      = 16      # Number of LEDs in the strip
5 LED_PIN        = 18      # GPIO pin
6 LED_FREQ_HZ    = 800000  # Digital Frequency(800kHz)
7 LED_DMA        = 10      # DMA channel
8 LED_BRIGHTNESS = 255     # Brightness
9 LED_INVERT     = False    # Polar invert
10 LED_CHANNEL    = 0       # PWM Channel 1
11 LED_STRIP      = ws.WS2812_STRIP
12
13 # create NeoPixel object
14 strip = Adafruit_NeoPixel(
15     LED_COUNT,
16     LED_PIN,
17     LED_COUNT,
18     LED_PIN,
19     LED_FREQ_HZ,
20     LED_DMA,
21     LED_INVERT,
22     LED_BRIGHTNESS,
23     LED_CHANNEL
24 )
25
26 strip.begin()      # Module initialize
27
28 # red, green and blue each 8 bits
29 red, green, blue = 200, 100, 150
30 color = (red << 16)|(green << 8)|blue
31
32 # Set colors for the first 4 LEDs
33 for x in range(4):
34     strip.setPixelColor(x, color)
35
36 strip.show()
```

```

2
3 #LED灯带配置:
4 LED计数          = 16          # 条灯带中的LED数量
5 LED_PIN          = 18          # GPIO 图钉
6 LED_FREQ_HZ      = 800000     # 数字频率 (800 kHz)
7 LED_DMA          = 10          # DMA通道
8 LED亮度 = 255          #亮度
9 LED反相器        = 否          #极性反转
10 LED通道          = 0          #PWM 第I 频道
11 LED灯带          = 西部 WS2812_STRIP
12
13 # 创建 NeoPixel 对象
14 条条带 = Adafruit_NeoPixel (
15             LED数量:
16             LED_PIN ,
17             LED数量:
18             LED_PIN ,
19             LED_FREQ_HZ ,
20             LED_DMA ,
21             LED反相器;
22             LED亮度:
23             LED通道
24             )
25
26 条条带. 开始 ( )          # 模块初始化
27
28 # 红色、绿色和蓝色各占用 8 位
29 红色、绿色、蓝色 = 200、100、150
30 种颜色 = (红色 << 16) | (绿色 << 8) | 蓝色
31
32 # 为前 4 个LED 设置颜色
33 对于范围 (4) 内的x:
34     条形. 设置像素颜色 (x, 颜色)
35
36 条形图. 显示()

```